

VECTOR

Using APL with COM and OCX ...

- Thomson on Numeric Methods 33
- Holmes on Composite Arithmetic 49
- APL meets MAPI (Manktelow) 70
- FlexGrid with APL+Win (Barman) 77
- FlexGrid with Dyalog APL (Sandles) 93
- DDE in APL+Win (Askoolum) 111



*The Journal of the
British APL Association*

A Specialist Group of the British Computer Society

ISSN 0955-1433

www.vector.org.uk

Vol.15 No.3 January 1999

Contributions

All contributions to VECTOR may be sent to the Journal Editor at the address on the inside back cover. Letters and articles are welcome on any topic of interest to the APL community. These do not need to be limited to APL themes, nor must they be supportive of the language. Articles should be accompanied by as much visual material as possible (b/w or colour prints welcome). Unless otherwise specified, each item will be considered for publication as a personal statement by the author. The Editor accepts no responsibility for the contents of sustaining members' news, or advertising.

Please supply as much material as possible in machine-readable form, ideally as a simple ASCII text file on an IBM PC compatible diskette (any format). APL code can be accepted as camera-ready copy, in workspaces from I-APL, APL+Win, IBM APL2/PC or Dyalog APL/W, or in documents from Windows Write (use the APL2741 TrueType font, available free from Vector Production), and MS Word.

Except where indicated, items in VECTOR may be freely reprinted with appropriate acknowledgement. Please inform the Editor of your intention to re-use material from VECTOR.

Membership Rates 1998-99

Category	Fee	Vectors	Passes
UK Private	£12	1	1
Overseas Private	£14	1	1
(Supplement for Airmail, not needed for Europe)	£4		
UK Corporate Membership	£100	10	5
Overseas Corporate	£135	10	
Sustaining	£430	10	5
Non-voting Member (Student, OAP, unemployed)	£6	1	1

The membership year normally runs from 1st May to 30th April. Applications for membership should be made to the Administrator using the form on the inside back page of VECTOR. Passes are required for entry to some association events, and for voting at the Annual General Meeting. Applications for student membership will be accepted on a recommendation from the course supervisor. Overseas membership rates cover VECTOR surface mail, and may be paid in sterling, or by Visa, Mastercard or JCB, at the prevailing exchange rate.

Corporate membership is offered to organisations where APL is in professional use. Corporate members receive 10 copies of VECTOR, and are offered group attendance at association meetings. A contact person must be identified for all communications.

Sustaining membership is offered to companies trading in APL products; this is seen as a method of promoting the growth of APL interest and activity. As well as receiving public acknowledgement for their sponsorship, sustaining members receive bulk copies of VECTOR, and are offered news listings in each issue.

Advertising

Advertisements in VECTOR should be submitted in typeset camera-ready format (A4 or A5) with a 20mm blank border after reduction. Illustrations should be photographs (b/w or colour prints) or line drawings. Rates (excl VAT) are £250 per full page, £125 for half-page or less (there is a £75 surcharge per page if spot colour is required).

Deadlines for bookings and copy are given under the Quick Reference Diary. Advertisements should be booked with, and sent to: Gill Smith, Brook House, Gilling East, YORK YO62 4JJ. Tel: 01439-788385 Email: apl385@compuserve.com

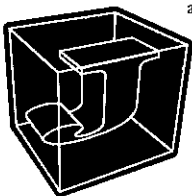
Contents

	Page
Editorial	3
APL NEWS	
Quick Reference Diary	5
Correspondence	7
APL Product Guide	8
	Gill Smith
The Education Vector	
Zark Newsletter Extracts	22
J-ottings 19: Getting to the Root of the Matter	33
Towers of Hanoi Generalized in J	38
	Howard Peelle
GENERAL ARTICLES	
Some Basic Functions for Composite Arithmetic	49
	Neville Holmes
40 Years On	62
	Sylvia Camacho
TECHNICAL SECTION	
Hacker's Corner: APL meets MAPI	70
Using VideoSoft FlexGrid Pro with APL+Win	77
VideoSoft FlexGrid Pro with DyalogAPL	93
	Jon Sandles
At Play with J: Crosswords and Life	99
	Gene McDonnell
More Utilities for the Dyalog Session	107
DDE in APL+Win	111
From Fibonacci to Horodam	122
	Joseph De Kerf
Index to Advertisers	131



IT'S SURPRISING
HOW JSOFTWARE
MAKES YOUR DATA
JUMP OUT!

J for Windows 95/NT from Iverson Software Inc. provides everything you need to solve problems quickly. You can build complete stand alone Windows applications, or easily integrate J with other programs. For example, use J as an ActiveX control for a web page, an OLE server for Excel,



J in a Box

or a 32-bit DLL called from a C program. J is also available on a variety of other platforms including Windows CE, Mac, Linux, Solaris and AIX. The language is identical in all versions, and programs not making use of platform-specific features will work unchanged on all systems.

Strand Software 19235 Covington Court Shorewood, MN 55331
Tel: (612) 470-7345 Fax: (612) 470-9202
www.jsoftware.com

Editorial

by Stefano Lanzavecchia

"I'm dreaming of a White Christmas"

Irving Berlin

"Dreaming I was dreaming"

Namie Amuro

The Internet is like a dream. Allow me to show you the reason why.

If you have ever paid attention to any of your dreams, you are probably aware of the fact that they are not simply composed of images. Every single experience during a dream is a complex conglomerate of sensations, which are layered and intermixed. Think of them a cloud, the way you look at it when awake: a cloud in the sky is no more than a patch of steam with an irregular shape. With a little bit of fantasy you could recognise the profile of somebody in the shape. Maybe the wind carries a particular smell but it is quite distinguished from the cloud itself. Even rain or a lightning flash would not normally be considered parts of the cloud. And that is it.

A cloud in a dream is a completely different matter. Not only has it a colour and a shape, but it also has a taste [1], a scent and a consistency under your fingers, despite the fact that it could be miles away. It is almost impossible to analyse in detail the sensations transmitted during a dream because the "meaning" is not conveyed by one of them but by them all together as a unit. Now, if you try to reproduce in a conventional book an experience like the one described, you will have to work quite hard with the language, struggle with the words and force the syntax to its limits [2]. In fact even a non dream-like description can be very poor and sterile when matched with reality, because of the strictly mono-layered medium used.

The Internet opens up a whole new set of possibilities. First, along with the text, colours, images, icons, and sounds can be transmitted to make the experience richer. However, there is more: since ideas can be linked together (namely through the use of hyperlinks), the reader gets the chance to walk in an orthogonal fashion through the content-rich pages. An extra dimension is added by the fact that one author can devise a texture of links that can point anywhere, not just to pages of the same work, but to pages of different works as well as to the work of other authors. Even in the case of the total absence of images or

sounds, links can be used to unfold multiple layers of meaning for a single word or a thought. The choice of which links to follow and to what depth is up to the reader. Because of this, the reader can easily find himself following a path which will lead him into unexpected areas, just as in a dream, where the logic of the events is somehow disrupted by the continuous change in focus of the interest of the dreamer. Our cloud could take us to a site devoted to recipes for ice-creams, linked to another site about the history of China, in turn linked to a site maintained by a fan of traditional Japanese board games. Here we are, in front of a chessboard, after a trip, which took us through countries and centuries.

The surprising thing is that the trip was not linear and logical, the way reality is usually described, but jump-like and emotion-driven, the way we are used to in our dreams. With a few mouse-clicks we can quickly walk our way back to the point from where we started, and begin a new journey with the same edge conditions but different goals in mind. Science and magic, poetry and gossip, knowledge and entertainment, cooking and space travel, side by side, no walls, only fantasy, *esprit* and time to limit the exploration.

References

- [1] *Earth is Room Enough*, Isaac Asimov.
 - [2] *The Unconsoled*, Kazuo Ishiguro
-

Quick Reference Diary 1998-2000

Date	Venue	Event
August 10-14 1999	Scranton, Penn.	APL99
Late July 2000	Berlin	APL'00

The APL99 Conference is sponsored by SIGAPL and NYSIGAPL and is now in cooperation with ACM/SIGPLAN. The scope of the conference will be extended to include all array-processing languages including Mathematica, MATLAB, Array Based Fortran, SISAL and SAC. SIGPLAN members will be able to attend the conference at member rates. We are grateful to acknowledge the Arthur J. Kania School of Management at the University of Scranton for providing the conference facilities.

An electronic mailing list for information about APL99 has been set up.
Subscription information is available on the web site

<http://www.acm.org/sigapl/apl99.htm>

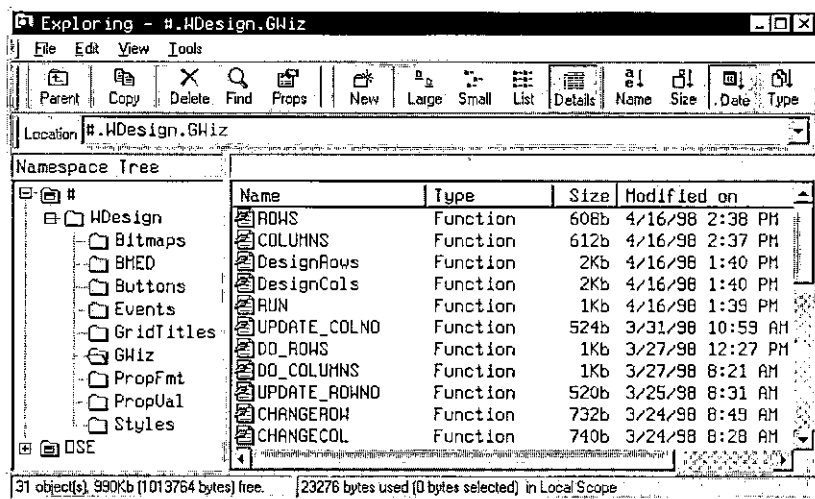
We are tentatively planning to schedule pre-conference tutorials on Tuesday, August 10, 1999.

Dates for Future Issues of VECTOR

	Vol.15 No.4	Vol.16 No.1	Vol.16 No.2
Copy date	5th March	4th June	10th September
Ad booking	12th March	11th June	17th September
Ad Copy	19th March	18th June	24th September
Distribution	April 99	July 99	October 99

dyalog APL

The Definitive APL for Windows™



Dyalog APL Namespaces let you ...

- ⇒ Organise your workspace into **self-contained** sub-systems
- ⇒ **Encapsulate** functions and variables *within* GUI objects
- ⇒ **Isolate** utilities from your application code
- ⇒ Construct **COM/DCOM** object hierarchies
- ⇒ Support **multiple instances** in APL web server applications

That's why Dyalog APL/W remains the **professional** choice. For further information, contact Dyadic or your local distributor today.

Dyadic Systems Limited, Riverside View, Basing Road, Old Basing,
Basingstoke, Hants. RG24 7AL, United Kingdom.
Tel:+44 1256 811125 Fax:+44 1256 811130 Email: sales@dyadic.com

Microsoft is a registered trademark and Windows and the Windows Logo are trademarks of Microsoft Corporation



CORRESPONDENCE

From: Eugene McDonnell

Dec 1998

Your article on Dyalog APL/W 8.2 had an example on p.26 of Vector 15.2. It occurred to me that the bordering could be done in two steps, possibly with a slight saving in time. I wrote them in J and my hunch was right:

```
f=: 13 : '(3+/\3+/\ "1(0.,(0,y.,0),.0))-y.'
g=: 13 : '(3+/\ "1(0.,(3+/\0,y.,0),.0))-y.'
```

Functions *f* and *g* give identical results on boolean matrices. The function *f* is the same as your APL expression, where a complete border of zeros surrounds the matrix before any reductions occur. The function *g* puts rows of zeros on top and bottom only, inserts + between the rows, then puts columns of zeros on the left and right of this temporary result, and inserts + between the columns. Timings give *g* a 20% edge over *f* on small matrices such as one encounters in Life games.

A very small point probably not worth worrying about, but when one has learned to program in the 1950s, like me, the habit of looking for any small edge is hard to forsake.

Vector Back Numbers

Back numbers of Vector are available from:

British APL Association,
c/o Gill Smith,
Brook House, Gilling East,
YORK YO62 4JJ

Price in UK: £10 per complete volume (4 issues);
£12 (overseas); £16 (airmail) including postage.

The Vector Product Guide

compiled by Gill Smith

VECTOR's exclusive Product Guide aims to provide readers with useful information about sources of APL hardware, software and services. We welcome any comments readers may have on its usefulness and any suggestions for improvements.

We reserve the right to edit material supplied for reasons of space or to ensure a fair market coverage. The listings are not restricted to UK companies and international suppliers are welcome to take advantage of these pages.

For convenience to readers, the product list has been divided into the following groups ('poa' indicates 'price on application'):

- Complete Systems (Hardware & Software)
- APL and J Interpreters
- APL-based Packages
- Consultancy
- Other Products
- Overseas Associations
- Vendor Addresses
- World Wide Web and FTP Sites

Every effort has been made to avoid errors in these listings but no responsibility can be taken by the working group for mistakes or omissions.

We also welcome information on APL clubs and groups throughout the world.

Your listing here is absolutely free, will be updated on request, and is also carried on the Vector web site, with a hotlink to your own site. It is the most complete and most used APL address book in the world.

Please help us keep it up to date!

All contributions and updates to the Vector Product Guide should be sent to Gill Smith, at Brook House, Gilling East, York, YO6 4JJ. Tel: 01439-788385, Email: apl385@compuserve.com

COMPLETE APL SYSTEMS

COMPANY	PRODUCT	PRICES(£)	DETAILS
Dyadic	IBM RS/6000 MD320	11,736	APL POWERstation (Greyscale) 27.5 MIPS, 7.4 Mflops RISC Processor 8Mb RAM, 120Mb Disk 19" 1280x1024 Greyscale Graph Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD320	13,817	APL POWERstation (Colour) 27.5 MIPS, 7.4 Mflops RISC Processor 8Mb RAM, 120Mb Disk 16" 1280x1024 Colour Graphics Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD320	22,858	Advanced APL POWERstation 27.5 MIPS, 7.4 Mflops RISC Processor 16Mb RAM, 320Mb Disk, 150Mb Tape 16" 1280x1024 Colour Graphics Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD520	37,114	APL POWERsystem (8-users) 27.5 MIPS, 7.4 Mflops RISC Processor 16Mb RAM, 320Mb Disk, 150Mb Tape CD-ROM Drive, 16 Ports AIX, Dyalog APL (2-8 user licence)
	IBM RS/6000 MD530	72,054	APL POWERsystem (16-users) 34.5 MIPS, 10.9 Mflops RISC Processor 32Mb RAM, 1.34Gb Disk, 2.3Gb Tape CD-ROM Drive, 16 Ports AIX, Dyalog APL (8+ user licence)
	IBM RS/6000 MD540	122,842	APL POWERsystem (32-users) 41 MIPS, 13 Mflops RISC Processor 64Mb RAM, 1.7Gb Disk, 2.3Gb Tape CD-ROM Drive, 32 Ports AIX, Dyalog APL (8+ user licence)
Optima	IBM Compatible	poa	Complete networked or stand-alone solutions including configuration installation, maintenance and commissioning.

APL INTERPRETERS

COMPANY	PRODUCT	PRICES(£)	DETAILS
APL Software	APL*Plus/PC Release 10	450	STSC's APL for IBM PCs & compatibles. Upgrades from earlier releases also available.
	Run-time	poa	Closed version of APL*Plus/PC which prevents user exposure to APL.
	APL*Plus II	1,395	All the features of mainframe APL*Plus for your 386PC!
	Run-time Dyalog APL	poa	2nd generation APL for Unix systems
	APL2/PC	poa	IBM's APL 2 for the PC.
Beautiful Systems	Dyalog APL/W for Windows	poa	US Distributor of Dyalog APL products from Dyadic.
	Dyalog APL for Unix	poa	See Dyadic listing for product details.
The Bloomsbury Software Company	APL+PC Version 11	250	Upgrade to version 11 gives free runtime (£120 from any version)
	APL+Win v3.0	1350	A 32-bit Windows-hosted Interpreter that runs under all Windows platforms including Windows 95. Note: Upgrade for £350 from version 1.8 or 2, £920 from version 1.0
	Migration to APL+Win	1060	from APL*PLUS/PC APL*PLUS/DOS any version. from earlier versions of APL*PLUS II
	APL+DOS	1250	APL*PLUS II DOS is renamed to APL+DOS.
	Migration to APL+DOS	760	from APL*PLUS/PC
	APL Link	200	Database Access
	APL Link Pro	500	

	APL*PLUS II for UNIX	poa	APL2000's 2nd generation APL for all major Sparc and Risc Unix workstations.
	APL*PLUS VMS	poa	2nd generation APL for DEC VAX computers under VMS.
	APL*PLUS Mainframe	poa	Enhances VS APL with many high performance, high productivity features. For VM/CMS and MVS/TSO offers simple upgrade from VS APL.
Dinosoft Oy	Dyalog APL/W for Windows	poa	Finnish distributor of Dyalog APL products.
	Dyalog APL for Unix	poa	See Dyadic's listing for product details.
Dyadic	Dyalog APL for DOS/386	995	Second generation APL for DOS. Runs in 32-bit mode, supports very large workspaces. Unique "window-based" APL Development Environment and Screen Manager. Requires 386/486 based PC or PS/2, at least 2Mb RAM, EGA or VGA, DOS 3.3 or later.
	Dyalog APL/W for Windows	995	As above, plus object-based GUI development tools. Requires Windows 3.0 or later.
	Dyalog APL for Unix	995-12,000	Second generation APL for Unix systems. Available for Altos, Apollo, Bull, Dec, HP, IBM 6150, IBM RS/6000, Masscomp, Pyramid, NCR, Sun and Unisys machines, and for PCs and PC/2s running Xenix or AIX. Oracle Interface available for IBM, Sun and Xenix versions.
IAC/Human Interfaces			
	I-APL/Mac	13	Macintosh version of I-APL.
I-APL Ltd	I-APL/PC or clones	8	ISO conforming interpreter. Supplied only with manual (see 'Other Products' for accompanying books).
	I-APL/BBC Master	8	As above
	I-APL/Archimedes	8	As above
	Strand Software Inc		Strand Software Inc has the sole selling rights to Iverson Software Inc products. I-APL stocks a few of these (mainly APLWIN and the personal J products and books), but is no longer an agent.
IBM APL Products	TryAPL2	free	APL2 for educational or demonstration use. Write, fax or Email to APL Products; specify disk size desired.
	APL2 PC (US Version)	\$630	Product No. 5799-PGG. PRPG Number RJ0411. Order from 1-800-IBM-CALL.
	APL2 PC (European Version)	£348	Product No. 5604-260. Part number 39F1753. From all IBM dealers, including MicroAPL.
	APL2 for OS/2 Entry Edition	\$185	Part No 89G1556.
	APL2 for OS/2 Advanced Edition	\$650	Part No 89G1697. Contains all facilities of the Entry Edition plus: DB2 interface; co-operative processing TCP/IP interface; tools for writing APIs; TIME facility.
	APL2 for Windows version 1.0	\$1500	Product No. 5639-d46, part number 4229558.
	APL2 Runtime environment	\$250	Part No. 39L8419 - Packager and runtime CDs. One additional runtime install \$200, 5 additional \$900, 10 \$1,500.
	APL2 for Sun Solaris	\$1500	Product No. 5648-065.
	APL2 for AIX 6000	poa	Product No. 5765-012.
	APL2 Version 2	poa	Product No. 5688-228. Full APL2 system for S/370 and S/390
	APL2 Application Envir Vn2	poa	Product No. 5688-229. Runtime environment for APL2 packages
Insight Systems	APL*PLUS/PC	poa	APL systems marketed and supported ...
	Dyalog APL	poa	from: Dyadic, Manugistics, IBM
	APL2	poa	under: Windows, OS2 and Unix
Iverson Software Inc.	J on the Web online registration ...		
	J Educational Edition	\$50	
	J Standard Edition	\$150	
	J Professional	\$325	
	Books and accessories (discounts for reg users)		

	J Dictionary	\$50	
	J User Manual	\$50	
	J Phrases	\$40	
	J Primer	\$40	
	Set of the above 4 books	\$180	
	Concrete Math	\$40	
	Fractals, Visualization & J	\$50	
	Exploring Math	\$50	
	J User Conference Proceedings	\$35	
	Mugs, T-shirts, Mousepads	\$10 each	
	APLWIN	\$70	For 386/PC under Windows 3.1
J Austria	J	poa	Distributor for Austria and Switzerland
	Dyalog APL	poa	Distributor
	Causeway Products	poa	Distributor
	Structural Analysis Software	poa	Complete package by IG Zenkner&Handel to perform structural analysis/engineering calculations. Also suitable for dynamic problems, e.g. earthquake simulation.
Lescasse Consulting	APL+PC	poa	Lescasse Consulting is the exclusive APL2000 distributor in France and also
	APL+Unix	poa	distributes in Switzerland and Belgium. Call for price quotes.
	APL+DOS	poa	
	APL+W/n	poa	
	Dyalog APL/W	poa	French distributor for Dyalog
MasterWork Software	Manugistics Products and ISI	poa	New Zealand distributor
MicroAPL	APL.68000 Level I	2000	First generation APL with numerous enhancements. Multi-user version (Unix, Mirage, MCS).
	APL.68000 Level II	2500	Second generation APL. Nested arrays, user defined operators, selective specification etc. Multi-user version (Unix, Mirage, MCS)
	APL.68000/X	1500-6000	Second-generation APL. Nested arrays, user defined operators, selective specification, etc. Multi-user AIX version with full OSF/Motif support.
	APL.68000 Level I Mac, ST, Amiga	87	First generation APL. Single user, full windowing interface, software floating point support.
	Mac, Amiga	260	First generation APL. Single user, full windowing interface, hardware floating point.
	APL.68000 Level II ST	170	Second generation APL. Full windowing interface, software floating point support.
	Amiga	260	Second generation APL. Full windowing interface. Hardware and software floating point support.
	Mac	520	Second generation APL. Full windowing interface. Hardware and software floating point support.
	APL*PLUS Rel 10	450	
	APL*PLUS II V 4.0	1395	
Oasis	Dyalog APL	poa	Dyadic Systems
	APL*PLUS	poa	Manugistics
	APL.88000	poa	MicroAPL Ltd
	APL2	poa	IBM
Optima	Dyalog APL/W	995	Fully fledged Windows development environment.

RE Time Tracker Oy	APL+PC (APL*PLUS/PC)	poa	Complete APL+ and Statgraphics product range and links to various 3rd party products.
	APL+DOS (APL*PLUS II)		
	APL+Win (APL*PLUS III), APL+Link		
	APL+UNIX		
	APL*PLUS Sharefile		
Soliton Associates	SHARP APL for MVS	poa	for IBM MVS mainframes
	SHARP APL for Unix	poa	for IBM RS/6000 and Sun SPARC
Strand Software	Canada		
	All APL*PLUS Products	poa	All APL*PLUS products including upgrades and educational.
	Dyadic and ISI products	poa	
	USA		
	Dyadic and ISI products	poa	

APL PACKAGES

COMPANY	PRODUCT	PRICES (£)	DETAILS
ADAPTA Software	MPS	poa	Master Production Scheduling
	FBS	poa	Forecasting and Budgeting System
	DRP	poa	Distribution Requirements Planning
Adaptable Systems	FLAIR	poa	Finite loader and interactive rescheduler. Customisable full-function scheduling system. (Available outside Australia by special arrangement only.)
Adaytum Software	Adaytum Planning	poa	Full-featured Budgeting and Financial Planning system for medium to large enterprises.
Eventra	Qualedi	\$1500-4000	Electronic Data Interchange (EDI) translation software for the PC, with strict compliance checking.
APL Software Ltd (mainframe)	RDS	poa	Relation Data Base System
	IPLS	poa	Project Management System
	REGGPAK	poa	Regression Analysis Package
(microcomputer)	POWERTOOLS	295	Assembler written replacement function for commonly used CPU-consuming APL functions, includes a Forms Processor.
	REGGPAK	poa	Regression Analysis Package
	RDS	990	Relational Database System
Beautiful Systems	ASF_FILE	\$399	Dyalog APL/W auxiliary processor for access to APL*PLUS/PC APL component files (*.ASF).
	NAT_FILE	\$299	Dyalog APL/W auxiliary processor which emulates the APL*PLUS/PC quad-N native file subsystem for access to the DOS file system.
	DBF_FILE	\$299	Dyalog APL/W auxiliary processor for efficient block mode access to dBASE format files. Designed to get large amounts of data in and out of dBASE. Not suited for random access to small amounts of data (it does not handle keys).
	SF_READ	poa	Dyalog APL/W functions to read APL*PLUS data objects of any type or structure from *.SF style component files created by APL*PLUS II or III.
The Bloomsbury Software Company (for VSAPL)	Enhancements & Sharefile	poa	Component files, quad-functions & nested arrays for VSAPL under VM/CMS & MVS/TSO
	Compiler	poa	The First APL compiler!

(for APL2)	Sharefile/AP	poa	STSC's shared access component file system for APL2. Comparable to all APL*PLUS file systems: multi-user storage of APL2 arrays with efficient disk usage.
Causeway	CausewayPro for Dyalog/W	400/\$600	Causeway application development platform for Dyalog APL/W.
	RainPro Business Graphics	250	The ultimate graphics toolkit for the APL developer. Adds 3D charting capability, Web publishing and clipboard support to the shareware product. Charts can be included in NewLeaf reports. Functionally compatible across Dyalog/W and APL+Win.
	NewLeaf for Dyalog and +Win	400	Frame-based reporting tool with comprehensive table-generation and text-flow support. Offers multiple master-page capability, bitmap wrap-around and on-screen preview with pan and zoom. Fully supported on Dyalog/W and APL+Win (1.8 and above)
Cinerea AB	ORCHART	250	Organization chart package for IBM APL2/PC. Full & heavily commented source code included - free integration into other applications. NB: ASCII output with line-drawing (semi-graphic) characters for boxes.
CODEWORK	HELM	poa	Decision Support System for top management. Handles large multi-dimensional tables, data analysis, EIS presentations, generates HTML and Latex output. Platforms: MS-DOS, LAN, Windows 3.1/95/NT. Ideal for APL customisation (APL*PLUS II and Dyalog APL); more than 150 installed.
H.M.W.	4XTRA	poa	Front-end Foreign Exchange dealing / pos keeping
	Arbitrage	poa	Arbitrage modelling
	Basket	poa	Basket currency modelling
	Menu-Bar	poa	pull-down menu for APL*PLUS/PC
HRH Systems	APL Utilities	poa	Software to transfer workspaces between APL*PLUS and Sharp, and between APL*PLUS and I-APL. Software to import IBM .ATF files to APL*PLUS.
	APL*PLUS Utilities		Public domain software, unlock locked fns, a user-friendly alternative to locking, fns of mathematical physics, menus, and others.
IAC/Human Interfaces	SPARKS	poa	Educational simulation of electric circuit (for Apple Mac.)
	EPIDEMIC	poa	Educational simulation of spreading infection (for Apple Mac.)
	COINS	poa	Educational simulation (KS3) of coin-tossing experiment with simple stats (for Apple Mac.)
	FIBONA	poa	Educational simulation of Fibonacci's rabbits (for Apple Mac.)
I-APL Ltd	Educational workspaces	5	PC format disks with the examples from: Thomson, Espinasse (Kits 1-4), Kromberg, Jizba & FinnAPL. All the examples to save your fingers!
IBM APL Products	A Graphical Statistical System	\$250	for DOS, Product Number 5764-009
	(AGSS)	\$500	for Workstations (OS/2, Aix, Solaris), Product Number 6764-092
		\$2500	for CMS, Product Number 5764-011
INFOSTROY	APL*PLUS/Xbase Interface (II/386 Version 2)	\$198	Complete package written in C. Comparable with the data, index & memo files of FoxPro, dBASE, & Clipper. Multi-user support. No DBMS license required.
	(DLL Version 1)	\$198	The same in a DLL form! Gives your Windows applications all advantages of DLLs.
Insight Systems	IUTILS/XP	20-95	Cross-platform utility library including simple OS calls (DIR, COPY, DEL, RENAME) and DATE functions. For APL*PLUS II, APL2 and Dyalog APL under Windows, OS/2 and Unix.
	ASI	95	APL Spreadsheet Interface. "Device-Independent" spreadsheet driver supporting Excel, 123 and Quattro-Pro for Dyalog APL/W
	WinCom	95	Asynchronous comms package for Dyalog APL/W
	S2D,22D,X2X	poa	Advanced APL syntax analysis and conversion packages from Sharp and APL2 to Dyalog, and between any two APLs

	SQAPL Client	poa	Interface from APL*PLUS II, APL2 and Dyalog (Windows, OS/2 or Unix) to most SQL databases over most networks.
	SQAPL Server	poa	Makes APL*PLUS II, APL2 or Dyalog APL (Unix) available as SequelLink servers. Can be called from SQAPL clients or other applications such as Excel, C++, Smalltalk, Visual Basic.
JAD Software	JAD SMS	150-500	Software management system for APL*PLUS II based on hierarchical databases; Includes full-screen interface and stand-alone functions. Price depends on number of users.
Lescasse Consulting	APL+Win Monthly Training Program	\$800	Download 50+ page document about APL+ programming each month. You also get one or more workspaces full of re-usable APL code and sometimes additional files or products.
	Advanced Windows Programming ...	\$95	200-page book plus companion disk on interfacing APL and Delphi. Contains full coverage of Delphi-2, +Win and Dyalog.
	DLL parser for APL	\$250	Parse any Visual Basic DLL declaration file into a set of quadNA definitions. Turn constants and structures into APL variables. Available for APL+Win and Dyalog/W.
	Delphi Forms Translator	\$195	Design forms with Delphi and turn them automatically into APL programs which recreate the same form (+Win and Dyalog/W).
	APL+Link Pro	poa	ODBC interface for APL+Win
	SQAPL Pro	poa	ODBC interface for Dyalog APL/W
	RainPro	poa	Highly customisable 2D and 3D publication graphics for APL+Win and Dyalog APL/W
	NewLeaf	poa	Page layout and printing tools for APL+Win and Dyalog
	GraphX and ChartFX	poa	High-quality business graphics for APL+Win
	Formula One and Dyalog APL	\$95	100-page book + companion disk on how to use the Formula One VBX with Dyalog APL/W
Lingo Allegro	FRESCO Business Graphics	poa	Fast and easy business graphics DLL
	AP126/PC	poa	GDDM interface for Dyalog APL/W
	AP127/PC	poa	ODBC interface for Dyalog APL/W
	AP119/PC	poa	TCP/IP interface for Dyalog APL/W
	FACS	poa	EMMA-like interface to DB2 or ODBC databases
	ISAP	poa	MS Windows DLL for calling Dyalog APL/W from web pages via MS Internet Information Server. Visit www.lingo.com for a demo.
RE Time Tracker Oy	UIT/W	poa	Comprehensive high-level Windows User Interface library for APL+Win and +II v 5.1. Comprehensive spreadsheets, replicated fields, special field types, etc. 16 and 32 bit versions available.
	AJGRAPH	poa	Graphpak-compatible 2D graphics package for +Win and +DOS. Includes multi-window support, print and metafile support. No DLLs required.
	ECCO PRO with APL	poa	Leading group and personal information management system with comprehensive customising. Supplied with sample +Win workspace to interface to ECCO databases via DDE.
	NEWT TCP/IP SDK with APL	poa	Lead TCP/IP SDK with interfaces to all protocols. Supplied on 3 CD ROMs together with a sample +Win workspace.
	DB+	poa	Database interface for APL+DOS under Windows. Allows combining character-based APL applications with ODBC-compliant databases such as Oracle and SQL-server.
Soliton Associates	LOGOS	poa	Application Development Environment
	MAILBOX	poa	Electronic Mail
	VIEWPOINT	poa	Report generator with interfaces to DB2 and MVS data
Warwick University	BATS	250	Menu driven system for time series analysis and forecasting using Bayesian Dynamic modelling. Price is reduced to £35 for academic institutions.
	FAB	free	Training program for the above.

Zark	APL Tutor (PC)	\$299	APL computer-based training. Available for APL*PLUS PC & APL*PLUS II. Demo disk \$10.
	APL Tutor (MF)	\$5000	Mainframe version.
	Zark ACE	\$99	APL continuing education. APL tutor news and hotline phone support.
	APL Advanced Techniques....	\$59.95	488pp. book, (ISBN 0-9619067-07) Including 2-disk set of utility functions (APL*PLUS PC format).
	Communications	\$200 pc, \$500 mf	Move workspaces or files between APL environments.

APL CONSULTANCY AND DEVELOPMENT

COMPANY	PRODUCT	PRICES (£)	DETAILS
Adfee	Consultancy	poa	Development, maintenance, conversion, migration, documentation, of APL products in all APL environments
Ajay Askoolum	Consultancy	poa	APL+Win development and migration of actuarial, financial, mathematical applications.
Andrews	Consultancy	poa	APL programming and analysis, Year-2000 legacy systems, algorithms, tree-processing.
APL Solutions Inc	Consultancy	poa	APL systems design, development, maintenance, documentation, testing and training. Providing APL solutions since 1969.
AUSCAN Software	Consultancy	poa	APL software development, training
Bloomsbury Software	Consultancy	300-750+VAT	
Camacho	Consultancy	poa	Manuals; feasibility reports and estimates; analysis and programming; APL and MS Windows applications; Sharp, ISI APL, APL*PLUS, APL2/PC and other APLs spoken. Fixed price systems a speciality
Ray Cannon	Consultancy	poa	APL, C, Assembler, Windows, Graphics: PC and mainframe
Causeway	Consultancy and Training	poa	On-site training for Causeway, RainPro and NewLeaf. Customisation and enhancement to meet local needs. Code review and pre-implementation check of Causeway applications.
Paul Chapman	Consultancy	250-500	24-hour programmer: APL, Smalltalk, C; Windows front end design a speciality.
CODEWORK	Consultancy	poa	Development, maintenance, migration, documentation of APL applications. Speciality: Info systems for top executives, Internet applications.
Dinosoft Oy	Consultancy	poa	Specialised in very large databases.
Dyadic	Consultancy	poa	APL and Unix system design, consultancy, programming and training.
Entropy Software	Consultancy	poa	Company reporting, business graphics, Windows applications with Dyalog APL/W
Evestic AB	Consultancy	poa	Excellent track record from 15+ years of APL applications in banking, insurance, and education services. All dialects, platforms and project phases. SQL expertise.
General Software	Consultancy	from 120	
Godin London Inc	Software Development	poa	We have applications in the food manufacturing field, travel agency and airline bookings field and in product lease management.
H.M.W.	Consultancy	poa	System design consultancy, programming. HMW specialize in banking and prototyping work.
Hoekstra Systems Ltd	Consultancy	poa	APL consultancy, programming, etc. Also UNIX system administration
Michael Hughes	Consultancy	poa	Consultant with 10+ years experience with various APL interpreters and C.
IAC/Human Interfaces	Consultancy	poa	APL and ergonomics consultancy services.

	Documentation	100-200	On-line assistance, product demos & mock-ups, manual writing; foreign language software localization.
	Training	poa	Using I-APL for courseware & distance learning materials; Mac programming in C, APL & HyperCard.
INFOSTROY	Consultancy	poa	Moving applications between platforms. Client/server development. Multilingual user interface.
Insight Systems	Consultancy	poa	Experts in APL conversions between any combination of: APL*PLUS, APL2, Dyalog APL and Sharp APL. We are also experienced right-sizers, comfortable with networks and relational databases (that also means when NOT to use SQL) and client/server development in APL, C and Visual Basic.
JAD Software	Consultancy	poa	Systems design and development, project management, technical manuals, financial and actuarial expertise in APL.
Phil Last	Consultancy	poa	APL consultancy, modelling and programming.
Lescasse Consulting	Consultancy	poa	A range of consultants, experts in Windows programming, with APL+Win and Dyalog APL/W. More than 100 major APL applications already developed. We all have additional expertise in Formula One and Delphi.
Lingo Allegro	Consultancy	poa	General APL consulting, internet website development, migration and downsizing, performance tuning, education and training.
Mackay Kinloch Ltd	Consultancy	200-400	Design, analysis and programming for banking, insurance, financial planning and modelling, corporate performance and legal reporting.
MicroAPL	Consultancy	poa	Technical & applications consultancy.
Millinta Inc	Consultancy	poa	Design, development, maintenance, conversion, documentation in all APs, most APs and some specific Sharp products (LOGOS, ViewPoint, Retrieve). Experience in multi-user, multi-task systems, databases, Y2K, Windows programming.
Ellis Morgan	Consultancy	250-500	Business Forecasting & APL Systems.
Oasis	Consultancy	poa	Expertise in APL system design, Project management, conversion, migration, tuning; for all APL versions (10+ years experience)
Object Oriented Ltd	Consultancy	poa	General APL consulting, code recycling — mainframe to PC, performance tuning.
Optima	Consultancy	poa	A range of consultants specialising in all areas of pharmaceutical, industrial and financial systems with 5-15 yrs experience on both PC and mainframe.
RadSys Technologies Consultancy		poa	Areas of expertise: financial systems, risk analysis systems, healthcare systems.
RE Time Tracker Oy	Consultancy	poa	APL application conversions, APL Windows Interfaces, APL to APL-level interfacing to any system under Windows, TCP/IP network and database connectivity.
Rex Swain	Consultancy	poa	Independent consultant, 20 years experience. Custom software development & training, PC and/or mainframe.
Rochester Group	Consultancy	poa	Specialise in MIS using Sharp APL
Shepp & Associates	Consultancy	poa	APL applications development and consulting, especially in the travel industry, especially on small computers. 25 years experience in APL programming.
Snake Island Research Inc	Consultancy	poa	APL interpreter and compiler enhancements, intrinsic functions, performance consulting. APL parallel compiler APEX is giving very good initial performance tests with convolution somewhat faster than FORTRAN.
Strand Software	Consultancy	poa	Advice on migrating to and from all flavours of APL and hardware platforms. Full-screen interface implementation, APL utilities, benchmarking, efficiency analysis, actuarial software, system development tools, valuation, pricing and modelling systems.

Sykes Systems Inc	Consultancy	poa	Complete APL services specialising in audit, optimisation and conversion of APL systems. Excellent design skills. All dialects and platforms. 17-23 years experience.
Stephen Wynn	Consultancy	poa	Most experience of financial planning, and mathematical areas: operational research, quality control, experimental design.

OTHER PRODUCTS

COMPANY	PRODUCT	PRICES(£)	DETAILS
Adfee	Employment	poa	Contractors and permanent employees
APL-385	Typefaces	poa	Variants of the APL2741 typeface available to specification.
Blocmsbury Software	Training	poa	Contact the company for details.
ComLog	Comic-Logger	\$25.95+p&p	APL*PLUS II comic-book inventory system. Shareware version available on America OnLine.
HMW	Employment	poa	Contractors and permanent employees placed.
HRH Systems	APL lessons		On-screen interactive APL lessons for APL*PLUS, TryAPL2, Sharp and I-APL — in English or French.
	The BBS/APL	n/a	<i>The BBS/APL is no longer available. For an email catalogue of its archive files (and how to get them) email dlick.holt@juno.com.</i>
I-APL Ltd	An APL Tutorial	3	45pp by Alvord & Thomson
	An Encyclopaedia of APL (2d Ed)	6	228pp by Helzer
	APL In Social Studies	3	36pp by Traberman
	I-APL Instruction Manual (2d Ed)	3	55pp by Camacho & Ziemann
	APL Programs for the Mathematics Classroom (Springer-Verlag)	16	185pp by Thomson
	Programming In J	10	75pp by Ken Iverson
	Arithmetic	12	118pp by Ken Iverson
	Tangible math	8	36pp by Ken Iverson
	Sharp APL Reference Manual	42	349pp by Berry
	APL Press Books	poa	A comprehensive selection of early APL literature
	<i>Please note there is a packing charge of £3 per order</i>		
Oasis	Training	poa	Introductory courses in APL Advanced courses for different APL versions
Renaissance Data Systems	Booksellers		The widest range of APL books available anywhere. See Vector advertisements.
Sollton Associates	MVS/LINK	poa	Interface from Sharp APL (Unix & MVS) to non-APL data and software in the MVS environment.
	SSQL	poa	High-performance DB2 interface for Sharp APL (Unix and MVS).

OVERSEAS ASSOCIATIONS

GROUP	LOCATION	JOURNAL	OTHER SERVICES	Ann.Sub.
ACM SigAPL	International	APL QuoteQuad	Conferences; APL white pages; web site	\$30
APL Bay Area	USA N. California	APLBUG	Monthly Meetings (2nd Monday)	\$20
APL Club Austria	Austria		Quarterly Meetings	200AS(indiv), 1000AS(corp)
APL Club Germany	Germany	APL Journal	Semi-annual meetings	DM60
Ass. Francophone pour la promotion d'APL France		Les Nouvelles d'APL		FF350 (private) FF2800 (Company)
BACUS	Belgium	APL-CAM	Conferences & Seminars	£18 (\$30)
Capital PCUG	Washington, D.C.	Monitor	Monthly meetings, occasional classes	free
Danish SIG	Denmark			
Dutch APL Assoc.	Holland	Vector provided	Mini-congress, APL ShareWare Initiative	
FinnAPL	Helsinki, Finland	FinnAPL Newsletter	Seminars on APL	100FIM(private), 30(student), 1000 (Co)

Japan APL Assoc	Tokyo	APL Journal	Monthly meetings (4th Sat)	10,000yen to join
Rome/Italy SIG	Roma, Italy			
RusAPL	Moscow, Russia	APL Club	Seminars and Annual Meeting	100,000R (students 20,000)
SE APL Users Grp	Atlanta, Georgia	SEAPL Newsletter	Quarterly meetings	\$10
SovAPL	Obninsk, Russia			
SwedAPL	Sweden	SwedAPL Nytt	Semi-annual meetings, seminars	SEK 75
SWAPL	Texas, USA	SWAPL		\$18
Swiss APL (SAUG)	Bern	Part of Qity SI-Info		SF60 (SI) + SF20 (SAUG)
Toronto SIG	Toronto, Canada	Gimme Arrays!	Monthly Meetings, APL skills database, J SIG, Toronto Toolkit	\$25

ADDRESSES

ORGANISATION	CONTACT	ADDRESS, TELEPHONE, FAX, EMAIL etc.
ACM SigAPL	David Siegel	ACM, 1515 Broadway, 17th Floor, New York, NY 10036, USA (Subs only)
ADAPTA Software GmbH	Michael Baas	Marlenhoehe 88, 25451 Quickborn, Germany Tel: +49 4106 60977 Fax: +49 4106 67889 Email: 101523.1757@compuserve.com
Adaptable Systems	Lois & Richard Hill	49 First Street, Black Rock 3193, Australia. Tel: +61 3 9589 5578 Fax: +61 3 9589 3220 Email: adsys@ibm.net
Adaytum Software	Douglas Rowley	13 Great George Street, BRISTOL BS1 5RR UK Tel: 0117-921 5555
Adfee	Bernard Smoor	Doopsstraat 50, 4128 BZ Leidschendam, Netherlands. Tel +31 347 342 337 Fax: +31 347 342 342 Email: adfee@concepts.nl
Andrews	Dr Anne D Wilson	12 Thorny Hills, Kendal, Cumbria LA9 7AL, UK Tel: 01539-731205
APL-385	Adrian Smith	Brook House, Gilling East, York YO62 4JJ UK. Tel: 01439-788385 Fax: 01439-788194 Email: 100331.644@compuserve.com
APL Bay Area APLBUG	Curtis Jones (Sec)	228 South 15th Street, San Jose, CA 95112-2150, USA Tel: +1 (408) 292-4060 Email: jonesca@vnet.ibm.com
APL Club Austria	Harald F. Nelson	c/o N-TECH, Siebenbrunnentfeldg. 4-6, A-1050 Wien, Austria. Tel: +43 1 5458063 Fax: +43 1 5458065-17
APL Club Germany	Dieter Lattermann	Rheinstraße 23, D-69190 Walldorf, Germany. Tel: +49 6227-63469 Compuserve: 100332,1461
APL Solutions Inc	Eric Landau	1107 Dale Drive, Silver Spring, MD 20910-1607 USA Tel: +1 (301) 589-4621 Fax: +1 (301) 589-4618 Email: elandau@cals.com
Ajay Askoolum	Ajay Askoolum	42 Hanworth Road, Redhill, Surrey RH1 5HT Tel: 01737 771643 Email: 106173.3347@compuserve.com
Association Francophone pour la promotion d'APL	Ludmila Lemagnen	174 Boulevard de Charonne, F-75020 Paris, FRANCE Email: lemagnen@aol.com
AUSCAN Software Ltd	Richard Procter	PO Box 39, Mansfield, Ontario L0N 1M0 Canada Tel: +1-705-434-1239 Email: rjp@interlog.com
BACUS	Joseph De Kerf	Roolberg 72, B-2570 Duffel, Belgium. Tel: +32 15 31 47 24
Beautiful Systems, Inc.	Jim Goff	308 Old York Road, Suite 5, Jenkintown, PA 19046, USA Tel: +1 (215) 886-2636; Fax: +1 (215) 886-4888
Bloomsbury Software	Peter Day	Bloomsbury House, 74-77 Great Russell St., London WC1B 3DA UK. Tel: 0171-436 9481 Fax: 0171-436 0524 Email: pd@bloomsbury-software.co.uk
Camacho	Anthony Camacho	11 Auburn Road, Redland, Bristol BS8 6LS UK. Tel: 0117-9730036. email: acamacho@dx.compulink.co.uk Reuternet (Sharp): ACAM
Ray Cannon		21 Woodbridge Rd, Blackwater, Camberlay, Surrey GU17 0BS UK Tel: 01252-674697 Email: 100430.740@compuserve.com
Causeway Graphical Systems Ltd	Adrian Smith	The Mallings, Castlegate, MALTON, North Yorks YO17 7DP UK Tel: 01653-696760 Fax: 01653-697719 Email: causeway@compuserve.com
Paul Chapman		51B Lambs Conduit Street, London WC1N 3NB UK. Tel: 0171-404 5401. Compuserve: 100343,3210
Cinerea AB	Rolf Kornemark	Box 61, S-193 00 Sigtuna, Sweden. Tel/Fax: +46 859 255 421 Email: rolf@cinerea.se

CODEWORK	Mauro Guazzo	Corso Cairoli 32, 10123 Torino, Italy. Tel: +39 11 885188 Fax: +39 11 812 2652 Email: codework@inrete.it
ComLog Software	Jeff Pedneau	18728 Bloomfield Road, Olney, MD 20832 USA Tel: +1 (301) 260-1435 Email: jeff@softmad.com
CPCUG	Lynne Sturtz	Capital PC User Group, 51 Monroe Street, Suite PE-2, Rockville, Maryland 20850-2421, USA. Tel: +1 (301) 762-9372 Fax: (301) 762-9375. Email: gjerlov@ibm.net
Danish User Group	Per Gjerlov	
Dinosoft Oy	Pertti Kalliojärvi	Lönnrotinkatu 21C, 00120 Helsinki, FINLAND. Tel: +358 9 70028820 Fax: +358 9 70028824 Email: dinosoft@dinosoft.fi
Dutch APL Association	Bernard Smoor (Sec)	Postbus 1341, 3430BH Nieuwegein, Netherlands. Tel: +31 347 342 337 Fax: +31 347 342 342
Dyadic Systems Ltd.	Peter Donnelly	Riverside View, Basing Road, Old Basing, Basingstoke, Hants RG24 0AL UK. Tel: 01256-811125 Fax: 01256-811130
Entropy Software Ltd	George MacLeod	37 Newhouse Rd, Bovingdon, Herts, HP3 0EJ UK Tel: 01442-834015 Email: 100412.1305@compuserve.com
Eventra	Stuart Sawabini	644 Danbury Road, WILTON, CT 06897 USA. Tel: +1 (203) 762-3933 Fax: +1 (203) 762-2108 Email: ssawabini@aol.com, eshaw@eventra.com
Evestic AB	Ole Evero	Bertellusvagen 12A, S-145 38 Tullinge, Sweden Tel/Fax: +46 778 4410 Email: ole.evero@mailbox.swipnet.se
FinnAPL	Olli Paavola	Suomen APL-Yhdistys RY, FinnAPL RF, PL 1005, 00101 Helsinki 10, Finland Email: olli.paavola@pyr.fi
General Software Ltd	M.E. Martin	22 Russell Road, Northolt, Middx, UB5 4QS UK. Tel/fax: 0181-864 9537
Godin London Incorporated	Gaëtan Godin	12 Gerrard St., London, Ontario, Canada N6C 4C5 Tel: +1 (519) 679-8290 Fax: +1 (519) 438-6381 Email: info@godin.on.ca
H.M.W.Trading Systems Ltd		Hamilton House, 1 Temple Avenue, Victoria Embankment, London EC4Y 0HA UK. Tel: 0171-353 8900; Fax: 0171-353 3325; Email: 100020.2632@compuserve.com
Hoekstra Systems Ltd	Bob Hoekstra	Dominique, Salisbury Road, Woking, Surrey, GU22 7UR UK Tel: 01483-771028 Email: bob.hoekstra@khamlin.demon.co.uk
HRH Systems	Dick Holt	3802 N Richmond St, Suite 271, Arlington, VA 22207 USA Tel: +1 (703) 528-7624; Fax: +1 (703) 528-7617; Email: dick.holt@juno.org
Michael Hughes		28 Rushton Road, Wilbarston, Market Harborough, Leics. LE16 8QL UK. Tel: 01536-770998 Email: 101740.1203@compuserve.com
IAC/Human Interfaces	Ian A. Clark	IAC Human Interfaces, Unit R21, Auckland New Business Centre, St. Helen Auckland, Bishop Auckland, County Durham, DL14 9TX. Tel: 01368 605544. Email: 100021.3073@compuserve.com
I-APL Ltd	Anthony Camacho (for queries, order forms) J.C. Business Services (for pre-paid orders only)	11 Auburn Road, Redland, Bristol BS6 6LS UK. Tel: 0117-9760036 email: acamacho@clx.compulink.co.uk Reuternet (Sharp): ACAM 56 The Crescent, Milton, Weston-super-Mare, Avon, BS22 8DU UK Tel: 01934-625181
IBM APL Products	Nancy Wheeler	APL Products, IBM Santa Teresa, Dept M46/D12, 555 Bailey Avenue, San Jose CA 95141, USA. Tel: +1 (408) 463-APL2 (~2752) Fax: +1 (408) 463-4488 Email: APL2@vnet.ibm.com Cserve: GO IBMAPL2
INFOSTROY	Alexei Miroshnikov	3 S. Tulen Lane, St. Petersburg 191186 Russia. Tel: +7 812 312-2673 Fax: +7 812 311-2184 Email: aim@infostroy.spb.su
Insight Systems ApS	Morten Kromberg	Nordre Strandvej 119C, DK-3150 Hellebæk, Denmark Tel: +45 49 76 20 20 Fax: +45 49 76 20 30 Email: info@insight.dk
Iverson Software Inc.	Eric Iverson	33 Major Street, Toronto, Ontario, Canada M5S 2K9 Tel: +1 (416) 925-6096; Fax: +1 (416) 488-7559 Email: info@jsoftware.com
JAD Software	David Crossley	580 Eyer Drive, #81 Pickering, Ontario, Canada L1W 3B7 Tel: +1 (905) 837-1895 Fax: +1 (905) 831-5172
Japan APL Assoc	Toshio Nishikawa	1-8-13 Masujima Bldg.6F Higashi Gotanda Shinagawa-ku, Tokyo Japan 141-0022 Tel: +81 (03) 3280-0411 Fax: +81 (03) 3280-0418 Email: KYY00361@niftyserve.org.jp
J Austria	Joachim Hoffmann	c/o Causeway Graphical Systems. Email: JoHo@ping.at
Mackay Kinloch Ltd	Alastair Kinloch	519 Webster's Land, Edinburgh EH1 2RX, Scotland, UK Tel/Fax/Answerphone: 0131 228 3580 Pager/Voicemail: 01426 98 3858 Email: Alastair_Kinloch@compuserve.com
Phil Last Ltd	Phil Last	146 Crossbrook Street, Cheshunt, Herts, EN8 8JY UK. Tel: 01992 633807 Fax: 0121 359 0375 Email: phil_last@compuserve.com

Lescasse Consulting	Eric Lescasse	18 rue de la Belle Feuille, 92100 Boulogne, France Tel: +33.1.46.05.10.76 Fax: +33.1.46.04.60.23 Email: eric@lescasse.com
Ungo Allegro USA, Inc	Steven J Halasz	1105 Chicago Avenue, Suite 155, Oak Park, IL 60302, USA. Tel: +1 800 548 4621-1 Email: sjhalasz@interaccess.com
Mercia Software Ltd.	Gareth Brentnall	Holt Court North, Heneage Street West, Aston Science Park, Birmingham B7 4AX UK. Tel: 0121-359 5096. Fax: 0121-359 0375
MicroAPL Ltd.	Richard Nabavi	South Bank Technopark, 90 London Road, LONDON SE1 6LN UK Tel: 0171-922 8866 Fax: 0171-928 1006 Email: MicroAPL@microapl.demon.co.uk
Milinta Inc.	Dan Baronet	4215 St-Andre, Montreal, CANADA H2J 2Z3. Tel: +1 (514) 529-1434
Ellis Morgan	Ellis Morgan	Myrtle Farm, Winchester Road, Stroud, Petersfield, Hants GU32 3PE UK. Tel: 01730-263843 Email: Ellis@mrlfirm.demon.co.uk
Oasis b.v.	Theo Zwart, Louis Rijkse	Lekstraat 4, 3433 ZB Nieuwegein, Holland Tel: +31 30 60 66 336 Fax: +31 30 60 65 844 Email: info@oasis.nl or rijkse@oasis.nl
Object Oriented Ltd	Walter G. Fil	Am Grendel 2, CH-6004 Luzern, Switzerland. Tel: 41 41 418 70 70 Fax: 41 41 418 70 77 Email: info@object-oriented.com
Optima Systems Ltd	Paul Grosvenor	115 Brighton Road, Purley, Surrey CR8 4HE UK Tel: 0181-763 2490 Fax: 0181-763 2491
RadSys Technologies AB	Randolph Schrab	Lovsångarv. 18, S-756 52 Uppsala, Sweden. Tel: +46 18 32 41 53 Fax: +46 708 1996 11 Email: 100564.2544@compuserve.com
Renaissance Data Systems Ed Shaw		P.O. Box 313, Newtown, CT 06470, USA. Tel: +1 (203) 270-9729 Email: rendata@aplbooks.cnhost.com or orders@aplbooks.cnhost.com
RE Time Tracker Oy	Richard Eller	Mikonkatu 8 A, 2.krs. PL 363, 00101 Helsinki, Finland. Tel: +358 9-621 3300 Fax: +358 9-621 3378 Email: re@rett.fi
The Rochester Group Inc.	Robert Miller	600 Park Avenue, Rochester, NY 14607-2926, USA. Tel: +1 (716) 271-1110. Fax: +1 (716) 271-1230
Rome/Italy SIG	Mario Sacco	Casella Postale 14343, 00100-Roma Truilo, Italy Email: marsac@vnet.ibm.com
RusAPL	Boris Makeev	Box 971 (for Makeev), Dmitrovskoe Sh., 2, 127434, Moscow, Russia Tel/fax: +7 95 210-7783 Email: makeev@atom.al.x-atom.net
SE APL Users Group	John Manges	413 Comanche Trail, Lawrenceville, GA 30044, USA Tel: +1 (770) 972-3755 Email: seapldoc@aol.com
Shepp & Associates LLC	Andrew Shepp	1312 Washington Avenue, 6th Floor St. Louis MO 63103, USA Tel: +1 (314) 621-3272 Fax: +1 (314) 621-4267 UK Address: Claridge House, 29 Barnes High St, London SW13 9LW Tel: 0181 8768666 Fax: 0181 8768660 Email: ashepp@compuserve.com
Snake Island Research Inc	Bob Bernecky	18 Fifth Street, Ward's Island, Toronto, Ontario M5J 2B9 Canada Tel: +1 (416) 203-0854 Fax: +1 (416) 203-6999 Email: bernecky@eeog.toronto.edu
SOCAL (South California)	Roy Sykes Jr	Sykes Systems Inc, 4649 Willens Ave, Woodland Hills, CA 91364-3812 USA Tel: +1 (818) 222-2759 Fax: +1 (818) 222-9250
Solliton Associates	Laurie Howard	Solliton Associates Ltd, Groot Blankenberg 53, 1082 AC Amsterdam, Netherlands Tel: +31 20 648 4475 Fax: +31 20 644 1206 Email: sales@solliton.com
SovAPL	Alexander Skomorokhov	PO Box 5061, Obninsk-5, Kaluga Region, Russia Tel: +7(08439)31463 Email: askom2@kaluga.rosmail.com
Strand Software Inc	Anne Faust	19235 Covington Court, Shorewood MN 55331 USA Tel: +1 (612) 470-7345 Email: amfaust@aol.com
Rex Swain	Rex Swain	8 South Street, Washington, CT 06793 USA. Tel: +1 (860) 868-0131 Fax: +1 (860) 868-9970 Email: rswain@acm.org
SwedAPL	Christer Ulfhielm	Novator Consulting Group AB, Svärdvägen 11C, S-182 33 Danderyd Sweden Tel: +46 8 622 63 50 Fax: +46 8 622 63 51 CServer: 100341.404
Swiss APL User Group		Swiss APL User Group, CH-3001, Bern 1, Switzerland Email: si@ifi.unizh.ch
Sykes Systems Inc	Roy Sykes Jr	4649 Willens Ave., Woodland Hills, CA 91364, USA Tel: +1 (818) 222-2759 Fax: +1 (818) 222-9250
Toronto SIG	Richard Procter	PO Box 55, Adelaide St. Post Office, Toronto Ontario M5C 2H8, Canada Email: info@torontoapl.org
Stephen Wynn		8 Clarence Gardens, Brighton, Sussex BN1 2EG Tel: 01273-327238 Email: centre@mistral.co.uk
Zark Incorporated	Gary A. Bergquist	23 Ketchbrook Lane, Ellington CT 06029, USA. Tel: +1 (860) 872-7806

WORLD WIDE WEB SITES

ACM SigAPL	www.acm.org/sigapl/
Adaytum Software	www.adaytum.com/
AFAPL	www.ensmp.fr/~scherer/angle/ (Journal available on line)
APL2000	www.APL2000.com/
APL-385	www.demon.co.uk/apl385/
AUSCAN	www.interlog.com/~rjp/auscan/
Capital PC User Group	http://cpcug.org/
Causeway	www.causeway.co.uk/
CODEWORK	www.inrete.it/cdwk/eng/homee.html
COSY (Bob Armstrong)	www.cosy.com/
Dinosoft Oy	www.dinosoft.fi/
Dyadic Systems Ltd	www.dyadic.com/
Eventra	www.eventra.com/
FinnAPL	www.pyr.fi/apl/
Godin London Inc	www.godin.com/
Hoekstra Systems	www.khamsin.demon.co.uk/about/hsl/noframe.html
IBM APL2	www.torolab.ibm.com/ap/apl/apl2.html
Infostroy	www.insight.dk/infostroy/
Insight Systems ApS	www.insight.dk/
Iverson Software Inc	www.jsoftware.com/
Japan APL Association	www.naska.co.jp/JAPLA/
Lescasse Consulting	www.lescasse.com/
Lingo Allegro USA Inc	www.lingo.com/
Mackay Kinloch Ltd	ourworld.compuserve.com/homepages/Alastair_Kinloch/
MicroAPL Ltd	www.microapl.co.uk/
Oasis b.v.	www.oasis.nl/
Renaissance Data	www.aplbooks.cnchost.com/
RE Time Tracker Oy	www.rett.fi/
The Rochester Group Inc.	www.rochgrp.com/
Shepp & Associates	www.digitravel.com/
SigAPL	www.acm.org/sigapl/
Strand Software Inc.	www.jsoftware.com/
Rex Swain	www.pcnet.com/~rhswain/
Toronto SIG	www.torontoapl.org/
Jim Weigang	www.chilton.com/~jimw/

FTP SITES

IBM APL2	ps.boulder.ibm.com/ps/products/apl2/
Toronto toolkit	see Toronto SIG home page
Waterloo Archive	archive.uwaterloo.ca/ftparch/languages/apl
APL-to-ASCII	archive.uwaterloo.ca/languages/apl/workspaces/aplascii

Zark Newsletter Extracts

Introduced by Jon Sandles

This section continues our series of extracts from the Zark APL tutor news. Once again, we publish the solution to last issue's exercise and also the solution to the crossword. Another 'Limbering Up' is reprinted along with a crossword. I hope you are all enjoying these reprints, and please remember, don't be afraid to submit a new solution, there are many new approaches around. (Why not use J? or K?)

Utility Corner:

Converting Between Character Vectors and Matrices

(The purpose of this column is to make you more productive by introducing you to utility functions. Think of utility functions as APL functions that have names instead of symbols. By expanding your function vocabulary, you'll be able to write APL code that's more concise, more efficient, and more readable.)

In the last issue's LIMBERING UP column, we discussed the three common forms used to store names in APL: character matrix with one name per row, delimited character vector with a delimiter character (e.g. '/') preceding each name, and character nest with one name per item.

We discussed the need to convert from any one of these forms to another. You were asked to write utility functions for converting from matrix to vector, or from vector to matrix. In this column, we'll show the winning solutions and present utility functions for converting between all the different forms. We'll also share some benchmarks performed to give you some feeling for the comparative sizes of these forms and for the processing time required to convert from one form to another.

First, the contest. Virtually all solutions received for the *MATAVEC* (matrix to vector) function used the same algorithm. A few readers managed to squeeze a bit more speed out of the algorithm by relaxing the assumptions of the problem. Not fair, but clever. For example, the problem required that trailing blanks be deleted from each row. If you're willing to delete all redundant blanks, including those at the beginning or in the middle of a name, as well as the trailing blanks, you can find a faster algorithm.

The winning solution, listed below, includes the primary algorithm (and an alternative that might be a tad faster on *your* machine) and a couple of cheating (though viable) algorithms.

```

V CV=DELIM MATΔVEC CM
[1]  A Converts left-justified character matrix CM
[2]  A to a delimited character vector CV, where
[3]  A DELIM is the delimiter character. The
[4]  A trailing blanks are deleted from each row.
[5]  A The delimiter precedes each segment.
[6]  A
[7]  CV+(',1,φv\' \'≠φCM)/,DELIM,CM
[8]  A CV+(',1,φv\φCM≠\' \')/,DELIM,CM  A Alternative
[9]  A
[10] A A bit faster, but doesn't handle all-blank
[11] A rows when the delimiter is a blank:
[12] A CM+DELIM,CM
[13] A CV+(',φv\' \'≠φCM)/,CM
[14] A
[15] A Even faster but deletes all redundant
[16] A spaces, not just the trailing ones:
[17] A CM+DELIM,CM
[18] A CV+\' \'≠CM
[19] A CV+(',CVv0 1+CV,0)/,CM
V

```

The solutions for the *VECΔMAT* function, on the other hand, offered considerable variety. The classic solution to this problem involves a costly outer product. Once you determine the lengths of the segments (by a shifting technique shown below), the outer product is:

```
LENGTHS◦.z\[/LENGTHS
```

The resulting Boolean matrix is then ravelled and used to expand the character vector, generating the needed trailing blanks. Reshape the expanded character vector into a matrix and pad or truncate the matrix to the desired number of columns and you're done.

The outer product is costly because it involves so many needless comparisons. We hoped for more clever solutions. And we got them. The winning solution capitalises on the fact that the replicate (/) function has both "compression" and "expansion" capabilities. For example:

```

1 1 0 3 1 2/4 5 6 7 8 9
4 5 7 7 7 8 9 9
1 1 0 1 1 1 2 0/'BET FRED'
BE FREE

```

The solutions for Howard Peelle and Donald Lagosz-Sinclair started the same way but drifted apart midway through. Howard used replicate to expand the character vector for the needed trailing blanks. He then reshaped it to a matrix and used take (+) to pad or truncate it to the desired number of columns. Howard's solution was quite fast and was among the smallest submitted.

Donald decided to avoid using the take function, and instead built the padding and truncating in the replication vector. The logic was more complex than Howard's and so the function was larger, but it was the fastest function submitted. Since Howard's solution was significantly smaller than Donald's, but only slightly slower, his score edged out Donald's (443 to 428).

The `VECΔMAT` function listed below is a merger of the best parts of Howard's and Donald's solutions. Both algorithms are shown. Use the one you prefer. As written, the function scored 481:

```

V CM+WIDTH VECΔMAT CV;D;DELIM;END;LEN;LONG;N;PAD;REPL
[1]  a Converts delimited character vector CV to a
[2]  a left-justified character matrix CM, having
[3]  a WIDTH columns. The delimiter precedes each
[4]  a segment. If WIDTH is empty, the result has
[5]  a as many columns as the longest segment.
[6]  a
[7]  N←pCV+CV,D+1+CV a Stick a delimiter at end
[8]  REPL←Np1 a Selection/replication vector
[9]  REPL[[]IO]+0 a Leading delim always removed
[10] a No. segments, indices of ending/all delims:
[11] N←pEND+1+DELIM+(CV=D)/N
[12] CV[END]+ ' ' a Replace delimiters by blanks
[13] LEN←END-1+DELIM a Lengths of segments (+1)
[14] a
[15] a Approach 1:
[16] PAD←1[[]LEN a 1[ for empty list
[17] REPL[END]+PAD-LEN a No. blanks to replicate
[18] PAD←PAD-1 a Exclude delims in total width
[19] CM←(N,1+WIDTH,PAD)+ (N,PAD)pREPL/CV
[20] a
[21] a Approach 2:
[22] a Handle empty left arg (1[ for empty list):
[23] a 1(0←WIDTH)/'WIDTH'-1+1[[]LEN'
[24] a Amount to pad (pos) or truncate (neg).
[25] a Insert pad amounts (blanks to replicate):
[26] a REPL[END]+0[[]PAD+(WIDTH+1)-LEN
[27] a +(0←pLONG+(PAD<0)/N)pL1 a Find long segs
[28] a PAD+|PAD[[]LONG] a No. of excess characters
[29] a Cumulative offsets to excess:
[30] a D←PAD/(END[[]LONG]-PAD+[]IO)-+~1+0,PAD
[31] a REPL[D+[]D]+0 a Mark excess chars to delete
[32] a Select/expand chars and form matrix:
[33] a L1←CM+(N,WIDTH)pREPL/CV
V

```

One warning: The first algorithm builds a matrix (as an intermediate result) that has as many columns as the **longest** name. If the longest name is very long and your workspace available is small, you may get a *WS FULL*. The second algorithm avoids this limitation.

Also included for completeness are four other utility functions for converting between nested vectors and character matrices or vectors:

```

▽ CN←MATANEST CM
[1]  A Converts left-justified character matrix CM
[2]  A to a nested vector of character vectors CN.
[3]  A The trailing blanks are deleted from each
[4]  A row.
[5]  A
[6]  CN←(+/\v\ '⊥CM)ρ⌈CM A APL*PLUS
[7]  A CN←(+/\v\ '⊥CM)ρ⌈[1+⊥IO]CM A APL2
[8]  A CN←(+/\v\ '⊥CM)ρ⌈>⊥ 1 CM A Sharp APL
▽

```

```

▽ CN←VECDNEST CV
[1]  A Converts delimited character vector CV to
[2]  A a vector of character vectors CN. The
[3]  A delimiter precedes each segment
[4]  CN←1+⌈(CV=1+CV)⊥CV A APL*PLUS
[5]  A CN←(CV≠1+CV)⊥CV A APL2
[6]  A CN←~1⊥CV A Sharp APL
▽

```

```

▽ CM←WIDTH NESTMAT CN
[1]  A Converts nested vector of character vectors
[2]  A CN to a left-justified character matrix CM,
[3]  A having WIDTH columns. If WIDTH is empty,
[4]  A the result has as many columns as the
[5]  A longest segment.
[6]  A
[7]  A APL*PLUS
[8]  A ⊥(0∈ρWIDTH)/'WIDTH+⌈/⌈ρ⌈CN' ⊙ CM←+WIDTH+⌈CN
[9]  A
[10] A APL2
[11] A ⊥(0∈ρWIDTH)/'→0,0ρCM←⊥CN'
[12] A CM←⊥WIDTH+⌈CN
[13] A →0
[14] A
[15] A Sharp APL:
[16] A ⊥(0∈ρWIDTH)/'WIDTH+⌈/,ρ⌈>CN'
[17] A CM←WIDTH+⌈>CM
▽

```

```

      V CV+DELIM NESTΔVEC CN
[1]  A Converts nested vector of character vectors
[2]  A CN to a delimited character vector CV,
[3]  A where DELIM is the delimiter character.
[4]  A The delimiter precedes each segment
[5]  CV←⊃,/DELIM,"CN A APL*PLUS, APL2
[6]  A Sharp APL:
[7]  A CV←(⌈/,ρ⊃>CN)⊃>CN
[8]  A CV←(⌈1,ϕ∨\CV≠' ')⌈/,DELIM,CV
      V

```

You might be interested to see the sizes of our 500 test names (whose longest name had 23 characters) in each of their different forms for some different implementations of APL:

	Size, in bytes		
	Vector	Matrix	Nest
APL*PLUS PC	8,192	11,512	—
APL*PLUS II	8,208	11,528	22,416
APL*PLUS mainframe	8,200	11,528	19,288
APL2 mainframe	8,197	11,520	13,697
Sharp APL mainframe	8,200	11,520	22,404

Notice that the vector form is the smallest for all implementations, and the nested form is the largest. The nested form can be as much as three times as large as the vector form. Remember that. The nested form provides you convenience at the cost of object size. Take your pick.

We also timed the conversions from one form to another, using the utility functions above. All timings have been scaled, within each environment, so the processing time for the *MATΔVEC* function is 100.

	APL* PLUS PC	APL* PLUS II	APL* PLUS mf	APL2 mf	Sharp APL mf
<i>MATΔVEC</i>	100	100	100	100	100
Assembler (1)	1	1	—	—	—
<i>VECΔMAT</i>	42	41	122	172	71
Assembler (2)	4	3	—	—	—
<i>MATΔNEST</i>	—	54	354	180	193
<i>VECΔNEST</i>	—	42	316	52	151
<i>NESTΔMAT</i> (3)	—	35	379	22	136
<i>NESTΔMAT</i> (4)	—	22	215	105	91
<i>NESTΔVEC</i>	—	106	346	180	235

Notes:

1. Assembler routine *MATtoSS* from workspace *ASMFNS*
2. Assembler routine *SStoMAT* from workspace *ASMFNS*
3. With left argument 10
4. With left argument 25

You can see how much the results vary from one implementation to another. The difference between PC-based systems and mainframe-based systems is especially striking. You should learn the peculiarities of your own version of APL.

Also notice how much faster the assembler routines are than their APL counterparts. As fast as our APL functions are, they can't beat the assembler. This doesn't mean that the APL functions are so slow as to be useless. On the contrary, they're quite fast. It's just that the assembler is greased lightning.

If you have assembler routines available to you, and if extreme speed is important, use them. Be forewarned though: by using assembler routines, you're making your APL application less portable. If you decide to move the application from one implementation of APL to another, the assembler routines will have to be replaced. By what? Why, one of the APL functions above, of course!

LIMBERING UP CONTEST: Entering Ranges of Numbers

(The purpose of this column is to work some flab off your APL midsection. Like muscles, your APL skills can atrophy if not exercised with adequate frequency and variety. This column presents a task for you to perform. Set aside a few minutes from your busy schedule and work the task. Mail in your solution and stay tuned for the results.)

Sometimes, a question in an interactive application requires the entry of a bunch of numbers. For example:

```
OFFICE NUMBERS: 13 17 18 19 20 21 22 23 24 25
                  28 29 30 31 34 36
```

Entering a lot of numbers is both time-consuming and error-prone. A form of "short-hand" can eliminate these problems:

```
OFFICE NUMBERS: 13, 17 TO 25, 28 TO 31, 34 36
```

Your task is to write a monadic function named *TO* that converts an input character vector of this form into the numeric vector it represents. For example:

```
      TO '13, 17 TO 25, 28 TO 31, 34 36 '
```

13 17 18 19 20 21 22 23 24 25 28 29 30 31 34 36

The *TO* function allows commas or blanks between numbers, and allows minus (-) or negative (°) signs for negative numbers. Wherever the word "TO" appears, the set of continuous integers between the specified limits is inserted. If the second limit is smaller than the first limit (e.g. '9 TO 5'), the continuous integers are in descending order (9 8 7 6 5)

Normally, *TO* returns a numeric vector. However, if the character vector argument is poorly formed, the result is an integer scalar that indicates the problem.

Here are the possible error code results:

- 1 - "TO" is missing arguments: N+TO '3 TO '
- 2 - Non-numeric values: N+TO '5 TO ONE'
- 3 - Non-integral values: N+TO '6.2 TO 8'

Here's an illustration of the use of the *TO* function:

```
[4] ASK:[]+P+'OFFICE NUMBERS: '
[5] ONOS+TO(P)+[]
[6] →OK IF 1=PPONOS
[7] []+'** ENTER LIST OF NUMBERS, USING <TO> FOR RANGES.'
[8] →ASK
[9] OK:etc.
```

When writing *TO*, you may use the system functions *VI* and *FI*. (Use the utility function *ΔVI*, presented in the 1989 Quarter 4 issue of Zark APL Tutor News, if your version of APL does not support *VI* and *FI*.)

Thanks to Eric Baelen for suggesting this as a Limbering Up problem.

The *TO* function should be as efficient as possible and as small as possible. Mail your solution on paper or disk to:

Jon Sandles, 138 Burton Stone Lane
YORK YO30 6DF UK

Email: jon_sandles@csi.com

250 points will be awarded for the fastest functions: 250 points for the smallest. Functions that are not the fastest or the smallest will be awarded points on a pro-rata basis. For example, if your function is 5 times slower than the fastest, you will receive 50 points ($250 \div 5$). The highest score possible is 500, but only if the same function is both fastest and smallest. The highest-scoring functions will be listed in the next issue of this newsletter. The rules:

1. At least 20 percent of the functions must be comments. The size of the function will be determined after stripping out all comments.
2. The speed of the function will be measured by running it on a set of representative arguments.

Good luck and happy limbering!

P.S. For those of you who are ambitious, or who feel you can't work on the Yin without working on the Yang, try writing the *UNTO* function too. The *UNTO* function does the opposite of the *TO* functions. For example:

```
UNTO 13 17 18 19 20 21 22 23 24 25 34 36
13, 17 TO 25, 34 36
```

The right argument of *UNTO* is a numeric vector. The result is a character vector. It should always be true that:

```
NV=TO UNTO NV
```

For every set of continuously increasing or decreasing integers that span at least 4 numbers, *UNTO* replaces all but the extreme numbers of the span by the word *TO*. For example:

```
UNTO 5 6 7 8 9
5 TO 9
UNTO 5 6 7
5 6 7
```

UNTO also inserts commas before and after "TO" phrases (except at the beginning or ending of the result) and replaces negative signs (*~*) by minus signs (*-*). For example:

```
UNTO 3 2 1 0 ~1 ~2 ~4 ~5 ~10 ~9 ~8 ~7 ~6
3 TO -2, -4 -5, -10 TO -6
```

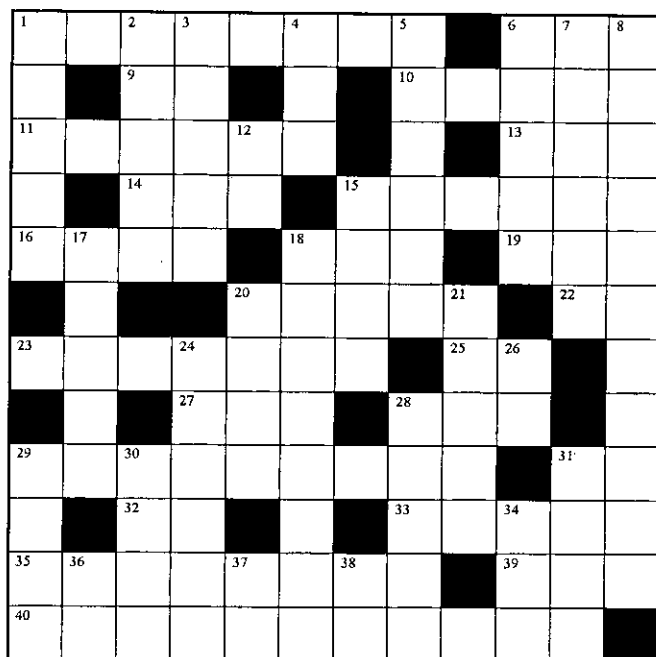
Reprinted with kind permission from Zark APL Tutor News, a quarterly publication of Zark Incorporated, 23 Ketchbrook Lane, Ellington, CT06029, USA

Solution to Crossword from Vector 15.2

1 ϕ	2 L	3 (	4 N	5 $\sim$	6 2	7 $\times$	8 ι	9 5	10 0
11 1	12 E	13 5	14 $\times$	15 ,	16 M	17 5	18 5	19 ρ	20 5
21 +	22 /	23 	24 N	25 V	26 +	27 R	28 ρ	29 	30 V
31 (	32 ι	33 N	34)	35 $\circ$	36 .	37 =	38 ι	39 N	40 $\rightarrow$
41 C	42 M	43 I	44 $\wedge$	45 .	46 =	47 1	48 5	49 $\uparrow$	50 C
51 $\uparrow$	52 ?	53 A	54 +	55 2	56 0	57 T	58 -	59 	60 T
61 N	62 -	63 N	64 $\in$	65 ι	66 7	67 ?	68 F	69 ι	70 4
71)	72 S	73 I	74 N	75 L	76 ϕ	77 S	78 L	79 1	80 0
81 $\in$	82 T	83 ϕ	84 ρ	85 M	86)	87 ρ	88 V	89 0	90 0
91 V	92 $\sim$	93 0	94 ϕ	95 (	96 ϕ	97 ρ	98 M	99)	100 ρ
101 E	102 v	103 $\times$	104 ρ	105 N	106 M	107 $\div$	108 S	109 ρ	110 E
111 C	112 /	113 M	114 M	115 $\neq$	116 M	117 C	118 I	119 I	120 C

Erratum: there were a couple of mistakes in the published solution to the last crossword. The correct answer to 11 across is B + ϕ ($\phi \rho B$) ρVEE and that to 31 down is VS , PP. Sorry!

Crossword



Across

1. The inverse of $+/$, i.e. the first difference of V
6. $1 + \phi + '[(A)]'$
9. $1 \perp 14$
10. Accept numeric input; call it NMV
11. Negate the vector MB and show its running total, $LIFO$ (last in, first out) basis
13. Given distinct values in A , return ρA without using ρ
14. Where T is found in V
15. $(R111)\rho 11/V$
16. The elements of V , capped at 99
18. $3 + '(0 \in 3 + V)'''$
19. SN is contained in...
20. The two row matrix YM of year and month numbers, converted to months since the year 0000, for date arithmetic

22. If SV represents Scalar left argument, Vector right argument, what combination can cause `LENGTH ERROR` for scalar functions?
23. $(V \text{ I } 25) + (25 \text{ I } N > 0) - (25 \text{ I } N < 0)$
25. The magnitude of K
27. Outer product product
28. The location of V 's first 9
31. Evaluated input as a vector
32. The opposite of $+ /$ (silly)
33. For scalar MA , $+ / 20 \mu MA$
35. 10 13 16 19 ... 34
39. The number of elements in array V
40. $\{ 5 \} \circ . = \{ 5 \}$

Down

1. The fractions used to spread a value, given relative weights in the vector V
2. $\neg 1 + (\neg 1 + V9), V9$
3. 9 8 7 6 ... 1
4. $\neg \wedge / , B$
5. One more than VN if any of Y are true
6. The system command for $\square NL \ 2$
7. Set A as the indices of the elements of the vector NV
8. The distinct values in the character vector V
12. A nice acronym for $\imath 0$
15. If N is 1 9 9 2, an expression that returns 1992
17. The values of the matrix M , rounded to the nearest integer
18. If $VN=0$, return $\imath 0$; but if VN is positive, return two elements of ...
20. The number of 1s in the vector ...
21. $\imath 0 \rho \imath M$, in origin 0
24. Add to *EACH* of the elements of V the numbers 1 to 5
26. Fido's favourite variable name
28. The location in V of 27 and ...
29. $(2 / (2 \rho (1 \rho () ')))$
30. Are there some elements in ...
31. The matrix M expressed as a vector, with a 0 after each row
34. A is true only if A is a nonscalar
36. $2 * 5$
37. $\neg 2 + ' \pm (\imath 9) '$
38. The most efficient expression that returns 1

J-ottings 19: Getting to the Root of the Matter

by Norman Thomson

Einstein is reputed to have said "God doesn't care about our mathematical difficulties; he integrates empirically." The nonchalance with which thousands of scientists of all complexions, and even greater battalions of students, nowadays obtain numerical results involving huge amounts of computation prompts the observation that the combined skills of numerical analysis and computing science appear to have brought about at least some tiny encroachment on this aspect of the divine mind.

The new release of J contains a link to LAPACK (short for Linear Analysis Package) with which it is claimed that J joins the league of MATLAB, Mathematica, Maple etc. in providing access to such computing power, thereby countering the argument in Mathematics departments "Why should I concern myself with things like J when I have MATLAB?" This article concerns the many mathematically expressible problems which can be solved without recourse to the heavy artillery of MATLAB and LAPACK. Specifically, a root finder and an integrator can cope with a large range of tolerably well-behaved mathematical functions.

It is useful to start with a verb for linear interpolation between points (x_0, y_0) and (x_1, y_1) . Its content should create little difficulty in understanding even for those with quite modest computer awareness:

```
linp=.3 : 0
:
r=.((|.x.)-/. *y.)%-/y.      NB. x. and y. are coords of 2 points
)
    0 1 linp _0.1 0.5
0.1666667
```

An implicit, albeit less immediately comprehensible form, which is almost twice as fast is:

```
linp=.%/( -/. *|. @. . &1)~      NB. (x0.y1-x1.y0)/(y1-y0)
```

Now consider root finding. The differential operator $\mathcal{D}$. allows the Newton-Raphson formula to be written as an adverb which looks very like its mathematical equivalent:

```
newton=.1 : ']-x.%x.D.1'
```

NB. Newton-Raphson for one variable

This describes just one step of the process, so use the power adverb to carry on until further iteration produces no change:

```
Newton=.1 : ']-x.%x.D.1' (^:_)("0) NB. Newton-Raphson iteration
```

To test this and subsequent functions here are three simple verbs providing mathematical functions:

```
f1=.#.&1 _1 _2          NB. polynomial x2 -x -1
f2=.2&0.-(*^ )          NB. cos x - x.exp(x)
f3=.2&-@^&3             NB. verb to find the cube root of 2
f1 Newton 1
2
f2 Newton 1
_0.51774
f3 Newton 1
1.25992
```

The Secant Method is another root-finding technique. Since two points are needed to define a secant as opposed to one for a tangent, the secant adverb requires two start values, and at every step delivers a further two start values to define the next approximating secant. This means that the "infinity" option with power is not available, however it is still possible to give the number of iterations explicitly as a left argument:

```
sec=.1 : '}.@],] linp (x.&>@)]' (^:[])
3 f1 sec 0.5 1          NB. 3 steps of Secant Method
1.4 1.66667
3 f2 sec 0.5 1
0.5158504 0.5177667
3 f3 sec 0.5 1
1.19838 1.24658
```

To clear up the untidiness of receiving two result items whereas eventually only one is really wanted, write a short iteration verb which stops when things get sufficiently close:

```
sec1=.1 : '}.@],] linp fn&>@)]' NB. secant single step (monadic)
```

```

secant=.3 :0
:
t=.fn sec1 y.
while.(x.<|t-r=.fn sec1 t) do.t=.r end.{r
)
  eps=.0.00001
  fn=.f1
  eps secant 0.5 1
2
  fn=.f3
  eps secant 0.5 1
1.25992

```

A variation of the Secant Method, called Regula Falsi, or the Method of False Position, reorders the end points at each step in order to ensure that the function values at the next iteration are of opposite sign:

regf=.3 :0	NB. Regula Falsi=Method of False Posn
:	
yi=.fn i=.y.linp y=.fn&y.	NB. i=interpolate, yi=fn. value at i
if. +./x.> i-y. do.r=.i return. end	NB. stop if successive i's close
if.yi=&*1{y do.r=.x.regf(0{y.},i	NB. if yi,y1 of same sign choose y0
else.r=.x.regf i,1{y. end.	NB. if yi,y0 of same sign choose y1
)	
fn=.f3	
eps regf 0.5 1	
1.25992	

As an aside on the J language, I find the verb =&* (equal with regard to sign) in the penultimate line is a pleasing application of the "with" conjunction.

A variation of the Regula Falsi Method called the Illinois Method was described in APL terms in Vector Volume 12 (see [1] and [2]). This compensates for a curve turning steeply at one side of the interval by halving the relevant ordinate. A second interpolation step is inserted into regf :

```

illi=.3 :0
: yi=.fn i=.y.linp y=.fn&>y.          NB. start and finish as for regf
if. +./x.>|i-y. do.r=.1 return. end.
if.yi=&*0{y do.          NB. three lines inserted to include
    yi=.fn i=.y.linp y=.1 0.5*y      NB. an Illinois step which reduces
else.yi=.fn i=.y.linp y=.0.5 1*y end. NB. one of the ordinates by one half
if.yi=&*1{y do.r=.x.illi(0{y.),i
else.r=.x.illi i,1{y.end.
)
fn=.f1
eps illi 0.5 1
_0.9999968          NB. obtains different root from secant
fn=.f3
eps illi 0.5 1
1.25991

```

There is no particular magic about the 0.5, and presumably other fractions could be experimented with. The broad objective of such variations of the Secant Method is either to improve convergence for "difficult" functions, or sometimes to achieve it where simpler methods fail. There is no overall best root finding function, and the choice of method in any particular case belongs to the realm of the Numerical Analyst's art, even if Mathematica, etc. is at his disposal. For well-behaved functions the variations given above rarely outperform the Newton or Secant methods in execution time.

Turn now to integration. As every schoolboy knows (provided he is in the Mathematics sixth!) Simpson's rule is the basic way of doing numerical integration when analytic methods fail:

```

Simpson=.3 :0
:
(h*+/{1,((x.-1)$4 2),1)*fn&>({.y.)+(h=-y.-/.%x.)*i.x.+1)%3
)
fn=.f2
2 Simpson 0 0.5
0.3037372
4 Simpson 0 0.5
0.3037831

```

Not perhaps a J verb of compelling beauty, but the basic mathematics is reasonably recognisable. Two variations can be used to enhance it, first the adaptive method which recursively divides intervals in half and performs successive Simpson integrations in each half. If these agree within tolerance, stop the process for that interval, if not further divide the interval into halves, and perform the adaptive method in each half-interval.

```

adapt=.3 : 0
:
r=.4 Simpson y.
if. x.>|r-2 Simpson y. do. return.
else. m=.0 1|.each y.,each-:+/y.
r=./(x.adapt {.&>m).x. adapt {:&>m
end.
)
fn=.f2
eps adapt 0 0.5
0.303786

```

NB. Stop when first two Simpson
 NB. estimates sufficiently close
 NB. Divide interval in two and
 NB. integrate in each sub-interval
 NB. separately

The second variation is the Romberg method which uses series expansions for the error terms in Simpson estimation and applies corrections to progressively more refined Simpson estimates. The process is too complicated to describe in succinct comments, but can be found in most elementary texts on Numerical Analysis.

```

romb=.3 : 0
: r=.(2^1+i.x.)Simpson each<y.
while.1<#r
do. r=.).r+(_1+2^4*2+x.-#r)%~r-_1|.r end.
)

Romberg=.3 : 0
:
t=(i=.1) romb y.
while.(x.<|t-r=(i=.i+1)romb y.) do.t=.r end.
)
fn=.f2
eps Romberg 0 0.5
0.3037833

```

These few verbs and adverbs make root-finding and integration a practical proposition in many cases where the default approach might be to invoke MATLAB (if available!). Doing it this way is inherently satisfying, and offers insight into the beginnings of what lies inside the more sophisticated mathematical powerhouses.

References

- [1] Dietrich Trenkler, *Computing Clopper-Pearson Confidence Limits by the Illinois Method*, Vector 12.2, pp. 87-94
- [2] Norman Thomson, *Technical Note on Confidence Limits*, Vector 12.4, pp. 98-106

Towers of Hanoi Generalized in J

by Howard A. Peelle (hapeelle@educ.umass.edu)

J programs are defined for computing the minimum number of moves to solve the Towers of Hanoi puzzle for any number of discs and any number of pegs — including a new closed form solution.

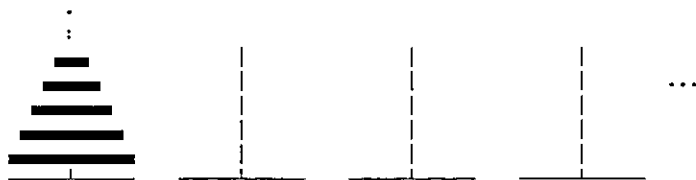
Introduction

The Tower of Hanoi puzzle is traditionally presented with 3 pegs and a small number of discs of different sizes, all stacked on one peg in increasing order. The puzzle involves moving one disc at a time from one peg to another while never placing a larger disc on top of a smaller disc. The goal is to move all discs to another peg.

Generalized to N discs, the Towers of Hanoi (sic) is often used to illustrate recursion in teaching computer science (as in [0]) and has been studied extensively for over a hundred years [1]. A simplified algorithm and J programs for generating optimal moves were defined recently [2]. It is well known that the minimum number of moves is $(2^N)-1$. Extending this solution to 4 pegs or more, however, has been regarded as a difficult challenge [3].

General Towers of Hanoi

Here the puzzle is generalized to P pegs, as well as N discs; and J programs are developed to compute the minimum number of moves required for this "General Towers of Hanoi".



A fundamental recursive algorithm (based on a presumed optimal algorithm for 4 pegs cited in [4],[5]) is to move a certain number of discs off the top of the start peg onto another peg temporarily, then move the remaining 'bottom' discs off the start peg onto the goal peg, and then move the 'top' discs onto the goal peg.

The question is: how many top discs? Surely the minimum number of moves is the minimum of the sum of three lists of moves for all possible numbers of DISCS from 1 to N-1 — that is, moves for the top DISCS using all P pegs plus moves for the bottom N-DISCS using P-1 pegs plus moves for the top DISCS using P pegs (again). An initial J program for this is:

```

Else =: '
When =: @.
DONE =: +:@(P < 2:) >. (N < 2:)
MOVES =: Mine(TOPS + BOTTOMS + TOPS) Else N Else _: When DONE
Min =: <./
Each =: "0
TOPS =: P MOVES Each DISCS
BOTTOMS =: (P-1:) MOVES Each N-DISCS
P =: [
N =: ]
DISCS =: i.@N ~. 0:

```

NB. Else is Tie
 NB. When is Agenda
 NB. DONE is (Double Atop (P<Two))
 Larger (N<Two)
 NB. MOVES is ((Min Atop (TOPS +
 (BOTTOMS + TOPS))) Else N) Else
 Infinity) When DONE
 NB. Min is Smaller Insert
 NB. Each is (input verb) Rank Zero
 NB. TOPS is P (MOVES Each) DISCS
 NB. BOTTOMS is (P - One) (MOVES Each)
 (N-DISCS)
 NB. P is Left input
 NB. N is Right input
 NB. DISCS is (Integers Atop N) Less 0

Examples:

```

3 MOVES 10
1023
4 MOVES 10
49
5 MOVES 10
31

```

NB. MOVES for 3 pegs and 10 discs
 NB. 4 pegs, 10 discs
 NB. 5 pegs, 10 discs

The MOVES program handles any P with any integral N, although only integral P>0 and N>=0 are meaningful. Note that the default value for P=2 with N e. 0 1 is N, and with N>1 is infinity (.) to indicate that the puzzle is not solvable.

In general, this is terribly inefficient due to the exhaustive recursions. We could build in knowledge of the 3-peg solution to bypass several levels of recursion throughout by avoiding the degenerate P<3 cases and economize the coding:

```

MOVES =: Min@(+:@TOPS + BOTTOMS) Else Solution When Done
NB. MOVES is ((Min Atop ((Double Atop TOPS) + BOTTOMS)) Else Solution)
When Done

```

```

TOPs =: P MOVes Each DISCS
BOTTOMs =: (P-1:) MOVes Each N-DISCS
Solution =: (2: ^ N) - 1:      NB. Solution is (Two Power N) - One
Done =: (P=3:) +. (N < 2:)    NB. Done is (P = Three) Or (N < Two)

```

Examples:

```

Table =: Each/      NB. Table is ((input verb) Each) Table

3 4 5 6 MOVes Table 1.11 NB. MOVes Table for 3,4,5,6 pegs and 0 to 10 discs
0 1 3 7 15 31 63 127 255 511 1023
0 1 3 5 9 13 17 25 33 41 49
0 1 3 5 7 11 15 19 23 27 31
0 1 3 5 7 9 13 17 21 25 29

```

But this is still impractical. So, we propose a direct way to find the optimal number of discs D to move off the top (for $P > 3$): round off the sum of the first N items of a list of copies of ratios of adjacent items in the P-3 diagonal of Pascal's Triangle, where the numbers of copies are respective items in that diagonal (see subprogram D).

```

MOVes =: (+:@TOP + BOT) Else Solution When Done
TOP =: P MOVes D
BOT =: (P-1:) MOVes N-D
D =: [: <. [: +/ N {. COPIES@PASCAL

NB. D is Cap Floor (Cap Sum (N Take (COPIES Atop PASCAL)))

COPIES =: # Ratios      NB. COPIES is input Copy Ratios
Ratios =: %~ 0: . }:]   NB. Ratios is input (Divide Commute)
                          (Zero Append Curtail)
PASCAL =: (P-3:) Diagonal i.@N  NB. PASCAL is (P - Three)
                          Diagonal (Integers Atop N)
Diagonal =: [ ! +      NB. Diagonal is Left input
                          Out-of Plus

```

Note that in program D a rounding function or ceiling ($>.$) could be used instead of floor ($<.$) and that MOVes returns a domain error, appropriately, for degenerate $P < 3$ with $N > 1$.

This program is much better, but it demands more and more memory as N increases because of many unnecessary copies of the PASCAL diagonal. We can calculate how much is enough of the diagonal to truncate:

```

D =: [: <. [: +/ N {. COPIES@{P PASCAL >:@Enuf)
NB. ... COPIES Atop (P PASCAL (Increment Atop Enuf))

Enuf =: N +/ . > (P-2:) Diagonal i.@N
NB. Enuf is N (Sum Dot >) {(P-Two) Diagonal (Integers Atop N))

```

Tracing an example — 4 MOVes 11 — illustrates how this program works:

```

NB. Computing 4 Enuf 11

    ,diagonal =: (4-2) Diagonal i.11 NB. 11 items of 2nd diagonal
1 3 6 10 15 21 28 36 45 55 66

    ,enuf =: 11 +/ . > diagonal NB. Sum of 11 > items in diagonal
4

    >:enuf NB. Length of diagonal needed
5 NB. Computing 4 PASCAL 5
    ,pascal =: (4-3) Diagonal i.5 NB. Truncated 1st diagonal
1 2 3 4 5 NB. Computing COPIES 1 2 3 4 5
    Ratios pascal NB. Ratios of adjacent items
0 0.5 0.66667 0.75 0.8

    ,copies =: pascal # Ratios pascal NB. Copies of Ratios
0 0.5 0.5 0.66667 0.66667 0.66667 0.75 0.75 0.75 0.75 0.8 0.8 0.8 0.8 0.8

    ,first =: 11 {. copies NB. Computing 4 D 11
0 0.5 0.5 0.66667 0.66667 0.66667 0.75 0.75 0.75 0.75 0.8 NB. First 11 items
    ,d =: <. +/ first NB. Floor of sum
31 NB. Computing 4 BOT 11
    ,bot =: (4-1) MOVes (11-6) NB. MOVes for 3 pegs, 5 discs
31 NB. (computed similarly by recursion)

    NB. Computing 4 TOP 11

    ,top =: 4 MOVes 6 NB. MOVes for 4 pegs, 6 discs
17 NB. (computed similarly by recursion)

    (+:17) + 31 NB. Computing 4 MOVes 11
65 NB. (Double top moves) + bottom moves

    4 MOVes 11
65

```

Nevertheless, MOVes is still inefficient for large N due to the (double) recursion. So, we seek a direct solution.

Closed Form Solution

A new direct algorithm for solving General Towers of Hanoi is: sum the first N powers of 2 copied by the P-3 diagonal of Pascal's Triangle. Here is an initial program for this:

```
Moves =: [: +/ N {. POWERS      NB. Moves is Cap Sum (N Take POWERS)
POWERS =: 2: ^ PASCAL # i.@N    NB. POWERS is Two Power
                                   (PASCAL Copy (Integers Atop N))

PASCAL =: (P-3:) Diagonal i.@N
Diagonal =: [ ! +
P =: [
N =: ]
```

Note that here the degenerate result for P=2 with any N>0 is 1, which can be interpreted to mean that only one move is possible.

This program can be greatly improved by the following modifications, designed strictly for P>2 pegs and N>0 discs.

```
Moves =: 2: +/ . ^ N {. P Copies i.@>:@Enuf
                                   NB. Moves is Two (Sum Dot Power)
                                   (N Take (P Copies ((Integers Atop
                                   Increment) Atop Enuf)))
Copies =: Pascal # ]              NB. Copies is Pascal Copy Right input
Pascal =: (P-3:) Diagonal ]      NB. Pascal is (P - Three)
                                   Diagonal Right input
```

For instance, with inputs P=4 and N=1000 on a Pentium PC running J4.01, speed is increased by a factor of more than 700, and space is reduced by a factor of about 200.

As an alternative, consider the following program which generalizes the formula for 4 pegs from [4]: sum the product of enough of the Pascal diagonal times enough powers of 2, minus an extra amount to compensate for how much N is exceeded by the sum of the diagonal (the corresponding item in the next Pascal diagonal):

```
SM =: Sum - Extra                NB. SM is Sum - Extra
Sum =: P (Pascal +/ . * Power) i.@>:@Enuf
NB. Sum is P (Pascal (Sum Dot Times) Power)
    ((Integers Atop Increment) Atop Enuf)
Pascal =: (P-3:) Diagonal ]
Power =: 2: ^ ]                 NB. Power is Two Power Right input
Extra =: (N ~ (P+1:) Pascal Enuf) * Power@Enuf
NB. Extra is (N (~ Commute) ((P + One) Pascal Enuf))
    Times (Power Atop Enuf)
```

This can be improved by extending the Pascal diagonal instead and by avoiding redundant computations — making it about 3 times faster, using about half the space:

```

M =: (+/ . * Powers)@(N Extend P Pascal i.@Enuf)
NB. M is (input (Sum Dot Times) Powers) Atop
    (N Extend P Pascal (Integers Atop Enuf))

Powers =: 2: ^ i.@#
NB. Powers is is Two Power (Integers Atop Tally)

Extend =: ] , (- +/)
NB. Extend is Right input Append (left input Minus Sum)

Pascal =: (P-3:) Diagonal ]
Diagonal =: [ 1 +
Enuf =: N +/ . > (P-2:) Diagonal i.@N

```

Tracing an example — 4 M 11 — illustrates how this program works:

```

      ,enuf =: 4 Enuf 11
4
      ,pascal =: 4 Pascal i.4
1 2 3 4

      ,extend =: 11 Extend 1 2 3 4
1 2 3 4 1

      ,powers =: Powers extend
1 2 4 8 16

      extend +/ . * powers
65

      4 M 11
65

```

This program computes efficiently the minimum number of moves to solve the General Towers of Hanoi puzzle for virtually any number of pegs ($P > 2$) and any number discs ($N > 0$).

Further Exploration

If we focus on $P=4$ pegs, results are easy to obtain when N is a triangular number, as reviewed in [4]. Generalizing this for any $P>2$, here is a simple program to compute the number of moves needed when N is a number in the $P-2$ diagonal of Pascal's Triangle:

```
Hanoi =: (2: * Row , 0:) + (1: , Row)
NB. Hanoi is (Two Times Row Append Zero) + (One Append Row)

Row =: ]                      NB. Row is Same input
```

For example:

```
Repeat =: ^: (i.`1:)
NB. Repeat is (input verb) Power (Integers Tie One)

Hanoi Repeat 10

1   0   0   0   0   0   0   0   0   0
3   1   0   0   0   0   0   0   0   0
7   5   1   0   0   0   0   0   0   0
15  17   7   1   0   0   0   0   0   0
31  49  31   9   1   0   0   0   0   0
63 129 111  49  11   1   0   0   0   0
127 321 351 209  71  13   1   0   0   0
255 769 1023 769 351  97  15   1   0   0
511 1793 2815 2561 1471 545 127  17   1   0
1023 4097 7423 7937 5503 2561 799 161 19   1
```

The items in this table are minimal moves for the General Towers of Hanoi puzzle in row $(P-3)+(P \text{ Enuf } N)$ and column $P-3$ (origin 0). For instance, the number of moves for $P=4$ and $N=10$ is 49 in row 4 and column 1 (since 4 Enuf 10 is 3).

In this blend of exponential $(2: ^N)$ and linear $(2: *N)$ functions, some interesting patterns emerge. For instance:

```
+/"1 Hanoi Repeat 10                      NB. Sum of each row

1 4 13 40 121 364 1093 3280 9841 29524      NB. Compare with +/\3^1..10
```

Also recall that Pascal's Triangle can be produced similarly:

```
Pascal =: (Row , 0:) + (0: , Row)
NB. Pascal is (Row Append Zero) + (Zero Append Row)
```

```
Pascal Repeat 10
1 0 0 0 0 0 0 0 0 0
1 1 0 0 0 0 0 0 0 0
1 2 1 0 0 0 0 0 0 0
1 3 3 1 0 0 0 0 0 0
1 4 6 4 1 0 0 0 0 0
1 5 10 10 5 1 0 0 0 0
1 6 15 20 15 6 1 0 0 0
1 7 21 35 35 21 7 1 0 0
1 8 28 56 70 56 28 8 1 0
1 9 36 84 126 126 84 36 9 1
```

And there are well-known patterns therein. For instance, the sums of rows (binomial coefficients) are powers of 2:

```
+/"1 Pascal Repeat 10
1 2 4 8 16 32 64 128 256 512
NB. Sum of each row
NB. Compare with 2^i.10
```

Notice how these tables are related. The items in the Hanoi table are minimal moves for items in row $(P-2)+(P \text{ Enuf } N)$ and column $P-2$ of the Pascal table. In other words, overlay the two tables and find the result of any Pascal item one row up and one column to the left in Hanoi (as also noted in [5]). For example, for $P=4$ and $N=10$ (row 5, column 2 in Pascal table), the result is 49 (row 4 column 1 in Hanoi table). Let's generalize these programs to handle any pair of coefficients for the addends and any pair of single values to extend each row:

```
Triangle =: (n * Row , a) + (m * b , Row)
NB. Triangle is (n Times Row Append a) + (m Times b append Row)

n =: {.@{.@[
m =: {:@{.@[
a =: {.@{:@[
b =: {:@{:@[

NB. n is (Head Atop Head) Atop Left input
NB. m is (Tail Atop Head) Atop Left input
NB. a is (Head Atop Tail) Atop Left input
NB. b is (Tail Atop Tail) Atop Left input
```

For example, $(2 \ 1, : 0 \ 1) \& \text{Triangle Repeat } 10$ produces the same result as Hanoi Repeat 10 above and $(1 \ 1, : 0 \ 0) \& \text{Triangle}$ is equivalent to Pascal.

Readers are encouraged to experiment with this program to discover other interesting triangles, including $(2 \ 1, : 0 \ 0) \& \text{Triangle}$ — the "Pelle Triangle" [6].

Acknowledgements

My appreciation for colleague Dr. Alan Lipp's collaboration on related mathematical analysis continues to grow. Additional thanks for their valuable feedback on previous drafts of this article are due to voluntary reviewers Professors Murray Eisenberg (University of Massachusetts), Keith Smillie (University of Alberta), and Paul Stockmeyer (College of William and Mary).

University of Massachusetts
Amherst, MA 01003 USA
email: hapeelle@educ.umass.edu

References

- [0] Graham, Knuth, and Patashnick (1989) *Concrete Mathematics: A Foundation for Computer Science*, Reading, MA: Addison-Wesley
- [1] Stockmeyer, Paul K. "The Tower of Hanoi: A Historical Survey and Bibliography" (1998) 200+ references downloadable from www.cs.wm.edu/~pkstoc/h_papers.html
- [2] Peelle, Howard A. "Towers of Hanoi Simplified in J", *Vector*, Vol. 12, No. 3 (January 1996) pp. 27-32
- [3] Poole, David G. "The Towers and Triangles of Professor Claus (or, Pascal Knows Hanoi)", *Mathematics Magazine* 67 (1994), p. 339
- [4] Stockmeyer, Paul K. "Variations on the Four-Post Tower of Hanoi Puzzle", *Congressus Numeratum* 102 (1994) pp. 3-12
- [5] Lipp, Alan "Computing Solutions to P-Peg Towers of Hanoi" (to appear)
- [6] Lipp, Alan "The Peelle Triangle" *Mathematics Teacher* 80 (1987) pp. 56-60

GENERAL ARTICLES

This section of Vector is intended for general readers who have a working knowledge of APL or J. Authors are encouraged to submit articles which illustrate effective APL applications.



**Renaissance Data Systems
P.O. Box 313
Newtown, CT 06470**

+1 (203) 270 9729

**Books on APL and J and other curiosities
of merit!**

Renaissance Data Systems announces a change in its mailing address. Please note that the telephone number remains the same.

If you would like a copy of our latest **catalog**, please send us a self-addressed legal sized envelope with one first class stamp (if in the U.S.).

Included are such titles as: **APL is EASY**, **APL - An Interactive Approach**, **APL2 at a Glance**, **APL - the Language and its Actuarial Applications**, **APL as a Tool of Thought** proceedings, **I-APL publications and software**, **Boolean Functions and Techniques**, **The FinnAPL Idiom List**, **The Toronto APL Toolkit**, **Mathematical Experiments on the Computer**, **Probability in APL**, **APL - Stat: Do it yourself guide to computational statistics**, **A Source Book in APL** - Approximately 80 titles in all!

We also carry J publications, including **Programming in J**, **An Implementation in J (structure and source code)**, **Arithmetic**, and **Calculus**.

Shareware and commercial APL interpreters and J2 interpreters are available as well.

Some Basic Functions for Composite Arithmetic

by Neville Holmes (Neville.Holmes@utas.edu.au)

Abstract

This paper briefly describes proposals for a standard general purpose arithmetic called *composite* arithmetic, gives special attention to the register form of that proposal, reviews some of the basic functions needed for that arithmetic, compares those functions to the practice followed by APL and J, and urges the APL/J community to support and contribute to the establishment of a standard for composite arithmetic.

Introduction

That a recently published proposal for a standard for general purpose arithmetic[1] should have an APLish flavour is not a coincidence. The proposal followed directly from dissatisfaction with the ideas expressed in an earlier paper published in Quote-Quad[2] which itself arose from a consideration of extensions made by J[3] to the system traditionally used by APL to represent numbers.

The proposal also embraces a viewpoint on automatic calculation which characterizes APL and other typeless programming notations, the viewpoint which holds that users of a notation should not have to make decisions about how numbers shall be represented that could be better made automatically.

The composite arithmetic proposal goes further than this in holding that calculations that can be done exactly should be done exactly, and if they can't then the trustworthiness of their results should be made plain. In this, the proposals go beyond APL's present compass.

Adherence to these principles means that the representation of numbers and the machinery that is needed to work on those representations are more complex than say the traditional so-called floating point representations and machinery[4]. But composite arithmetic can also be more reliable under the kind of extreme conditions that the users and designers of APL interpreters take pride in being able to deal gracefully with. And improved technology now makes such complexity practical.

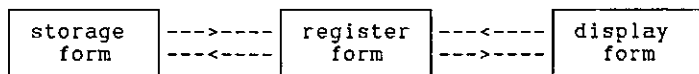
The purpose of this paper is to outline the central area of the composite arithmetic proposal, and to disclose some of its difficulties, in the hope that APL people might bring their very particular experience in the area to improve the proposal and to promote work towards getting a general purpose arithmetic standard adopted.

Composite Arithmetic

The proposal for composite arithmetic proposes three forms for the representation of numbers, where by *forms* is meant sets of closely related formats.

1. *Storage form* is for the efficient storage of numbers in data and program files. Binary encoding is used.
2. *Register form* is for the effective storage of numbers undergoing arithmetic. Binary encoding is used in a long accumulator format [5].
3. *Display form* is for the clear and truthful display and input of numerical values. Character encoding of a variety of decimal representations is used.

Since all arithmetic is carried out notionally on register form values, all conversions between forms are as shown in the following diagram.



The main concern of this paper is to consider the operation of some of what to APL people are primitive functions, otherwise to be known as arithmetic operations. The results that can be given by these functions are limited by the means available to represent those results in register form. Therefore the main explication of this paper must be preceded by a short description of register form.

The proposed composite arithmetic anticipates that results, though they are always produced in register form, will normally be stored in files in much shorter storage form[6], and presented to users, and keyed in by them, in the much more familiar display form[7]. Register form, being much longer, is not expected to be used normally for the long term storage of results.

Values are encoded binarily in register form, but, for the purposes of explanation here, a decimal encoding is needed for use in explanations. In what follows, it is therefore convenient to use the proposed composite arithmetic's display form

conventions in giving examples and values. The following summary merely gives a basis for understanding display form, in advance of explanation of different kinds of values as they are needed in describing register form. A summary of display form is given in a companion article in APL Quote-Quad [7] prepared before this one.

Display Form

The approach suggested for the display form of composite arithmetic is to suppose a poor character set (such as ASCII or EBCDIC) and a rich character set (such as is easily supported by modern printers), and to define alternative usages for the two. Because this paper is being keyed in on an ASCII machine, only the poor alternative is used here.

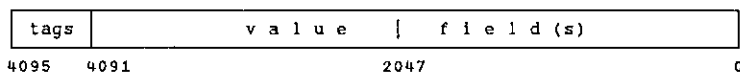
The decimal digits are the basis for display form. However, composite arithmetic requires that credible digits be distinguished from dubious digits. In this paper the underline character signals the onset of dubiety. Thus 1234 represents an exact value. However, 1234 represents an uncertain or inexact value, but guarantees that it is somewhere in the range 1184 to 1284 including both the extremes and all the integral and nonintegral intervening values.

Beyond the digits themselves, certain punctuation marks are used to compose number representations. These are shown in the following table alongside examples of their usage.

^	negative sign	^1	negative one
.	fraction point	0.1	one tenth
r	denominator point	1r2	one half
k	scaling up point	1k2	one million
m	scaling down point	1m2	one millionth

Register Form

Register form comprises four formats, all of the same length, namely 4096 binary digits. Which format is used depends on the kind of value loaded into it from one of the other forms, or from a calculation. The arithmetic chooses the format, not the programmer.



The first four bits of register form are used for tags. The first two tag bits specify the format in use, along the following lines. (The original proposal [1], for

simplicity of presentation, described only three formats, subsuming the integer within the rational.)

00	exact	integer	2s complement
01	exact	rational	signed
10	inexact	fractional	2s complement & signed
11	inexact	logarithmic	2s complement & signed

An integer of format 00 takes up all the bits that aren't tag bits, all 4092 of them, but the other three formats have a separation point in the middle of the form, so that for format 01 the low order denominator is 2048 bits long and for the other formats the same low order 2048 bits are used for the fractional part of the representation. The numerator of format 01 is 2044 bits long, and the sign of the rational value is kept in the fourth tag bit.

Between them, formats 00 and 01 allow exact arithmetic to be applied to a great range of values, typically to produce exact results. Whether or not such results can be put into exact storage or display forms is another question, but the arithmetic is without error of calculation on such values. The provision of a rational form means that even divisions are carried out exactly for typical exact divisors and dividends.

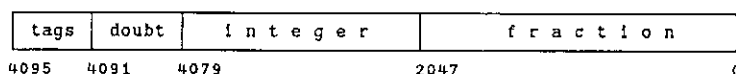
Certain special values are standardly represented. Exact zero and two infinities, represented strictly as 0 1r0 and ^1r0 in display form, are coded in the third and fourth tag bits of format 00. Indefinity, a value including arithmetic indeterminacy [8] and represented strictly as 0r0 in display form, is coded in the third tag bit, and the arithmetic sign is coded in the fourth tag bit, of format 01. Other special values are coded into inexact formats, but inexact formats are more complex, and need particular explanation.

The sheer size of register form means that the majority of inexact arithmetic, the arithmetic customarily carried out on so-called floating-point representations [4], is actually carried out on very long fixed point representations, and so has the potential to give very accurate results.

It is perhaps worthwhile to review here the difference between accuracy and precision, given that the terms are often confused [9]. Precision is about how exact the representation of a number is, whereas accuracy is about how well the representation expresses the value it represents. Thus, 3.24159 is precise to five decimal places, but as a value of π it is not even accurate to one decimal place.

Inexact Formats

Two register formats are proposed to hold inexact values. These formats are more complex than the exact formats because they are designed to keep a record of accuracy. For this, both formats include a pointer to the last credible bit in the value, that is, to the rightmost bit of completely reliable value. The other bits are here called "dubious". Thus the arithmetic is required to keep track of reliability. Note that the dubious bits are not negligible because they mark the centre of the interval of unreliability. (The original proposal [1], again for simplicity, only mentioned reliability tracking in passing, but its importance warrants its inclusion in composite arithmetic from the beginning.)



Both the inexact register formats hold signed values with a fractional part, and in twos complement form.

The first inexact format (10) keeps its value directly, the second (11) keeps its value as its binary logarithm. The long accumulator design of the register [5] means that the vast majority of inexact arithmetic can be done in the first format, and the second format is called into use only when results from the first format overflow or threaten to lose accuracy.

Certain special inexact values need to be held in register form. The first such class of special values is that of the inexact zeroes, and the second class is that of the reciprocals of such values. These two kinds of values are effectively intervals which include zero or infinity. The third tag bit signifies which class, and the inexact infinities are represented by their reciprocal, which is apt because they are usually generated by an inexact zero divisor. An interval including both positive and negative values is called "double", the other kind "single".

These special inexact values arise, because inexactness is being acknowledged and kept track of, when all significant digits are dubious. The arithmetic intended for inexact values is a kind of interval arithmetic [10]. The proposed standard does not constrain the arithmetic to this particular kind of arithmetic, but does require any arithmetic operation to maintain at least as much accuracy as arithmetic carried out directly on register form would. An implementation could include better arithmetic, but the arithmetic implied by the register format as defined is a kind of hybrid of conventional range arithmetic [11] and significance arithmetic [12].

In inexact register formats, the full value normally gives the centre of the interval, and the doubt pointer indirectly gives the width of the interval. In the primary format the centre is the arithmetic mean of the ends of the interval, in the secondary format it is normally the geometric mean. Inexact infinities in either format, and all double intervals in secondary format, need exceptional representations which it is not relevant to review here.

The Arithmetic

The results of any arithmetic, that is, of the application of any arithmetic operations to register form values, is straightforward for straightforward values. But there are two broad classes of values that are not straightforward, the inexact and the special.

An examination of the arithmetic of certain operations, then, is largely an examination of the results when they are applied to inexact and to special values.

However, there are some general rules. Indefinity yields indefinity. Otherwise, inexactness yields inexactness. Otherwise, exactness yields exactness.

Of course, the application of such rules depends on the particular arithmetic operation. The arithmetic operations are what APL has traditionally called primitive functions. Since it is usual in discussions of computer arithmetic to talk of instructions operating on data, the APL tradition is inappropriate here, but it is useful to contrast *monadic* operations and *dyadic* operations.

Just what operations are provided in a standard is a matter that can only be decided after considerable discussion. Basic monadic operations like negation and magnitude and maybe reciprocal are necessary, and the dyadic operations of addition, subtraction, multiplication, and division are also necessary. And there are many other possibilities which would spring to the mind of an APLer.

There are other operations which might be considered necessary to deal with some of the aspects peculiar to composite arithmetic, in particular to inexact representations.

Operating on Instructions

A primary requirement is to be able to test whether a value is exact or not. But this test should not be considered in isolation. Given that testing is needed, what tests are needed beyond exactness?

The traditional tests result from comparison of two values, and are basically tests of whether the first value is equal to, or is greater than, the second, or some

combination of these two and their opposites. For exact values in composite arithmetic, these tests are straightforward, and could easily be specified and provided. But it is not so for inexact or special values. It is extremely unlikely that one instance of `_1` (not negative, but entirely dubious, being in display form a particular value somewhere between `^4` and `6`) would equal another, or that `_1` would equal `_01`, and it is at the very least debatable whether `1r0` even equals itself.

The way around this is to revert to the idea that comparison is fundamentally testing the nature of the result of a subtraction of the two values being compared. Under composite arithmetic there is a need to be able to test for different kinds of values, and comparison can then be subsumed under such testing as simply the special case of testing the result of a subtraction.

Values can be different in kind along three dimensions, namely

- exact/inexact
- positive/negative/signless
- normal/indefinite/infinite/zero,

where "signless" is meant to include the ambiguity of a double interval of uncertainty in the case of an inexact zero or infinity. These three dimensions are not truly orthogonal since some combinations cannot occur, for example, an exact signed zero. It is maybe also necessary to be able to test whether a value is primary or secondary, but this is somewhat redundant.

In the case of comparison, now, the subtraction of two normal values would yield an exact zero if they were both exact and equal, and an inexact zero if either or both were inexact and the two were possibly equal.

It is of interest to note that there is not, in composite arithmetic as described here, any need for the comparison tolerance familiar to APL users. Comparison tolerance is fairly complex because it is a regularly implied doubt intermittently applied. In composite arithmetic, the comparison of two exact values is, appropriately, exact. If at least one of the values is inexact, then the actual tolerance is not absent but implicit in the result.

Monadic Arithmetic Operations

Only the relationship between one value and its result need be considered for monadic operations in composite arithmetic. In this sense they are simpler than

dyadic operations. Of main interest is the behaviour of different kinds of values rather than of any specific value.

Negation and Magnitude

Negation and magnitude operations are closely related. In both, negative values are converted to positive values of the same magnitude. Otherwise, for negation positive values are turned to negative ones of the same magnitude.

Indefinity and exact zero are unchanged by the operations. Exact values yield exact results, except only for the lowest possible integer value, whose positive equivalent cannot be represented exactly in register form. Interestingly enough, this maximum negative integer value would have to be converted straight to secondary inexact format because it would be much too big for the primary format.

Inexact values yield inexact values with unchanged doubt, again except only for the lowest possible primary value, which would have to be converted to secondary, or maybe to the maximum positive value with an extra bit of doubt.

Floor and Ceiling

Floor and ceiling monadic operations, familiar primitive functions for experienced APL people, have a lot of interest in composite arithmetic because they can change the very nature of a value, for example by converting an inexact value into an exact one. The two operations can be considered together, as the ceiling operation is only systematically different from the floor in a fairly obvious and consistent way.

Firstly, exact infinities and zero, indefinities, and integer values would all be unchanged. All rational values would be changed to integers. Some inexact values could be changed to integers, providing their interval did not have an integer value within it, and this could apply also to inexact zeroes with single intervals.

Inexact values whose doubt extends their interval across at least one integer, in particular all inexact zeroes with double intervals and all inexact infinities, would remain inexact, though their centre point might change. Indeed, just what the change of centre point in these cases should be could make an interesting study.

Centre and Tolerance

Operations that provide a breakdown of inexact values are needed, and the two obvious ones would give the centre of the interval of uncertainty and its radius (or tolerance). From these two values the limits of the interval can be calculated simply by addition and subtraction for all but inexact infinities.

Applied to exact values, the centre operation leaves the value of its operand unchanged, and the tolerance operation yields an exact zero. This can fairly arbitrarily apply also to indefinity and exact infinity. Inexact values never yield a tolerance of exactly zero, so taking the tolerance of a value also provides a check on whether it is exact or inexact.

For inexact values the centre operation will give an entirely credible value, that is, an inexact value with all its value bits credible. For primary inexact values this will merely involve changing the doubt to zero, but for secondary inexact values this involves calculating the arithmetic mean of the limits of the interval so that the limits can still be calculated by adding the tolerance to, or subtracting it from, the centre.

The tolerance of inexact values is derived from their doubt, and is itself entirely credible. It will either be an inexact zero, or a positive non-zero. Of interest, though, is the question of the circumstances under which the tolerance of a secondary inexact value should be given in primary format, as at least the tolerance of the centre of a secondary inexact value must be, since an entirely credible inexact zero cannot be represented in secondary inexact format.

Inexact zeroes, being always primary, can be handled the same as ordinary primary inexact values. Inexact infinities, however, must be anomalous, since their centres aren't any sort of mean of their limits. And what should the centre of a double infinity be? Probably the best result would be an indefinity. And if the centre of a single infinity is taken as the appropriately signed infinity (exact?), as it must be, the tolerance is infinite, and, even if it wasn't, the limits can't be calculated in the usual way because, apart from any other trap, subtracting the infinity of a tolerance from the infinity of a centre gives an indefinity, which is no kind of limit at all.

Dyadic Arithmetic Operations

The consideration of dyadic operations requires consideration of the results arising from combinations of various kinds of operands. The consideration is richer both because of the much greater number of cases of operand kinds, but

also because the nature of dyadic operations is typically more complex than the nature of monadic ones.

Higher and Lower

The higher and lower operations are very similar, and simpler than most other dyadic operations, so they serve as a useful starting point. These are sometimes called "greater" and "lesser", which is misleading, because these words really, in the ordinary meanings of these words, apply to the magnitude of operand values — clearly 1000000 is greater than 0.0001 even though it is lower in the number line. Linguistic considerations aside, only the higher operation need be considered here, since the operation of the lower can easily be inferred from it.

The result of higher is indefinite when an operand is indefinite. Positive infinity is higher than any definite value, and any definite value is higher than negative infinity. The higher of any two other exact values is standardly determinable simply from their relative position on the number line.

The result of higher when one operand is inexact and the other ordinarily exact is slightly more complicated but can be fairly easily pictured by imagining the two operands depicted on the number line. The exact value occupies a point, the inexact covers an interval. If the exact value is outside the interval and lower than it, then the inexact value is the result. If the exact value is outside the interval and higher than it, then the exact value is the result. If the exact value is inside the interval, then the exact value is the lower limit of the result and the upper limit of the inexact value is also the upper limit of the result.

In this last case a very important principle may be involved. While the lower and upper limits of the interval of the result may be specifiable and calculable, the interval may not be exactly representable in a register format. In this case the result must be the representable interval of least tolerance that encompasses the interval that can't be represented.

The combination of one exact and one inexact operand is closely analogous to the situation with two inexact operands where the tolerance of one of the operands is zero. The case of particular interest is where the two intervals overlap. Here the lower limit of the result is the higher of the lower limits of the two operands, and the higher limit of the result is the higher of the higher limits of the two operands. This is quite a general statement if exact value are considered to have two limits of the same value.

One remaining troublesome case remains, and that is where an operand is a double inexact infinity. The result is indefinite.

Addition and other Arithmetic

The considerations for the addition operation are somewhat the same as those for higher and lower. For the general case, the lower limit of the result is the sum of the lower limits of the two operands, and the higher limit of the result is the sum of the higher limits of the two operands. As for the higher and lower operations, particular behaviour is involved for particular kinds of operands, with double inexact infinities giving indefinite results, as also do adding infinities exact or inexact of opposite sign.

As for the subtraction operation, it is tempting to define it in terms of the addition of the negated right operand. This would be a mistake because then the subtraction by the largest negative integer would almost always give a secondary inexact result, whereas an exact result will often be more appropriate. Similarly, division should not be defined as multiplication by the reciprocal.

One group of operations could prove particularly useful for the manipulation of inexact values. These are the remainder, the quotient, and what might be called the bulk operation because it yields the product of the quotient and the divisor or modulator. The quotient is always an integer, and when the modulator and the divisor are exact then the bulk is also. The result of the bulk operation might be thought of as the extraction of as much of the dividend as can be put together from units of the modulator. The bulk plus the remainder reconstructs the dividend. However, these operations become complex when the modulator is inexact.

Conclusion

The paper outlining the proposal for using composite arithmetic as the starting point for a standard general purpose arithmetic also gives reasons why such a standard is needed to complement the highly successful binary floating-point arithmetic standard [1].

The floating-point standard is a relatively simple one, being mainly concerned with part of what is for composite arithmetic called storage form, and with the arithmetic to be carried out on values in that form [13,14].

Composite arithmetic needs three forms, and this paper has outlined the proposals for its register form, and for some of the arithmetic to be carried out on values in that form, a form on which the floating-point standard is silent [see also 15].

APL/J users and developers have long and most unusually relevant experience with the type-less style of calculation proposed for composite arithmetic. Should the APL community, particularly those members belonging to the IEEE Computer Society, agree that a standard like the one proposed would be of benefit in exposing and promoting the kind of computational approach that APL has for so long (and for so little popular result) promoted and embodied, then that community has an opportunity here to help build support for a standard for general-purpose computing, and to contribute significantly to making that standard as good as possible.

References

1. W.N. Holmes, *Composite Arithmetic: Proposal for a New Standard*, IEEE Computer, Vol.30, No.3, 1997 March, 65-73.
2. W.N. Holmes and C.B. Hallam, *The Philosophy of Nothing and Everything*, APL Quote-Quad, Vol.25, No.4, 1995 June, 75-82.
3. K.E. Iverson, *A Dictionary of APL*, ACM APL Quote-Quad, Vol.18, No.1, 1987 September, pp.5-40 (the beginnings of a distinct J).
4. D. Goldberg, *What Every Computer Scientist Should Know About Floating Point Arithmetic*, ACM Computing Reviews, Vol.23, No.1, 1991 March, pp.5-48.
5. U.W. Kulisch and W.L. Miranker, *The Arithmetic of the Digital Computer: A New Approach*, SIAM Review, Vol.28, No.1, 1986 March, 1-40.
6. W.N. Holmes, *Composite Arithmetic, A: Storage Form: A Proposed Draft Standard*, <ftp://ftp.comp.utas.edu.au/pub/nholmes/ca/dssf.ps>.
7. W.N. Holmes, *Composite Arithmetic and Number Display*, APL Quote-Quad, (submitted for publication).
8. H. Davies, *Infinity Arithmetic, Comparisons and J*, APL Quote-Quad, Vol.25, No.4, 1995 June, pp.28-34.
9. R.J. Fateman, *A Review of Mathematica*, Journal of Symbolic Computation, Vol.13, No.5, 1992, pp.545-579.
10. G. Alefeld and J. Herzberger, *Introduction to Interval Computations*, Academic Press, New York, 1983.
11. O. Abert and M.J. Schaefer, *Precise Computation Using Range Arithmetic, via C++*, ACM Transactions on Mathematical Software, Vol.18, No.4, 1992 December, pp.481-491.

12. L.H. Gray and J.C. Harrison, *Normalized Floating-Point Arithmetic with an Index of Significance*, in Proc. Eastern Joint Computer Conf., AFIPS, Vol.16, 1959, pp.244-248.
13. *ANSI/IEEE Std. 754-1985, Binary Floating-Point Arithmetic*, IEEE Press, Piscataway, N.J., 1985.
14. D. Stevenson, *A Proposed Standard for Binary Floating-Point Arithmetic*, IEEE Computer, Vol.14, No.3, 1981 March, pp.51-62 (accompanies other articles on the proposal).
15. W.N. Holmes, *Floating Point and Composite Arithmetics*, in Computational Techniques and Applications, ed. J. Noye et al., World Scientific, Singapore, 1998, pp.305-315
(see also <ftp://ftp.comp.utas.edu.au/pub/nholmes/ca/ctac97.ps>).

Forty Years On

by Sylvia Camacho

Granddaughter Gemma (2 years old) is usually full of sweetness and light but occasionally, at the second time of being told she may not have or may not do, she issues her 2-second warning:

'I CROSS NOW!'

before casting herself to the ground, drumming her heels and screaming at maximum volume.

Anthony and I were not able to go to Rome this summer (two more grandchildren imminent) so I looked forward to reading *Vector* 15.1 to get a feel for how the Conference went. I was very cheered by Stefano's upbeat Editorial and then by David Crossley's enthusiastic account of the optimistic news in the first two Plenaries. David continued, 'Then we had the Panel Session and a grey cloud descended.' How well I remember that cloud from all the Conferences I have been to: a series of whines that someone must be found to blame because APL does not have a mass following. Unbidden, into my head came the 2-second warning:

'I CROSS NOW!'

but at 65 years old, kicking and screaming is inappropriate. I decided that as a Grandmother with 40 years in The Computer Trade, I must write something SEVERE to go in *Vector*. Here it is.

I knew nothing of computers before 1957 when I started to work for a manufacturer. In those days choices between programming languages were easily resolved: there were no programming languages. The programmer encoded machine instructions, perhaps with the help of a mnemonic translator. The machine I first worked on was priced at £40,000 and I was valued at £500 p.a.

During the 1960s programming languages such as ALGOL, FORTRAN, PL1 and COBOL were developed and refined. IBM just missed the boat with the release of its own product PL/1 and so most mainframes were shipped with FORTRAN and COBOL. COBOL was written by the US Military and IBM was supported by the US Military. IBM could perhaps have made a commercial success of APL but only against competition from COBOL and by taking on the Academic Establishment in the teeth of the prejudice of mathematicians and IT departments.

During the 1970s machines became larger and faster without commensurate increase in price. I was trained in COBOL by my employer and learned the hard way that the requirement changes faster than the coders can work. At the end of

the 70s and in the 80s I was using mainframe APL*PLUS to give key Tesco staff interactive access to critical retail information distributed over private landlines but APL was constantly under attack for being too 'machine hungry' and expensive to rent. It was used only where there was support from Managers outside the Computer Department who were able to show a good return from fast development and flexible data access.

I retired a year and a bit early, in Spring 1992. I was still writing APL and earning £30,000 p.a. which would have valued the computer I started on at £2.4m at the same inflation rate. I now work freelance to support a PC-based 'legacy' system.

So I have been employable since 1957 despite using only COBOL and APL for 'serious' work. During these years I have seen many other much advertised database packages and so-called languages come and go. The major selling point for all of these has been that development times will be massively reduced. In no case have I found this claim to be justified. I have seen great benefit derived from word processors, spreadsheets and mailing networks but I have yet to find staff without programming experience able to set up anything but the simplest database system.

So my question for all the whingers is, 'What would be the advantage in making APL more like COBOL or PASCAL or PCEXPRESS or SMALLTALK or?' It has already shown itself to be more durable than any of these, perhaps with the exception of COBOL. I know that the source of the frustration expressed in these recurring 'Panel Sessions' has nothing to do with the rival merits of programming languages. It is a consequence of the Management structure of most large and medium sized companies. Expensive mainframes came to be managed by a Computer Department commonly divided between Operations and Programming. Success for Operations is to get the maximum throughput of programs and for Programming it is to produce systems error-free, to specification and to often arbitrary deadlines. Neither has any experience in assessing value for money in terms of the work to be done by the programs and both will measure their importance in terms of their budget and number of staff. Expertise in assessing value for money is held in the various user departments which generate profits for the business but it is extremely rare for a company to make computer personnel subservient to such users. APL threatens this common management structure and the Computer Department empire because it encourages a close co-operation between programmer and user.

It would be nice to think that the astonishing reductions in price and increase in power and data capacity of PCs will at long last give the whingers their quietus but this would be to misunderstand the relationship between hardware price and

programmer salary. Small employers (say £1m turnover or less) can easily afford a network of desktop machines and can make good use of word processor, spreadsheet and some graphical packages. The rub is that sophisticated systems incorporating databases can only be set up by professional programmers and their price is as high as or higher than 40 years ago. This is the challenge that ought to be addressed by Panel Sessions. If APL could be made as accessible to PC users as spreadsheets, freelancers like me would have a harder time. I have tried many times to introduce APL to those I hope will not be prejudiced by its reputation. I have had very little success. I-APL tried to popularise it in schools with a similar disappointing result. Why is it that children taught themselves BASIC when they found it distributed free on their home computers? I decided that with APL I was looking to persuade and that it would be better to imply that what I offer is simply 'there' for the reader or listener to take it or leave it – as is the case with a natural language: an apologetic or ingratiating tone is never going to intrigue. What follows is my latest attempt to answer the question, 'What is APL?'. *Vector* readers, I think, will find it too verbose but why not try it out on your non-APL friends?

Thinking, Speaking, Reading, Writing & Computing in APL

APL is a language. It is for thinking in. It can also be spoken and read and written. It is used to give instructions to a computer. Many different computers can interpret APL. Suppose I sit at a computer with an APL interpreter running. I am able to type and edit APL instructions. What will be going through my mind as I think APL?

Just now, for instance, I want to create pages of print. I have decided the paper size and the font (fixed space) I shall use so I know the maximum number of lines and the maximum number of characters per line that I can print on the page. I do not intend to calculate any other numbers in my head: I have a computer to do that.

The text for any particular page will be drawn from many different sources having varying numbers of lines and line lengths. Typically a page will have a Title above a Date above a Sub-title above a Statistical Table. I wish to arrange these components so that the horizontal centre of each is above the midpoint of all the others and the total composition is centred on the page.

I know that when I have set up an APL function to create such a result that I shall have something which can be used in innumerable other contexts for components of any number of lines and line lengths. Here I have only to think in simple terms about arranging one thing symmetrically over another thing. I need a name for

this function. I shall call it COVER which I think of as shorthand for 'Centre Over'. The Heading line of this function will read 'R gets A COVER B' written

```
R ← A COVER B
```

The next line I shall start with the comment symbol and type

```
⋄ R is variable A centred over variable B: SMC 4 Nov 1998
```

The comment symbol is called 'lamp' because comments should be illuminating.

A and B, left and right arguments to COVER, are set when the function is used. They are variables and APL variables are arrays and may contain any number of characters (or numbers) arranged in any number of dimensions. In a simple case such as COVER, A and B could be 'V' or '9' or '@' or '+' or a string like 'Statistical Tables' or could be a matrix of rows and columns of characters such as a statistical table itself might be. The job of COVER is to find out which of A and B is wider then to put spaces on either side of the other to make it equally wide and then to put one above the other.

In APL the shape (ρ) of a variable is held in addition to its contents. The APL response to the instruction ρ 'Statistical Tables' will be the number 18 because 'Statistical Tables' is a one dimensional object of 18 characters. However, if a variable T contains the table itself and supposing the table is a matrix of 22 rows each having 51 characters, the APL response to ρ T will be 22 51, this response itself being a 2-element numeric array of which one can also ask the shape. The response to $\rho\rho$ T ('what is the shape of the shape of T?') will be 2 which is the rank of T: that is to say T is a 2-dimensional array, a matrix.

A singleton, called a 'scalar', has no shape but it can be redefined to have a shape. ρ , '@' will be 1 because ' , ' ravel it to be a one-element string (known as a vector) whereas $1\ 1\rho$ '@' (read '1,1 reshape of at') will turn it into a 1 row 1 column matrix.

In order to compose R, the result of COVER, both A and B must be forced to be 2-dimensional: they must be reshaped as matrices. I think or say 'A gets negative 2 take of 1 catenate 1 catenate shape A reshape of A'. I write

```
A ← ( ~2 + 1,1,ρA)ρA
```

The instruction inside the parentheses takes the rightmost 2 numbers of 1,1 and ρ A and uses these two numbers to reshape A. Thus, if the shape of A were 18, A would be redefined as a 1 18 reshape of A: that is to say a single row matrix of 18

characters. Whereas if A were a scalar like '@' so that the result of ρA is null the numbers inside the parentheses will be just $\bar{2} \uparrow 1 \ 1$ which will turn '@' into a 1 row 1 column matrix as we have seen.

COVER needs to compare the width of A with the width of B. A and B have now both been reshaped as matrices so their width (number of characters in a row) is the last dimension of each. In APL this is 'WA gets the negative 1 take of the shape of the variable A' written $WA \leftarrow \bar{1} \uparrow \rho A$ and, of course, $WB \leftarrow \bar{1} \uparrow \rho B$. So what is the difference in width between A and B? Obviously it is $WA - WB$.

If WB is greater than WA the response to $WA - WB$ will be negative. This is inconvenient so I shall use the absolute value of this number, written $|WA - WB|$. This is the number of spaces we must add to the width of the smaller of A or B, half to the left hand side and half to the right hand side which is $(|WA - WB|) \div 2$ but the difference may not be an even number and so the result may not be a whole number. If so we must be content with a slight asymmetry and put one less space at either the beginning or end of the rows of the smaller matrix. So we write

$$N \leftarrow \lfloor (|WA - WB|) \div 2$$

which is read 'N gets the floor of the absolute value of the difference between WA and WB divided by 2' where 'floor' gives the next integer less than the result of the division. If instead I choose to write

$$N \leftarrow \lceil (|WA - WB|) \div 2$$

N will be the value of the integer next above the result (the ceiling, naturally). The extra space will then come on the left.

This is the point at which the function must decide which variable width is to be enlarged. The test is simply $WA > WB$ to which APL will respond 1 if it is true and 0 if it is false. The action consequent on the test is written $\rightarrow (WA > WB) \uparrow L1$ which reads 'Go to label L1 if WA is greater than WB'. By default, if $WA \leq WB$ the next instruction to be obeyed is taken from the next line.

The instructions on the line labelled L1: begin by transferring A unchanged to the result R. Then the width of B is adjusted to equal the width of A by adding trailing spaces to all its rows. N of these trailing spaces are then rotated (ϕ) to the left hand side of B and the new B is catenated to the bottom of R. As always with APL the fact that the shape of the variables can be queried is crucial to the method of adjusting the shape of B. Being a matrix B has a shape of two elements, number of rows, number of characters (columns). The number of rows is to

remain the same so this is $1 \uparrow \rho B$ but the number of columns must match A and this is $\neg 1 \uparrow \rho A$. Therefore, to make width A match with B we write

$$((1 \uparrow \rho B), \neg 1 \uparrow \rho A) \uparrow B$$

where $\uparrow B$ is read 'take of B'. Thus the whole action is

$$\begin{aligned} L1: R \leftarrow A \\ R \leftarrow R \neg (-N) \phi((1 \uparrow \rho B), \neg 1 \uparrow \rho A) \uparrow B \end{aligned}$$

The default where A is less than B is similar

$$\begin{aligned} R \leftarrow (-N) \phi((1 \uparrow \rho A), \neg 1 \uparrow \rho B) \uparrow A \\ R \leftarrow R \neg B \\ \rightarrow 0 \end{aligned}$$

where go to zero ends the function.

The function which I have written is therefore as follows. I would not normally include quite so many lines of comments. WA, WB and N are named in the Header line which means they are localised variables which are only included in the symbol table so long as the function is open which is from when it is invoked until its last instruction is complete or until $\rightarrow 0$. This has the advantage that the same variable names can be used in other functions. Variable names may be long and meaningful: there are no reserved words in APL. Below an example of the function in use follows the print of the function itself.

```

V R←A COVER B;WA;WB;N
[1]  A R is variable A centred over variable B: SMC 4 Nov 1998
[2]  A force A and B to be matrices
[3]  A←(⌊2+1,1,ρA)ρA
[4]  B←(⌊2+1,1,ρB)ρB
[5]  A get widths A and B and floor of half the difference
[6]  WA←⌊1+ρA
[7]  WB←⌊1+ρB
[8]  N←⌊(|WA-WB)÷2
[9]  A if width A greater than width B go to L1
[10]  →(WA>WB)⌊L1
[11]  A R gets A as wide as B with N of its trailing spaces rotated left
[12]  A B is then catenated beneath R and the function ends
[13]  R←(-N)ϕ((1↑ρA),⌊1↑ρB)↑A
[14]  R←R⌊B
[15]  →0
[16]  A R gets A, B is made as wide as A with N trailing spaces rotated left
[17]  A B is then catenated below R
[18]  L1:R←A
[19]  R←R⌊(-N)ϕ((1↑ρB),⌊1↑ρA)↑B

```

V

Now test it out. *BOX* is a function which does just what you might expect it to do.

```
T+BOX 22 51p(22x51) + 'This is where the table will go when I
have calculated it. '
PAGE+'Statistical Tables' COVER '29 November 1998' COVER
'Autumn' COVER T
pPAGE
27 51
```

This is a somewhat narrow page: if I want a wider one I can play a very pretty trick.

```
PAGE+(0 78p'')COVER PAGE
```

What I have done is to centre over *PAGE* an empty variable of zero lines and 78 columns thus forcing the page width to be 78 while leaving the number of rows unchanged.

```
pPAGE
27 78
```

```
Statistical Tables
29 November 1998
Autumn
```

<i>This is where the Statistical Table will go when I have calculated it.</i>
--

TECHNICAL SECTION

This section of VECTOR is aimed principally at those of our readers who already know APL. It will contain items to interest people with differing degrees of fluency in APL.

Contents

Hacker's Corner: APL meets MAPI	Jonathan Manktelow	70
Using VideoSoft FlexGrid Pro with APL+Win	Jonathan Barman	77
VideoSoft FlexGrid Pro with DyalogAPL	Jon Sandles	93
At Play with J: Crosswords and Life	Gene McDonnell	99
More Utilities for the Dyalog Session	Ray Cannon	107
DDE in APL+Win	Ajay Askoolum	111
From Fibonacci to Horodam	Joseph De Kerf	122

Hackers' Corner: APL Meets MAPI

by Jonathan Manktelow (jonathan.manktelow@nestlelb.nestle.com)

Whilst developing a multi-user scheduling system in Dyalog APL it became clear that it would be useful to send notification messages of various bookings, via e-mail, to other people on the network.

Nestlé have standardised on Microsoft Exchange so the obvious solution was to use Microsoft's MAPI (Messaging Application Interface) as the underlying messaging engine.

MAPI

Microsoft's MAPI is a software layer designed to sit between client software (such as our scheduling system) and the various mail systems and networks available (such as exchange or internet mail across NetBeui, TCPIP, Appletalk etc.). It is a standard abstraction layer available on all versions of Windows from 3.0 onwards. Microsoft have even declared parts of it a cross-platform standard, although I have not seen it running on anything other than Microsoft platforms yet.

As soon as I started to look at Microsoft's MAPI documentation it became clear that MAPI presents a number of different faces to anybody wishing to develop with it. (The overall structure of MAPI is shown in Figure 1 below.)

- Simple MAPI – The most basic set of API function calls available. This is an easy to understand set of DLL calls for performing various basic messaging tasks. However Microsoft's documentation states that this is only available for backwards compatibility, and all new applications should use CMC or COM messaging.
- CMC – A more up to date version of Simple MAPI that Microsoft would like us to use. Although this is more powerful than Simple MAPI it does not provide as much flexibility or control as COM Messaging.
- COM Messaging – Sometimes known as Active Messaging Library or OLE Messaging this is the version of MAPI implemented through Microsoft's COM (Component Object Model) framework. Using COM to access MAPI gives the programmer control over almost everything they will need. So much so that you could write a replacement for the Microsoft Inbox from within an OLE client such as Dyalog APL or +Win!

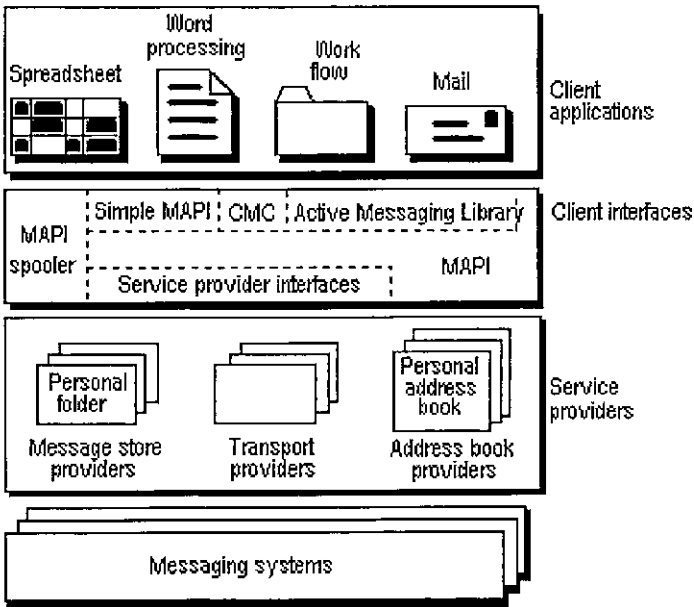


Figure 1 The Structure Of MAPI

As I discovered, leveraging existing API's and component libraries that are a standard part of the development platform can be a quick, easy and flexible way of adding sophisticated capabilities to your applications. Once you have found the documentation for the underlying engines...

The Utilities

Even though all I needed to do in the first instance was send a mail message, optionally with attachments, I wanted to build a utility set that could be extended in the future if necessary. I therefore chose to build the utility set on top of the COM Messaging MAPI interface. Which will allow the addition of more functionality, such as displaying all the addresses in the user's address book, simply by exposing more of MAPI through simple APL cover functions.

The utilities consist of 3 Mail functions and a windows API cover function in an APL namespace called `#.mail`.

- `#.mail.connect` connects the Dyalog session to MAPI

- *#.mail.disconnect* disconnects the Dyalog session from MAPI
- [*disconnect*] *#.mail.send_mail* [*subject*] [*message*] [*recipients*] [*attachments*] [*show_dialog*] sends some mail. This will connect and then disconnect from MAPI as needed.

The optional left argument (boolean) tells the function whether or not it should disconnect from the MAPI engine once the mail has been sent. The default of 0 will disconnect. If you need to send more than one mail message then you should not disconnect from MAPI between calls to *send_mail* to avoid slowing down your application.

You may specify as many of the arguments on the right as you need, but you must supply them in order, starting with *subject* and including all arguments up to the last one you need. i.e. if you want to specify the *recipients*, you must also specify the *subject* and *message*.

subject is a text vector.

message is a vector of text vectors containing the main text of the e-mail.

recipients is a vector of text vectors containing the names of all the people you would like to send the message to.

attachments is a vector a text vectors containing the fully qualified filenames of any attachments you would like to send with the e-mail.

show_dialog is a Boolean flag indicating whether or not the user should be shown the e-mail before it is sent. This defaults to 1 (do show the mail before it is sent.)

- *short_filename+#.mail.getshortpathname long_filename*
This converts a windows long_filename into its equivalent short filename. E.g. "c:\program files" becomes "c:\progra~1" This is necessary because MAPI only accepts short filenames.

The utilities have one configuration variable that you must specify in the *#.mail* namespace:

- *#.mail.profile* - This is a text vector containing the name of the exchange profile that should be used. For example 'Microsoft Exchange Profile'. This has been left as a configuration variable as it is likely to change from user to user.

With these functions you can add the ability to send e-mails from all of your Dyalog APL/W applications, with a single line of code. However there is still room for improvement! The functions do not return a flag to indicate whether or not the message was actually sent. If you can see a tidy way to do this, or of

adding any other features you feel would be useful please tell me. If there is sufficient interest I will put together another article developing this utility set further.

Other Information

This particular case only skims the surface of what is possible using MAPI. To find out more you should look at some of the following resources provided by Microsoft. My favourite is the on-line SDK documentation, as it is freely available to anyone with a web browser and an Internet connection.

- On-Line documentation library at <http://premium.microsoft.com/msdn/library/> is an excellent resource for technical information on any Microsoft product.
- Microsoft Technet - The subscription based source of CDs full of technical information, service packs etc. From Microsoft.
- The Dyalog APL/W 8.1 New Features help file for information on using COM/OLE objects from within Dyalog.

The Code

```
[0] Connect
[1]  a Connect to the MAPI server
[2]  :If #.mail.connected=0
[3]    '#.mail.mapi'[]NC'OleClient' 'MAPI.Session'
[4]    #.mail.mapi.Logon #.mail.profile
[5]    #.mail.connected+1
[6]  :EndIf

[0] disconnect
[1]  a Disconnect from a mail session
[2]  :If #.mail.connected=1
[3]    #.mail.mapi.Logoff
[4]    []EX+'#.mail.mapi'
[5]    #.mail.connected-1
[6]  :EndIf

[0] {disconnect}send_mail args;[]TRAP;message_text;subject_text;
    recipients_list; recipients_supplied;next_recipient;r;attachments_list;
    attachments_supplied; next_attachment;show_dlg
[1]  a Send an e-mail. The user will be asked for the details of the mail
[2]  a The arguments are subject text to files display
[3]  a where subject is a text vector
[4]  a message is a vector of text vectors, or a text vector
[5]  a to is a vector of text vectors of people to send the mail to.
[6]  a files is a vector of text vectors of valid file names
```

```

[7]  A display is a boolean indicating whether or not to show the send box
      before sending the message
[8]  A You can supply 1 ... 5 of the arguments, but they must be in order!
[9]  A Set up the error handling
[10]  [TRAP+<(0 1000)'E' '+[lc+1'
[11]  A Make sure that the system is connected to the mail server
[12]  #.mail.connect
[13]  A Extract the arguments
[14]  :If 1spargs
[15]      subject_text->args[1]
[16]  :Else
[17]      subject_text+''
[18]  :EndIf
[19]  :If 2spargs
[20]      message_text+<(a,[AV[3],w)/>args[2]
[21]  :Else
[22]      message_text+''
[23]  :EndIf
[24]  :If 3spargs
[25]      recipients_list->args[3]
[26]      recipients_supplied+1
[27]  :Else
[28]      recipients_supplied+0
[29]  :EndIf
[30]  :If 4spargs
[31]      attachments_list->args[4]
[32]      attachments_supplied+1
[33]  :Else
[34]      attachments_supplied+0
[35]  :EndIf
[36]  :If 5spargs
[37]      show_dlg->args[5]
[38]  :Else
[39]      show_dlg+1
[40]  :EndIf
[41]  A Create a new message
[42]  '#.mail.mapi.outbox'[NS #.mail.mapi.Outbox
[43]  '#.mail.mapi.outbox.messages'[NS #.mail.mapi.outbox.Messages
[44]  r+'#.mail.mapi.outbox.messages.message'#.mail.mapi.outbox.messages.Add 0
[45]  A Set the subject
[46]  :If [NC'#.mail.mapi.outbox.messages.message.subject'
[47]      #.mail.mapi.outbox.messages.message.Subject+subject_text
[48]  :Else
[49]      #.mail.mapi.outbox.messages.message.subject+subject_text
[50]  :EndIf
[51]  A Set the message text
[52]  :If 0=[NC'#.mail.mapi.outbox.messages.message.text'
[53]      #.mail.mapi.outbox.messages.message.Text+message_text
[54]  :Else
[55]      #.mail.mapi.outbox.messages.message.text+message_text
[56]  :EndIf
[57]  A If there were any supplied then set the recipients
[58]  :If recipients_supplied=1
[59]      '#.mail.mapi.outbox.messages.message.recipients'[NS
          #.mail.mapi.outbox.messages.message.Recipients

```

```

[60]   :For next_recipient :In \recipients_list
[61]       r+="#.mail.mapi.outbox.messages.message.recipients.Add>recipients_list
        [next_recipient]
[62]   :EndFor
[63]   a If the user does not want the dialog box showing, resolve the names
[64]   :If show_dlg=0
[65]       r+="#.mail.mapi.outbox.messages.message.recipients.Resolve 0
[66]   :EndIf
[67] :EndIf
[68] a Add any attachments that are required
[69] :If attachments_supplied
[70]   '#.mail.mapi.outbox.messages.message.attachments'\0NS
        #.mail.mapi.outbox.messages.message.attachments
[71]   :For next_attachment :In \pattachments_list
[72]       r+='#.mail.mapi.outbox.messages.message.attachments.attachment'
        #.mail.mapi.outbox.messages.message.attachments.Add 0
[73]       #.mail.mapi.outbox.messages.message.attachments.attachment.Position+0
        a End of text
[74]       #.mail.mapi.outbox.messages.message.attachments.attachment.Type+1
        a File data
[75]       #.mail.mapi.outbox.messages.message.attachments.attachment.Name+
        φ(-0+>\'\ '=φ>attachments_list[next_attachment])/
        φ>attachments_list[next_attachment]
[76]       #.mail.mapi.outbox.messages.message.attachments.attachment.
        ReadFromFile #.mail.getshortpathname>attachments_list
        [next_attachment]
[77]   :EndFor
[78] :EndIf
[79] a Send the mail
[80] #.mail.mapi.outbox.messages.message.Send 1 show_dlg
[81] a Tidy up
[82] □EX+'#.mail.mapi.outbox'
[83] a If we have been asked to disconnect - default= disconnect
[84] :If 0=□NC'disconnect'
[85]     disconnect+1
[86] :EndIf
[87] :If disconnect=1
[88]     a disconnect from the server
[89]     #.mail.disconnect
[90] :EndIf

[0] short_filename=getshortpathname long_filename;func;r;buffer_length
[1] a Cover for the windows get short filename function
[2] buffer_length+1000
[3] 'func'□NA'U4 kernel32.dll.C32\GetShortPathNameA <0T >0T U4'
[4] r+func long_filename buffer_length(buffer_length+1)
[5] short_filename+>r[2]

```

OpenGL Workspace & Russian Student Fund

Under the inspiration and guidance of Alexander Skomorokhov, Alexei Zalivine, a Russian Information Processing student in Obninsk, has written about 150 functions to access the OpenGL API functions under Microsoft Windows using Dyalog APL/W version 8 (32 bit). Included in the workspace are a number of utility functions to facilitate the use of the APIs as well as an interactive tool to experiment with the graphic image. For a more detailed description, please see the April, 1998 issue of *Vector*, pages 128-142.

To make it possible for Alexei and some of his colleagues to attend APL99, a Russian Student Fund has been established. For \$99 you will receive a copy of the software and the *Vector* article. \$50 will be contributed to the Fund and the remainder will compensate Alexei for his work (capitalism, you know) and cover the distribution costs. For \$150, the book, *OpenGL Superbible*, by Wright and Sweet, will be included as well. Additional contributions to the Fund are, of course, most welcome.

Please send a check (preferred in US) or money order in \$US made out to Renaissance Data Systems, or credit card information to:
Renaissance Data Systems; P.O. Box 421; Georgetown, CT 06829

Additional information may be posted to our web site, www.aplbooks.cnchost.com. Additional information on OpenGL may be found at www.opengl.org.

VideoSoft FlexGrid Pro with APL+Win

by Jonathan Barman

Introduction

A couple of months ago VideoSoft released Version 6.0 of their FlexGrid Pro control. It contains a mass of new features which make the standard Microsoft FlexGrid Version 5 look a very pedestrian affair. The Help About button of the Microsoft version says "Portions of this product were developed by VideoSoft", so I assume that this really means that Microsoft took VideoSoft's previous version and made a few modifications so that they could call it their own.

The major problem with the standard FlexGrid is that you cannot edit cells in place. Although it is reasonably straightforward to place an edit box over a cell so that it looks as though you are editing it, I never actually did that as the alternative grid TrueGrid (or the Microsoft Data Bound Grid) allows in-cell editing. Now that FlexGrid Pro allows in-cell editing and has so many seductive features I will be using it much more in future.

The thing I particularly like about VideoSoft is that they publish fully working software on the Internet (www.videosoft.com). If you haven't paid for it then you get an annoying message every time you run it. This means that you can download the OCX and really get down to business to see if you want to use it.

This is a short exposition of the FlexGrid Pro features using APL+Win.

Creating a Basic Grid

In order to play with the features we need a basic grid populated with some data. I usually define `io+0` as all the FlexGrid properties are index origin zero.

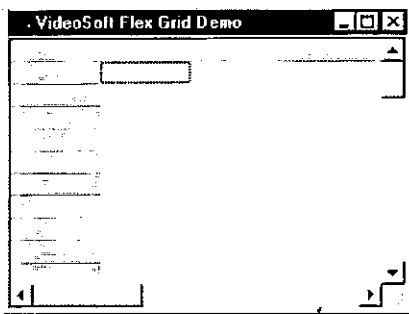
```
v vsFlexGridDemo;headings;data
[1] a Generate headings and some data
[2]   headings←'Region' 'Product' 'Type' 'Sales'
[3]   data←(4/'France' 'Germany' 'UK'),(12ρ2/'Gold' 'Silver'),
        (12ρ'I' 'E'),[1.5]*"212ρ200
[4] a Create the form and the grid
[5]   DemoForm←'fmFG'□w1'New' 'Form' 'Close' ('scale' 4)
        ('caption' 'VideoSoft ...')('where' 0 0 20 40)
[6]   DemoGrid←□wself+'fmFG.vsfgDemo'□wi
        'New' ':-) VideoSoft FlexGrid Control'
```

```
[7]      A Make sure the grid fills the form
[8]      DemoForm [wi 'onResize' 'FGonResize'      A ...
```

The *FGonResize* function just contains the statement

```
DemoGrid [wi 'where' (0 0, 2+[warg])
```

At this point the grid is displayed with the default settings of 50 rows and 10 columns with a fixed row and a fixed column:



Now assign the headings and data:

```
[9]      A Define the heading row and cols
[10]     [wi 'xRows' 1
[11]     [wi 'xFixedRows' 1
[12]     [wi 'xCols' (1+pheadings)
[13]     [wi 'xFixedCols' 1
[14]     [wi 'xColWidth' 0 300
[15]     A Assign the headings
[16]     [wi (c'Set'), c[2](c'xTextMatrix'),0,(pheadings)
           ,[1.5]headings
[17]     A Align the heading cells, from 0 1 to 0 4
[18]     [wi 'xCell' flexcpAlignment 0 1 0 ,(pheadings),
           flexAlignCenterCenter
[19]     A Add rows using the Add Item method which needs
           a tab between each element
[20]     [wi "(c'XAddItem'),'c',/tcht,"data
```

APL+Win requires that you prefix any OCX control properties with an *x*, methods with a *X* and events with *onX*. Rather annoyingly, I found that that I could not assign values to the *Cell* property without specifying all the arguments. In VB the syntax is (setting, R1, C1, R2, C2)=value and you can omit the R2 and C2 parameters for a single cell and the R1 and C1 parameters to assign the current cell. Also, in theory, I should have been able to assign the headings using

`wi 'xCell' flexcpText 0 1 0 4 (~1+εheadings,"tctl),` but for some reason this does not seem to work properly for multiple columns.

I use global variables containing the constant parameters as it is much easier to read `flexAlignLeftCenter` rather than trying to remember what a 4 means. The constants and their values are in the help text so it is very easy to copy and paste them into a function which assigns all the variables.

We have now got a grid with some data that we can play with:

VideoSoft Flex Grid Demo			
Region	Product	Type	Sales
France	Gold	I	117
France	Gold	E	33
France	Silver	I	5
France	Silver	E	138
Germany	Gold	I	152
Germany	Gold	E	142
Germany	Silver	I	15
Germany	Silver	E	117
UK	Gold	I	185
UK	Gold	E	185
UK	Silver	I	86
UK	Silver	E	101

There are a few more things to improve the functioning of the basic grid. The `AllowUserResizing` property allows the user to resize rows, columns or both. The `ExplorerBar` property allows the user to click in the header to sort a column, as in Explorer, and to move columns around. The options are None, Move, Sort and Sort and Move.

```
[21]  # Allow column and row resizing
[22]  wi 'xAllowUserResizing' flexResizeBoth
[23]  # Allow user to sort and move columns around.
[24]  wi 'xExplorerBar' flexExSortAndMove
```

I also like to use the keyboard to move rows and columns around and to insert and delete rows. Implementing the `KeyPress` event is quite simple:

```
[25]  # Key Press event to move rows and columns about
[26]  wi 'onXKeyDown' 'FGvsonKeyDown'
```

The functions needed to implement the event are as follows:

```

▽ FGvsonKeyDown
[1]  a on Key Down event handler for vs Flex Grid
[2]  :select □warg
[3]  :case 37 4 a Alt-Left
[4]      GridMove -1 'xCol' 'xFixedCols' 'xCols' 'XColPosition'
[5]  :case 38 4 a Alt-Up
[6]      GridMove -1 'xRow' 'xFixedRows' 'xRows' 'XRowPosition'
[7]  :case 39 4 a Alt-Right
[8]      GridMove 1 'xCol' 'xFixedCols' 'xCols' 'XColPosition'
[9]  :case 40 4 a Alt-Down
[10]     GridMove 1 'xRow' 'xFixedRows' 'xRows' 'XRowPosition'
[11]  :case 45 4 a Alt-Insert
[12]     DemoGrid □wi 'XAddItem' ((4p□tcht),'0')
[13]     (DemoGrid □wi 'Row')
[14]  :case 46 4 a Alt-Delete
[15]     DemoGrid □wi 'XRemoveItem' (DemoGrid □wi 'Row')
[15]  :endselect
▽

```

Notice that the *AddItem* method allows us to set the default values for the new row directly.

```

▽ GridMove a;move;posNm;minNm;maxNm;fnNm;newpos;curpos;
  min; max;□wself
[1]  a Move a grid row/column
[2]  a 5 element vector of Move, Position name, Min name,
      Max name, Function name
[3]  (move posNm minNm maxNm fnNm)+a
[4]  □wself+DemoGrid
[5]  a Current position and new position
[6]  newpos+move+curpos+□wi posNm
[7]  a get min and max
[8]  (min max)+□wi"minNm maxNm
[9]  a Wrap around
[10]  :if newpos<min ◊ newpos+max+newpos-min ◊ :endif
[11]  :if newpos>max ◊ newpos+min+newpos-max ◊ :endif
[12]  a move
[13]  □wi fnNm curpos newpos
[14]  a Set current position
[15]  □wi posNm newpos
▽

```

In-cell Editing

The default value for the *Editable* is False, so we have to set it before any editing can be carried out. However, this allows any sort of rubbish to be typed into the cells. If we want to limit some of the columns to a list then we can set the *ColComboList* property to provide a combo or list drop-down. There are many options. You can choose between a combo or a list drop-down, you can translate the data from codes to text and you can have multiple column lists and highlight one of the columns. In this short article there is not enough room to explore all the alternatives. One flaw is that the pipe character used to separate elements cannot be altered, so none of the list elements can contain a pipe character.

```
[27]  A Allow editing
[28]  [wi 'xEditable' ~1
[29]  A Restrict the Product and Type to lists
[30]  [wi 'xColComboList' 2 'Gold;Silver;Platinum;Iridium'
[31]  [wi 'xColComboList' 3 ('I', [tcht, 'Import;E', [tcht,
    'Export')
```

Note the use of the split bar, as APL+Win seems to hijack the vertical bar. The horizontal tab `[tcht` is the separator for multiple columns in the drop down list.

We can now use drop-down lists in the Product and Type columns.

Region	Product	Type	Sales
France	Gold	I	117
France	Gold	E	33
France	Silver	I	5
France	Gold	E	138
Germany	Silver	I	152
Germany	Platinum	E	142
Germany	Iridium	I	15
Germany	Silver	E	117
UK	Gold	I	185
UK	Gold	E	185
UK	Silver	I	86
UK	Silver	E	101

Region	Product	Type	Sales
France	Gold	I	117
France	Gold	E	33
France	Silver	I	5
France	Silver	Import	138
Germany	Gold	E Export	152
Germany	Gold	E	142
Germany	Silver	I	15
Germany	Silver	E	117
UK	Gold	I	185
UK	Gold	E	185
UK	Silver	I	86
UK	Silver	E	101

If you need to do more elaborate work you can set the *ColComboList* property to an ellipsis. A button is displayed in the cell and when the user clicks it the control fires the *CellButtonClick* event, at which point you can do clever things. The following is a contrived example, just to illustrate the facilities rather than doing anything useful, where we pop up an edit box just below the cell.

```

[32]  a Pop up an edit box when user clicks country
[33]  DemoEdit+ [wi ':edCountry.New' 'Edit'
        ('visible' -1)('onUnfocus' 'FGedonUnfocus')]
[34]  [wi 'xColComboList' 1 '...'
[35]  [wi 'onXCellButtonClick' 'FGvsonCellButtonClick'

      v FGvsonCellButtonClick;top;height;left;text
[1]  a Pop up edit box when user clicks button
[2]  a Find out where to place edit box and what to put in it
[3]  (top height left text)+DemoGrid [wi 'Ref' 'xCellTop'
        'xCellHeight' 'xCellLeft'('xCell' flexcpText)
[4]  DemoEdit [wi 'Set' ('visible' 1)('text' text)
        ('where' (top+height),left)
[5]  DemoEdit [wi 'Focus'
      v

```

Because the user can resize the rows and the columns it is essential to access the *CellTop*, *CellHeight* and *CellLeft* properties to get the cell position. A fully working system should also check that there is enough room to display the pop-up in the intended position.

The simple way out of the edit box is for it to hide itself as soon as it loses focus. Generally, you would want to have OK and Cancel buttons where the operation was sufficiently complex to warrant a pop-up.

```

      v FGedonUnfocus;t;[wself
[1]  a Edit box has lost focus
[2]  t+DemoEdit [wi 'text'
[3]  DemoEdit [wi 'visible' -1
[4]  [wself+DemoGrid
[5]  [wi 'xText' t
      v

```

There are many more features for controlling editing. A check box can be displayed for Yes/No type columns. The *EditMask* property allows detailed control over the characters to be input. The editing properties can be set on a cell by cell basis as well as for complete columns. There is a full set of before, after and validate events so that detailed control can be exercised over each cell. There is a separate set of key down/up/press events which fire when edit mode is active, so a different set of routines can be coded to control in-cell editing.

You have to watch out when coding the before and after edit routines if you allow the user to move columns about. For example, if editing was not allowed in the Sales column then the code would have to look like this:

```

v FGvsonBeforeEdit
[1] :if 'Sales' = DemoGrid [wi 'xTextMatrix' 0, warg[2]
[2]   wres[3] + 1
[3] :endif
v

```

Merge Cells

Cells may be merged in a number of ways. We start by allowing a merge on the left hand 3 columns and then specifying that all items in those columns may be merged:

```

wi                                     (c'Set'), (c'xMergeCol'), "(13), ""-1
wi 'xMergeCells' flexMergeFree

```

The following shows what happens before and after moving the Type column:

VideoSoft Flex Grid Demo				
Region	Product	Type	Sales	
France	Gold	I	117	
		E	33	
	Silver	I	5	
Germany		E	138	
	Gold	I	152	
	Silver	E	142	
UK		I	15	
		E	117	
	Gold	I	185	
		E	185	
	Silver	I	86	
		E	101	

VideoSoft Flex Grid Demo				
Region	Type	Product	Sales	
France	I	Gold	117	
	E		33	
	I	Silver	5	
Germany	E		138	
	I	Gold	152	
	E	Silver	142	
UK	I		15	
	E		117	
	I	Gold	185	
	E		185	
	I	Silver	86	
	E		101	

If we now restrict the merge then the lower order columns are not merged:

```

wi 'xMergeCells' flexMergeRestrictAll

```

The following shows the results before and after sorting the Type column by clicking in the header:

Region	Type	Product	Sales
France	I	Gold	117
	E	Gold	33
	I	Silver	5
	E	Silver	138
Germany	I	Gold	152
	E	Gold	142
	I	Silver	15
	E	Silver	117
UK	I	Gold	185
	E	Gold	185
	I	Silver	86
	E	Silver	101

Region	Type	Product	Sales
France	I	Gold	117
	E	Silver	5
Germany	I	Import	152
	E	Export	15
UK	I	Gold	185
	E	Silver	86
France	E	Gold	33
	E	Silver	138
Germany	E	Gold	142
	E	Silver	117
UK	E	Gold	185
	E	Silver	101

I particularly like the ability to change groups of merged cells in one operation.

Rows can be merged in exactly the same way as columns. Also, the merging can be restricted to the fixed columns or rows so that you can have pretty headings without having to merge the data.

Finally, the *flexMergeSpill* setting allows a spreadsheet style of merge where long items spill into the next column if it is blank:

Region	Type	Product	Sales
France	I	Gold	117
Long country name which spills			0
France	I	Silver	5
Germany	I	Gold	152
Germany	I	Silver	15
UK	I	Gold	185

Outlining

This is best demonstrated on the original data grid. First we have to specify what outlining we want and clear any existing outline:

```

wi 'xOutlineBar' flexOutlineBarComplete
wi 'xSubTotalPosition' flexSTAbove
wi 'XSubtotal' flexSTClear

```

The *Subtotal* method has 10 arguments and all but the first are optional. The last statement above illustrates a call to the method specifying the first, mandatory, parameter which is the required action.

Each of the following *Subtotal* calls specifies:

- Action, in this case *flexSTSum*
- Grouping column, or `-1` for a grand total
- Column to be summed
- Format string
- Background colour
- Foreground colour
- Bold

The remaining items can be left at their default values.

The grand total is added by specifying `-1` as the grouping column.

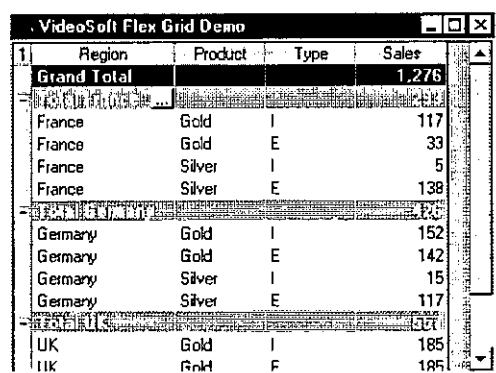
```
Dim 'XSubtotal' flexSTSum -1 4 '#,' 1 (256130255) -1
```

VideoSoft Flex Grid Demo			
Region	Product	Type	Sales
Grand Total			1,276
France	Gold	I	117
France	Gold	E	33
France	Silver	I	5
France	Silver	E	138
Germany	Gold	I	152
Germany	Gold	E	142
Germany	Silver	I	15
Germany	Silver	E	117
UK	Gold	I	185
UK	Gold	E	185
UK	Silver	I	95
UK	Silver	E	101

Each of the next groupings are specified by column numbers starting from the left. By using the last two of the 10 arguments to the *SubTotal* method you can specify groupings across arbitrary sets of columns.

The next set of totals is indicated by a dark grey background.

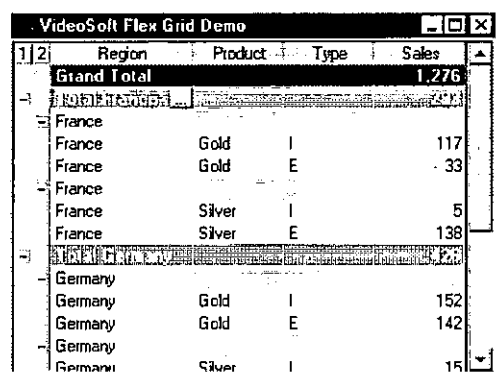
```
❑wi 'XSubtotal' flexSTSum 1 4 '#',(25613 2p64 255),-1
```



Region	Product	Type	Sales
Grand Total			1,276
France	Gold	I	117
France	Gold	E	33
France	Silver	I	5
France	Silver	E	138
Germany	Gold	I	152
Germany	Gold	E	142
Germany	Silver	I	15
Germany	Silver	E	117
UK	Gold	I	185
UK	Gold	E	185

Finally, the Products are grouped with a pale grey background.

```
❑wi 'XSubtotal' flexSTSum 2 4 '#',(25613 2p192 255),0
```



Region	Product	Type	Sales
Grand Total			1,276
France	Gold	I	117
France	Gold	E	33
France	Silver	I	5
France	Silver	E	138
Germany	Gold	I	152
Germany	Gold	E	142
Germany	Silver	I	15
Germany	Silver	E	117
UK	Gold	I	185
UK	Gold	E	185

The user can then click on the outline symbols to collapse or expand the totals:

The screenshot shows a window titled "VideoSoft Flex Grid Demo". Inside is a grid with columns: Region, Product, Type, and Sales. The grid is expanded to show a hierarchy starting with "Grand Total" (Sales: 1,276), followed by "France", "Germany" (expanded to show "Silver" products with sales of 15 and 117), and "UK". On the left side of the grid, there are small symbols (plus and minus signs) used to expand or collapse the rows.

	Region	Product	Type	Sales
1 2	Grand Total			1,276
+	France			
+	Germany			
-	Germany	Silver	I	15
-	Germany	Silver	E	117
+	UK			
+	UK			

Instead of getting the method to generate totals (with *flexSTSum*) you can specify Percentage of Total, Count, Average, Max, Min, Standard Deviation and Variance.

If you don't want totals inserted then there is an *RowOutlineLevel* method which allows you to specify the outline level of every row to get the precise effects that you need.

Speed

There are several techniques for posting data into the grid. If you want to display big APL arrays in the grid then some speed test are required to see which is best.

The *AddItem* method adds a row, but requires tabs between each column together with a leading tab if there is a fixed column. The *TextMatrix* and *Cell* properties are designed for assigning data on a cell by cell basis. The *Clip* property allows a bunch of selected cells to be assigned in one go, but you have to have a character vector with tabs between columns and new line characters between rows.

The following are some little timing functions to see which method is fastest.

```

v FGTimeAddItemEach data;rows;cols
[1] (rows cols)+pdata
[2] wi 'xRedraw' 0
[3] wi 'xCols' (1+cols)
[4] wi "(<<'AddItem'),'<<','/tcht','data
[5] wi 'xRedraw' 1
v

v FGTimeAddItemLoop data;rows;cols;i;t
[1] (rows cols)+pdata
[2] wi 'xRedraw' 0
[3] wi 'xCols' (1+cols)
[4] t+(wi 'xFixedCols')p<tcht
[5] :for i :in rows
[6] wi 'AddItem'(t,1+<tcht,"data[i;])
[7] :endfor
[8] wi 'Redraw' 1
v

v FGTimeTextMatrixLoop data;rows;cols;r;c
[1] (rows cols)+pdata
[2] wi 'xRedraw' 0
[3] wi 'Set' ('xRows' (1+rows))('xCols' (1+cols))
[4] :for r :in rows
[5] :for c :in cols
[6] wi 'xTextMatrix' r c (>data[r;c])
[7] :endfor
[8] :endfor
[9] wi 'Redraw' 1
v

v FGTimeClipAssign data;d;rows;cols;blocks;i
[1] (rows cols)+pdata
[2] d+(1+','/tcht,"data),"/tcnl
[3] wi 'xRedraw' 0
[4] wi 'Set' ('xRows' (1+rows))('xCols' (1+cols))
[5] blocks+rows[1000+1[rows+1000
[6] :for i :in (-1+0,blocks),"blocks
[7] wi 'XSelect' (1+1>i) 1 (2>i) cols
[8] wi 'XClip' (<d[(1>i)+1--/i])
[9] :endfor
[10] wi 'XSelect' 1 1
[11] wi 'xRedraw' 1
v

```

```

      v r←FGTime data;t1;t2;⊞wself
[1]  a Time various methods of loading Flex Grid with data
[2]  ⊞wself←DemoGrid
[3]  a Set up the result
[4]  r←1 2p'Method' 'Seconds'
[5]  a Add item method. Use Each
[6]  ⊞wi 'Set' ('xRows' 1)('xCols' 1) a Reset grid
[7]  t1←⊞ts ◊ FGTimeAddItemEach data ◊ t2←⊞ts
[8]  r←r⌊('AddItem Each')(Seconds t2 t1)
[9]  a Add item method. Loop to avoid WS Full
[10] ⊞wi 'Set' ('xRows' 1)('xCols' 1) a Reset grid
[11] t1←⊞ts ◊ FGTimeAddItemLoop data ◊ t2←⊞ts
[12] r←r⌊('AddItem Loop')(Seconds t2 t1)
[13] a Text Matrix property in loop
[14] ⊞wi 'Set' ('xRows' 1)('xCols' 1) a Reset grid
[15] t1←⊞ts ◊ FGTimeTextMatrixLoop data ◊ t2←⊞ts
[16] r←r⌊('TextMatrix Loop')(Seconds t2 t1)
[17] a Select cells and assign Clip property. Max size 500k
[18] ⊞wi 'Set' ('xRows' 1)('xCols' 1) a Reset grid
[19] t1←⊞ts ◊ FGTimeClipAssign data ◊ t2←⊞ts
[20] r←r⌊('Clip assign')(Seconds t2 t1)
      v

```

I have cheated slightly in the *FGTimeAddItemEach* function as it should really check to see how many fixed columns there are rather than assuming that there is exactly one. The job is done properly in the *FGTimeAddItemLoop* function. The loop in the *FGTimeClipAssign* function is required because any attempt to assign more than 500K in one go results in a GPF. The 1000 rows is just a guess that the columns won't be bigger than 512 bytes on average.

The results on a 1000 by 10 array are as follows:

Method	Seconds
AddItem Each	0.33
AddItem Loop	0.44
TextMatrix Loop	2.36
Clip assign	0.22

Obviously, APL is slow when looping like crazy assigning the *TextMatrix* property on every cell. Interestingly, the Flex Grid help says that assigning data using the *TextMatrix* property should be faster than using the *AddItem* method. However, the winner is selecting the required range and assigning the *Clip* property.

Running the timings on a bigger array shows that the looping version of *AddItem* is slightly faster than using *Each*, but the winner is still the assignment to the *Clip* property:

```
FGTime *"(100x10000)".+110
Method           Seconds
AddItem Each      4.99
AddItem Loop      4.45
TextMatrix Loop   23.35
Clip assign       2.75
```

My initial efforts indicated that the *Clip* assignment was much the slowest at 58.5 seconds! An analysis of the problem showed that it was line [8] that was causing all the delay. The Editor kindly pointed out that if I changed '*xClip*' ($\Rightarrow, /d[(1>i)+1--/i]$) to be '*xClip*' ($\Rightarrow d[(1>i)+1--/i]$) then the times would be dramatically improved. It just goes to show that you have to know your version of APL to get the best out of it.

Other Things

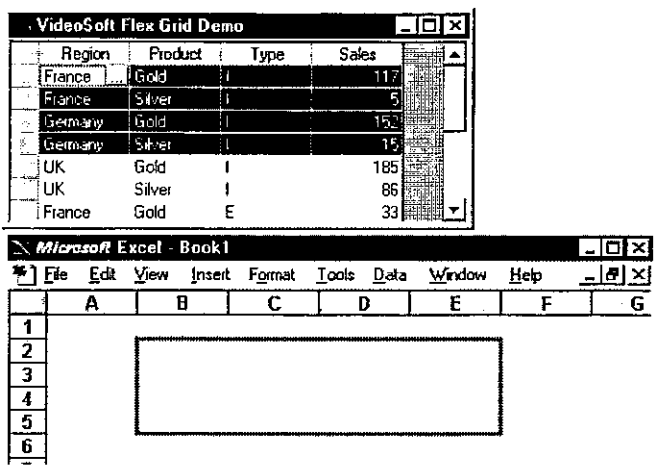
The *Autosearch* property allows the user to search for an item by typing the first few letters of the text. You can set it to search from the top every time or from the cursor. This enables you to use the grid like a sophisticated list box.

The *AutoSize* method and the *AutoSizeMode* property allow you to fit column widths or row heights to the data. It is easy to get two-row headings in this way:

```
⎕wi 'xSelect' 0 1 0 4
⎕wi 'xClip' (1+⎕tcht, "'Region name' 'Product code'
              'Import / Export' 'Sales'"))
⎕wi 'xWordWrap' 1 ⌘ True
⎕wi 'xAutoSizeMode' flexAutoSizeRowHeight
⎕wi 'XAutoSize' 1 4 ⌘ Columns to be sized
```

VideoSoft Flex Grid Demo				
Region name	Product code	Import / Export	Sales	
France	Gold	I	117	
France	Gold	E	33	
France	Silver	I	5	
France	Silver	E	138	
Germany	Gold	I	152	
Germany	Gold	E	142	
Germany	Silver	I	15	
Germany	Silver	E	117	

OLE Drag and Drop enables the user to highlight cells and drag them into another application. The following is a picture of where I am in the process of dragging a group of cells into Excel:



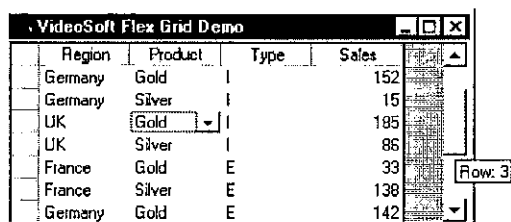
You can display a tool tip on the scroll bar by setting the *ScrollTips* property to True and assigning a function to the *BeforeScrollTip* event, which in turn assigns a value to the *ScrollTipText* property. For example:

```

wi 'xScrollTips' 1
wi 'onXBeforeScrollTip' 'FGvsonBeforeScrollTip'

v FGvsonBeforeScrollTip
[1] DemoGrid wi 'xScrollTipText' ('Row: ',*warg)
v

```



You can save the contents of the grid to a file using the *SaveGrid* method and then get it back again using the *LoadGrid* method. There is also an *Archive* method which enables you to save a number of different files in single,

compressed, file. I found this particularly useful when putting together a little demo which needed the contents of several grids to be saved together.

Copy and Paste are nicely implemented, though Paste always seems to need extensive code to check that the values being pasted are all OK.

Failures

There are several things that I have not been able to get to work from APL+Win, largely through lack of time and experience.

It ought to be possible to put icons on a grid using the *Imagelist* control, but I could not work out how it should be done.

There does not seem to be a *tooltip* property for the grid. If there was I could have assigned the following function to the *MouseMove* event:

```

      v FGVsonMouseMove;pos;wself
[1]  a Display tool tip on mouse move
[2]  wself+DemoGrid
[3]  pos+w 'Ref' 'xMouseRow' 'xMouseCol'
[4]  :if 0εpos
[5]    w 'tooltip' ''
[6]  :else
[7]    w 'tooltip' ('This is ',w 'xCell' flexcpText pos)
[8]  :endif
      v

```

I did not attempt to try and get the *BindToArray* method to work as I would not expect APL to know anything about the address of a VB array. It would be marvellous to have this facility in APL as the array expands and shrinks in line with the size of the grid and you can link several grids together in useful ways. It is also very fast.

These little failures would seem to be an excellent set of topics for the Hackers' Corner in *Vector*! Is there anyone out there who can fix the problems?

Conclusion

I am an enthusiastic convert to this OCX which is stuffed full of facilities that save coding time. I have no doubt that it might be possible to replicate most of the features in APL, but why bother when it is all available?

VideoSoft FlexGrid Pro with DyalogAPL

by Jon Sandles

Introduction

Following on from Jonathan Barman's article on the FlexGrid I decided to write an article which should highlight the way using this ActiveX control in DyalogAPL differs from using it in APL+Win. I used DyalogAPL 8.2 pre-release, and over time I have experienced quite a lot of resource leaks and crashes, but as each pre-release came out the situation improved; I finally managed to reproduce the worst of the errors and sent the Dyadic helpdesk a diagnostic workspace. The last I heard, they had located the bug and it will be fixed in the final version of 8.2. (By the way, one of the best new features of 8.2 is that it does not GPF anymore! Instead, you get a syserror:999! Plus, the added bonus is that you (normally) get an aplcore, which means you haven't lost all your changes and there is something to send the helpdesk. Fantastic!)

The basic structure of my article follows Jonathan's APL+Win one to ease comparison.

Creating a Basic Grid

Dyadic's ActiveX control support is quite similar to APL+Win's in that it attempts to integrate with the existing Windows GUI facilities (`□WC/S`).

```

▽ vsFlexGridDemo;headings;data;form;fg
[1]  ⍝ Create a class wrapper called FlexGrid
[2]  'FlexGrid'□WC'OCXClass' ':-) VideoSoft FlexGrid Control'
[3]  headings←'Region' 'Product' 'Type' 'Sales'
[4]  data←(4/'France' 'Germany' 'UK'),
      (12p2/'Gold' 'Silver'),(12p'I' 'E'),[1.5]⌘?12p200
[5]  form←'fm'
[6]  ⍝ Create a form
[7]  form □WC'form'('coord' 'pixel')
      ('caption' 'Videosoft FlexGrid Demo')
[8]  ⍝ Put the grid in it
[9]  fg←form,'.FG'
[10] fg □WC'FlexGrid'('posn' 0 0)('size'(form □WG'size'))

```

DyalogAPL requires you to setup the reference to the ActiveX control before you create any instances of it (see line [2]). A resize handler is not required as DyalogAPL's default behaviour is to resize the control automatically to fit the

form. At this point the grid appears in exactly the same way as it would in APL+Win at the same stage.

Continuing in roughly the same way as Jonathan did with APL+Win:

```
[11]  fg □WS'FixedRows' 1
[12]  fg □WS'FixedCols' 1
[13]  A Prepare for the headings
[14]  fg □WS'rows' 1
[15]  fg □WS'cols'(1+pheadings)
[16]  A Assign the headings .. could not resist a dynamic fn
[17]  (<fg)□WS''(⍝headings){('TextMatrix[0;',(⍝α),']')⍵}''headings
[18]  A Align the headings
[19]  fg □WS('Cell[',(⍝flexcpAlignment),';0;0;0;',(⍝pheadings),']')
      flexAlignCenterCenter
[20]  A Assign the data (□av[10] is tab, one at the front to
      skip the row labels)
[21]  3 □NQ''(<<fg),''(<<'AddItem'),''<'',/□AV[10],''data
```

DyalogAPL exposes an ActiveX control's properties, methods and events in the same way as a normal class, and in addition they are exposed as documented, unlike APL+Win. (So, no annoying x, X and onX prefixes.)

Like APL+Win, DyalogAPL also insists you must supply all the arguments to a property even if some are optional, but this is the least irritating thing about their implementation of properties that have arguments. These properties, which are really an array extension to the idea of a Visual Basic property, have been implemented using DyalogAPL's array syntax which must be passed with the property name as a string! (see line [17]). To explain what I mean I need an example. Consider the Visual Basic syntax for setting the text heading (row 0) of column 1:

```
TextMatrix(0,1) = headings(1)
```

Now consider the APL+Win syntax:

```
□wi 'Set' 'xTextMatrix' 0 1 (headings[1])
```

In DyalogAPL you need to do the following:

```
fg □WS 'TextMatrix[0;1]' (headings[1])
```

Which, to a certain extent, makes perfect sense. They have used the APL array notation to make it clear that this is an array property. The unfortunate effect is that the property and its array indices must be passed as a single string, which is

a huge overhead when looping and assigning to thousands of cells. It also leads to some ugly looking lines of code.

```
[22]  a Autofit the columns (set the font first!)
[23]  fg [WS'Font' 'MS Sans Serif' 8
[24]  3 [NQ fg'AutoSize' 0,phheadings
[25]  fg [WS'AllowUserResizing'flexResizeBoth
[26]  fg [WS'ExplorerBar'flexExSortAndMove
```

Here I have deviated a little bit from the script. On line [24] I take advantage of FlexGrid's autosize functionality (mentioned in the APL+Win article in the *Other Things* section). You should be sure to set the font of the data (potentially by cell, if you like!) before you do this. Notice that Dyadic make a clear distinction between properties and methods. Methods are 3 [NQ'ed to be invoked.

The rest of the main function is pretty much the same as in APL+Win:

```
[27]  fg [WS'Event' 'KeyDown' 'onFGKeyDown'
[28]  a This cannot be rejected but is shown as an example
[29]  fg [WS'Event' 'BeforeEdit' 'onFGBeforeEdit'
[30]  a Allow editing
[31]  fg [WS'editable' 1
[32]  a Restrict the product and type to lists
[33]  fg [WS'ColComboList[2]' 'Gold|Silver|Platinum|Iridium'
[34]  fg [WS'ColComboList[3]'
      'I',[AV[10],'Import|E',[AV[10],'Export'
[35]  a Popup an edit box when user clicks country
[36]  fg [WS'ColComboList[1]' '...'
[37]  fg [WS'Event' 'CellButtonClick' 'onCellButtonClick'
```

▼

In-Cell Editing

The implementation of *onFGKeyDown* is exactly as in APL+Win and is left as an exercise for the reader. I have also enabled editing (line [31]) and setup the combo boxes just as Jonathan did. On line [29] I have trapped the *BeforeEdit* event. This is the only reasonable way on the FlexGrid that you can make a cell read only. Jonathan gave an example of doing this in APL+Win where he rejected all editing of the Sales column. The equivalent code in DyalogAPL is:

```
▼ msg←onFGBeforeEdit msg
[1]  :If 'Sales'=(1>msg)[WG'TextMatrix[0;',(▼msg[4]),']]'
[2]      msg[5]+1
[3]  :EndIf
```

▼

But here I got a bit of a surprise. It doesn't work! The event is not rejected and the editing is allowed. After speaking to Dyadic, it turns out that the event handling from ActiveX controls is not yet fully featured and will be improved in the future. In particular, you cannot *reject* the events. This is a major failing of the current implementation and would probably force you into having to think up quite esoteric workarounds in order to use an ActiveX control like this in anger.

Merge Cells and Outlining

Once again the code for merging cells and outlining is pretty much the same as in APL+Win. The only differences to look out for are in the formatting of the array properties, and the use of 1 (rather than $\bar{1}$) to stand for boolean 'true':

```
fg ⍵WS'MergeCol[1]' 1
```

Another exercise for the reader!

Speed

I replicated Jonathan's speed experiments and got nearly identical results. Clip was by far the fastest method, and I also found I had to use *enlist* to get optimum performance. For some reason I found I had to reduce the size of the batches in the Clip routine to be a bit smaller than in Jonathan's, but nothing to worry about too much!

Other Things

I had no problems with any of the "Other Things" in Jonathan's review, until I tried the *Scroll Tips* example. The following code should enable a scroll tip to be shown as you scroll the grid up and down.

```
fg ⍵WS'ScrollTips' 1
fg ⍵WS'Event' 'BeforeScrollTip' 'onFGBeforeScrollTip'

vmsg←onFGBeforeScrollTip msg
[1] (1>msg)⍵WS'ScrollTipText'('Row : ',vmsg[3])
v
```

I found that this code didn't work at all. Sometimes the tip would appear and sometimes it wouldn't. It certainly never appeared as I was scrolling, which is where it is most useful.

If you trace the code you will find you get *all* the *BeforeScrollTip* events at once when the user lets go of the scroll bar. This is way too late! Once again, this is a result of Dyadic's incomplete handling of events from ActiveX controls. The events coming from the ActiveX control are just stuck on the message queue, and hence won't be processed until DyalogAPL gets a bit of idle time.

Failures

I have already mentioned a few major failures in my comparison with APL+Win:

- Incomplete event handling makes it difficult to do several essential tasks (e.g. trapping *BeforeEdit* to make cells read only, and handling *BeforeScrollTip* events to display Scroll Tips).
- Strange syntax for setting "array" properties, is difficult to code with and could lead to inefficient code.

Just like Jonathan I had trouble with the following

- I could not work out how to set images on the grid. In Visual Basic the Image must support *IPictureDisp*. I wasn't sure how to achieve this from DyalogAPL.
- I too found that the 'ToolTipText' property was not exposed by the FlexGrid. Is this a standard property that Visual Basic adds on to the interface? (If so APL should add it on as well.)
- I also didn't even try to get *BindToArray* to work. And I agree: it would be great if it worked!

Some new ones ...

- I found that some of the properties that are standard to Visual Basic and to DyalogAPL were not exposed, for example, the *Visible* property. (This one is easy enough to code around as you can get hold of the window handle of the control.)
- There was some confusion with colour parameters. For example, the *BackColor* property is defined in the helpfile as being of type OLE_COLOR. DyalogAPL allows you to pass parameters defined as OLE_COLOR as RGB triplets so in this case:

```
fg WS'BackColor'(255 0 0)
```

results in a lovely red background.

Some of the other colour properties were not so clear:

```
fg WS'Cell[6;1;1;2;2]'(0 0 255)
```

Should result in a 2x2 rectangular block of blue cells in the top left hand corner of the grid. When I found it didn't work I checked the parameter and found that it was Variant (because the Cell property can be used to set a whole range of stuff). Incidentally, the 6 is another magic number to tell the grid which property you are setting; the list of properties appears in the help file, but to get the numbers you either guess, or install VB! Some of the useful ones are:

Alignment = 2
Background colour = 6
Foreground colour = 7
Font name = 11
Font size = 12
Font bold = 13
Font italic = 14

Anyway, I eventually found the following worked:

```
fg [WS'Cell[6;1;1;2;2]'](256+0 0 255)
```

The moral is: tread carefully with colour parameters!

- The final failure is on the FlexGrid side. With so much flexibility it was disappointing that the only way to get non-scrolling regions was by using the *FixedRows* and *FixedCols* properties. These are strictly non-editable. There is quite a common requirement for non-scrolling regions that are still editable. I solved the problem by providing a keyboard shortcut to lock a number of columns and another keyboard shortcut to unlock the columns. This means that the users can lock the columns, scroll to the cell they are looking for, and then unlock the columns. Not ideal.

As Jonathan pointed out it would make a great Hackers' Corner to try and present fixes to some of these problems. Please try! One possible solution is to write a thin ActiveX wrapper in Visual Basic (there is a wizard in VB 6.0 for doing precisely this) which contains workarounds for some of the problems. The one thing I tried this with was to try and pass the data to the ActiveX control in one array and use the Flex Grid's *BindToArray* method to populate the grid. I found that I couldn't pass a *VariantArray* from DyalogAPL (LIMIT ERROR!), so I gave up! I will continue to work on this technique and if I get good results, I promise to write up my experiences in the future.

Conclusion

The FlexGrid has a very simple interface that allows you to get very impressive results very quickly. If Dyadic get round to ironing out some issues with their event handling from ActiveX controls, then this sort of control will soon become as popular as, if not more popular than, their own native grid control.

At Play With J – Crosswords and Life

by Eugene McDonnell

Making Shift

To refresh your memory about J's shift verb, here is what the online *J Dictionary* says:

The phrase `x |.!.f y` produces a shift: the items normally brought around by the cyclic rotation are replaced by `f` unless `f` is empty (`0=#f`), in which case they are replaced by the normal fill defined under `{.` (take):

```
t=: 'abcdefg'
2 _2 |.!. '#' 0 1 t
cdefg##
##abcde
```

The right shift is the dyadic case of `|.!.f` with the left argument `_1`. For example:

```
|.!. '#' t
#abcdef
```

This article uses the shift verb in numbering the squares of crossword puzzles and in Conway's Game of Life. The use of the shift verb is similar in both: it applies to tables along both the columns and the rows. It also applies to the neighbours of an atom: those reachable by a one-square rook move in the case of the crossword puzzle, and those reachable by a one-square queen move in the case of Life.

All of the shifts are of amount one. A shift of one is *ahead* if the last item is shifted out, and a fill item is introduced at the beginning; that is, the direction of shift is toward the larger index. A 'shift one ahead' verb is thus:

```
soa =: |.!. ''
```

Notice that the fill item is empty (`''`), so that `soa` can be used with nouns of any type, with the default fill for that type being used. In our cases we shall be working with numeric data, so the fill item will be the number zero.

```

      L =: 2 3 4 5 6
      soa L      NB. shift the last item out
0 2 3 4 5
      T =. 1 2 3 +/ L
      T
3 4 5 6 7
4 5 6 7 8
5 6 7 8 9
      soa T      NB. shift the last item out
0 0 0 0 0
3 4 5 6 7
4 5 6 7 8

      soa"1 T     NB. shift the last item of each item (row) out
0 3 4 5 6
0 4 5 6 7
0 5 6 7 8

```

Similarly, a shift of one is *back* if the first item is shifted out, and a fill item is introduced at the end; that is, the direction of shift is toward the smaller index. A 'shift one back' verb is thus:

```

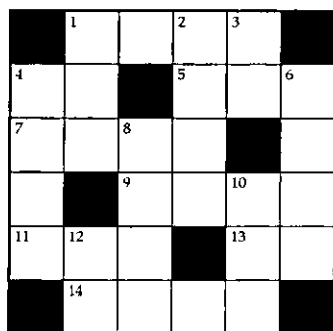
      sob =: 1&soa
      sob L      NB. shift the first item out
3 4 5 6 0
      sob T      NB. shift the first item out
4 5 6 7 8
5 6 7 8 9
0 0 0 0 0
      sob"1 T     NB. shift the first item of each item (row) out
4 5 6 7 0
5 6 7 8 0
6 7 8 9 0

```

A Crossword Problem

A crossword puzzle (the kind I am familiar with, that appear in the newspapers in the United States) consists of two tables of numbered clues, labelled 'Across' and 'Down', together with a usually square diagram containing regularly spaced rows and columns which divide it into smaller squares, some black, but most white, in a usually symmetrical pattern. Some of the white squares contain numbers, and these indicate the beginning of an infix of squares going across or down, where the letters of a word are to be written, and these are keyed to the clues. The numbers are in ravel, or row-normal order, beginning with 1.

Here is a small crossword puzzle diagram, with its clues:



Across

1. Competent
4. Direction
5. Australian bird
7. Ran away
9. Damn euphemism
11. Bath, for instance
13. That is
14. Leave out

Down

1. Common abbreviation
2. Helen's mother
3. Printer's measure
4. Phantasmic celestials
6. Tempt
8. Cheese
10. Small thing
12. Italian river

The pattern of a crossword puzzle can be represented by a matrix in which a white square is represented by a one, and a black square by a zero. For example:

```

M
0 1 1 1 1 0
1 1 0 1 1 1
1 1 1 1 0 1
1 0 1 1 1 1
1 1 1 0 1 1
0 1 1 1 1 0

```

A professional composer of such puzzles undoubtedly knows from experience which squares are to contain a clue number. An inexperienced computer has to be taught. We'll show how to teach a computer to number the squares, with the help of J's shift instruction.

In Exercise 1.3.2-23 of his book *Fundamental Algorithms*, Donald Knuth gives the rule as follows:

A square is numbered if it is a white square and either (a) the square below it is white and there is no white square immediately above, or (b) there is no white square immediately to its left and the square to its right is white.

Notice how Knuth carefully yet a bit awkwardly avoids saying 'black square above' and 'black square to the left', and instead says 'no white square'. I think this is because of the white squares in the top row and leftmost column. What is above or to the left of them? Shades of Jim Brown's empty array jokes! How can you tell whether what is above a white square in the top row is not a white square?

The problem is solved by shifting all of the one-square rook move squares to coincide with that square. For the border squares, this ensures that zeros, signifying 'black' squares, are shifted to coincidence. This is done in two steps, producing two matrices of the same size as the puzzle diagram matrix.

The verbs to solve the crossword numbering problem are:

```
f =: soa < sob
X =: ] *. f +. f"1
```

The use of `f"1` produces a one for each atom in the matrix in which the square to its left is less than the square to its right. This is true only if the square to the left is black (0) and the square to the right is white (1). This identifies the potential squares where an across word can begin.

The following use of the verb `f` produces a one for each atom in the matrix in which the square above it is less than the square below it. This is true only if the square above is black and the square below is white. This identifies the potential squares where a down word can begin.

The verb `X` *ors* these two results, producing a matrix showing all squares satisfying either of these tests (the same square can be 1 in both tests). This combined square is *anded* with the original square, yielding a result which has a 1 only where there is also a 1 in the original, thus definitely identifying squares to be numbered as the beginning of either an across or a down word.

These steps are summarized as follows:

M					:	(f M)					:	(f"1 M)					:	[]*.f+.f"1)M				
0	1	1	1	1	0	1	1	0	1	1	1	1	0	0	0	0	0	0	1	1	0	
1	1	0	1	1	1	1	0	0	0	0	1	1	0	0	1	0	0	1	0	1		
1	1	1	1	0	1	0	0	1	0	0	0	1	0	0	0	0	0	1	0	0		
1	0	1	1	1	1	0	0	0	0	1	0	0	1	0	0	0	0	0	1	0		
1	1	1	0	1	1	0	1	0	0	0	0	1	0	0	0	1	0	1	0	0		
0	1	1	1	1	0	0	0	0	0	0	0	1	1	0	0	0	0	0	0	0		

The verb X encapsulates these steps. We obtain the final result by applying X to M:

```

c =: X M
c
0 1 0 1 1 0
1 0 0 1 0 1
1 0 1 0 0 0
0 0 1 0 1 0
1 1 0 0 1 0
0 1 0 0 0 0

```

Knuth's exercise asks for a display of the puzzle using black and white squares made of plusses, minuses, and stiles. To complete the exercise we give with no further ado ...

```

clb=: [: +/\ [: *.\ \ ' '" _ = ] NB. count leading blanks

CWD=: monad define
NB. y. is boolean matrix giving a crossword puzzle pattern
NB. where 0 represents a black square and 1 a white square;
NB. yields crude numbered crossword puzzle diagram.
bw =. <' '+,\ '+'>:'+++++' NB. scalar white square
bb =. <'+++++','+++++','+++++' NB. scalar black square
g =. [: ,[: ,. / [: > ] NB. ravel stitch insert open
a =. , x y. NB. mark across and down
squares
    b =. (clb |.)"1 ":",.>:i.+/a NB. format & left adjust
numbers
    c =. <"2 b(<0;0 1)}"1 2>bw NB. insert numbers in blanks
    d =. cws0(bw;(.,y.)#a);<c) NB. insert blank white
squares
    e =. cws0(bb;(.,y.);<d) NB. insert black squares
    f =. g"1 ($y.)$e NB. stitch open of reshape
    '+,\.'+'+',(3 5*$y.)$,f NB. reshape and border )

```

When this is applied to M we get:

```

CWD M
+++++
+++++1 + +2 +3 +++++
+++++ + + + +++++
+++++
+4 + +++++5 + 6 +
+ + +++++ + + +
+++++
+7 + +8 + +++++ +
+ + + + +++++ +
+++++
+ +++++9 + 10 + +
+ +++++ + + + +
+++++
+11 +12 + +++++13 + +
+ + + +++++ + +
+++++
+++++14 + + + +++++
+++++ + + + +++++
+++++

```

Conway's Game of Life

In Knuth's *The Metafont Book*, Addison Wesley, 1986, Exercise 13.24, p. 121, and Answer 13.24, p. 245 are as follows:

Exercise 13.24: In John Conway's "Game of Life" pixels are said to be either *alive* or *dead*. Each pixel is in contact with eight neighbors. The live pixels in the $(n+1)$ st generation are those who were dead and had exactly three neighbors in the n th generation, or those who were alive and had exactly two or three live neighbors in the n th generation. Write a short METAFONT program that displays successive generations on your screen.

Answer 13.24: (We assume that `currentpicture` initially has some configuration in which all pixel values are zero or one; one means "alive.")

```
picture v; def c = currentpicture enddef;
forever: v := c; showit;
  addto c also c shifted left + c shifted right;
  addto c also c shifted up + c shifted down;
  add to c also c - v; cull c keeping (5 , 7); endfor.
```

(It is wise not to waste too much computer time watching this program.)

It is not apparent, but Knuth's approach assumes a flat universe, where, as is customary, if things are pushed beyond the edges, they fall off the surface. Many APL approaches, using the `rotate` verb, assume a toroidal universe, where the display is assumed to connect both horizontally and vertically, and if things are pushed beyond the edges, they wrap around, staying on the surface, and appearing on the opposite edge.

Assuming that you don't speak METAFONT, I'll explain the logic behind this program. It encodes the values of the pixel and its eight neighbours to indicate a pixel which is to be alive or dead in the next generation. The encoding gives a weight of one to the pixel and a weight of two to each of its neighbours. If we compute the sum of the pixel plus twice the sum of its neighbours, we can arrive at any of the eighteen values from zero through seventeen: zero for a dead pixel or one for a live pixel, plus 0, 2, 4, 6, 8, 10, 12, 14, or 16 for twice the sum of its neighbours, depending on whether the number of alive neighbours is 0, 1, 2, 3, 4, 5, 6, 7, or 8. Then "dead and exactly three live neighbours" is $(0+(2*3))$, or 6, and "alive and exactly two or three live neighbours" is $(1+(2*2))$, or 5, and $(1+(2*3))$ or 7, so we get the next generation by replacing each sum equal to 5, 6, or 7 by one, and all others by zero.

Knuth forms a new picture as the sum of the picture and the picture shifted one column left and the picture shifted one column right.

He adds this new picture to the new picture shifted down one and the new picture shifted up one, next doubles the value of each pixel, then subtracts the original pixel, giving us the value we desire. It only remains to see whether this is one of the life-giving values. That's what the cull verb does: it culls the chaff from the result, leaving only those having the value 5 or 6 or 7.

J verbs to do what Knuth's MetaFont program does are:

```
g =: ] + soa + sob
h =: ] -- [: +: [: g g"1
L =: h e. 5 6 7"_
```

The verb g will be used in somewhat the same way as the verb f in the crossword puzzle problem. It is used first (g"1) to form the sum of each column with the columns on either side, then applies g to this, forming the sum of each row with the rows on either side, doubles (+:) this, then subtracts (] --) the original picture. The verb h encapsulates this. The verb L uses h to go through these steps, then determines which atoms of the result are equal to either 5 or 6 or 7.

Given a picture:

```
picture =: 0 0 0 0 0,0 1 1 1 0,0 0 1 0 0,0 1 1 0 0,:0 0 0 0 0
```

Its original value and the next 5 steps of Life are:

```
q =: L^(1.6) picture
<"2 q
```

0	0	0	0	0	0	0	1	1	1	0	0	0	1	0	0	0	1	1	0	0	:0	0	0	0	0	0
0	1	1	1	0	0	1	1	1	0	0	1	1	1	0	0	1	0	0	1	0	0	0	0	1	0	0
0	0	1	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0
0	1	1	0	0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Utilities for the Dyalog Session (part 2)

by Ray Cannon (Ray_Cannon@compuserve.com)

In the first part of this series of articles, I described the code I use to implement pseudo "User Commands" via Dyalog's `⌈SE` namespace and APL code held in workspaces. In this article I will describe some of the utilities I use via this command processor.

All the Dyalog workspaces described in this series should be available on the *Vector* web site. The on-line versions will be the latest available, and will be added to over time, so may not exactly match what is written here. At the time of writing, the DWS will have been saved from Dyalog 7.3 (Windows 3.1) and have been tested under Dyalog 8.1 (Windows NT 4.0).

Utility Workspace UCMD\LATEST.DWS

This utility sorts every function (and operator) in the currently loaded workspace by its modified date, and then lists the fully qualified name (i.e. including its namespace path) of only those functions modified on the latest date found.

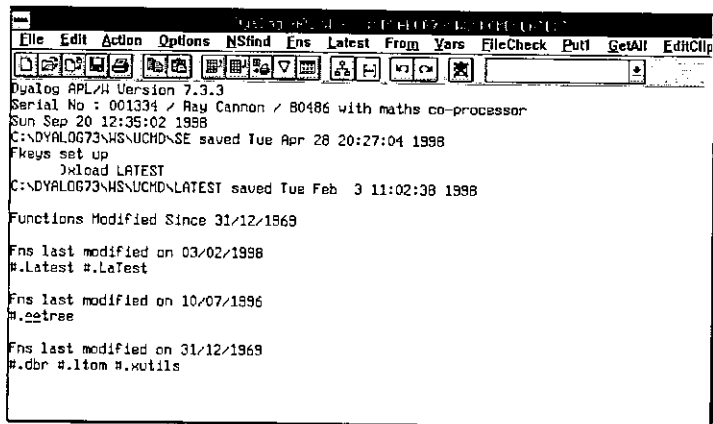
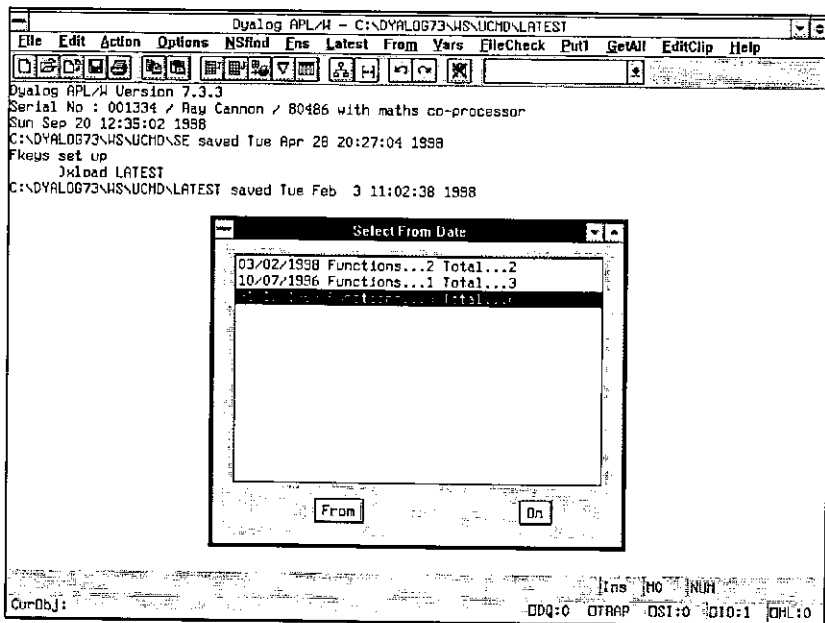
I find this very useful to show me either what functions I have modified today, or what functions were changed in the workspace when the last set of changes was made. Being a VERY lazy typist, to re-edit a function I have changed today, I will often click on the "Latest" menu option, and select the function to be edited by double clicking on one of the listed functions, rather than type in its name!

The utility uses the `⌈AT` system function to get the last modified date of each function / operator found in each namespace. The utility function "atree" is used to return a vector of namespaces making up the namespace tree.

Utility Workspace UCMD\FROM.DWS

This is a variation of LATEST. Here again the utility sorts every function by modified date, and then displays (in reverse chronological order) the date and (a) the number of functions last modified on this date, and (b) the total number of functions modified on or later than (from) this date.

The user then selects a date and presses either the "FROM" or "ON" button.



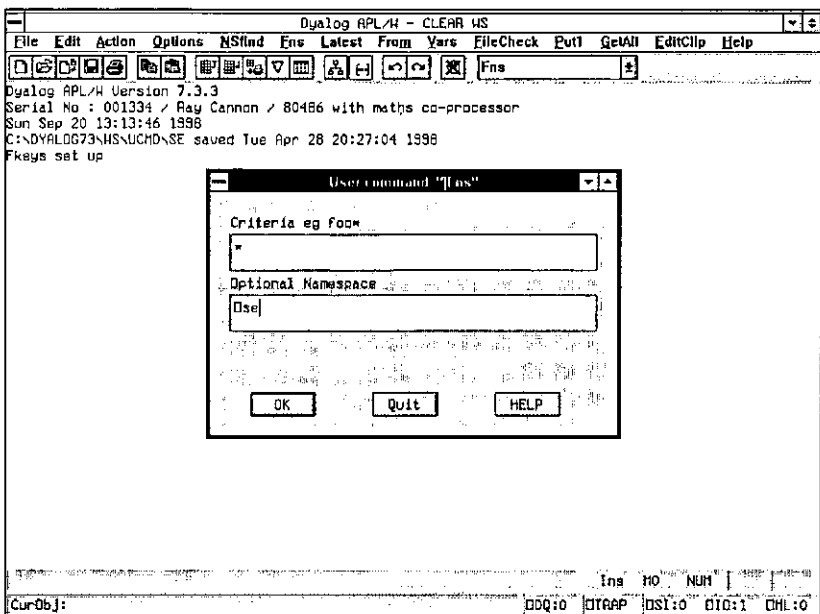
Utility Workspace UCMD\FNS.DWS

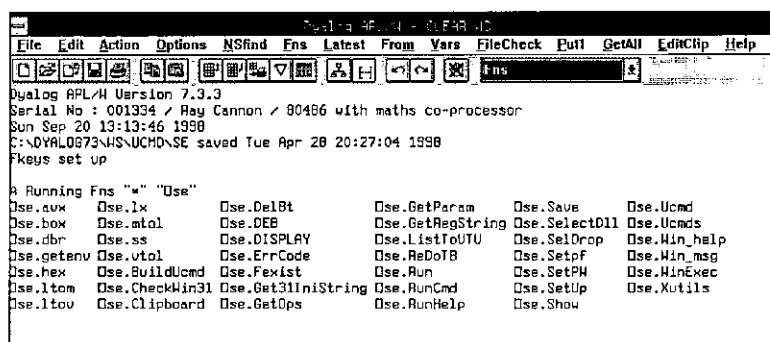
The utility *FNS* is based on the standard APL system command *)FNS* and is available in two forms, the niladic form via the menu bar and an ambivalent form via the drop down box on the tool bar. (Actually I find it more useful to return both Functions and Operators, not just functions.)

Unlike *)FNS*, the functions are listed in columns alphabetically down the page. The number of columns depends on the width of the page and the number of characters in each function name within that column. All rows are filled in every column except for the last.

The niladic form returns a list of all the functions in the current namespace of the currently loaded workspace without any selection. This is the same list as *⎕NL 3 4* would produce.

The ambivalent form requires a criteria selection (using * as a wild card e.g. *A*E* would select AGE and AXE but not ASS) and optionally a namespace. Unlike the niladic form, the resulting list is of fully qualified names. (i.e. including the namespace path).





Utility Workspace UCMD\VARS.DWS

The *VARS* utility is to *)VARS* as the *FNS* utility described above is to *)FNS* (need I say any more?).

Actually I have programmed my PFkey 12 to return :

```
[SE.Show [SE.UCMD 'Vars dbx_*'
```

I find this useful as I always start the names of my Causeway control variables with leading "dbx_" characters; pressing this PFkey will therefore list them all.

Utility Workspace UCMD\STORE.DWS

The commands *Store*, *Put1* and *GetAll* are related. You may use them to copy functions between workspaces or namespaces via the "store file".

The (menu bar) command *GetAll* retrieves all the objects currently "on file" and fixes them in the current namespace. This action "clears" the "store file".

The (menu bar) command *Put1* takes a copy of the current object and adds it to other objects (if any) already "on file".

The (combo box) command *Store* will add all the (user specified) objects to other objects (if any) already "on file". If no objects are specified, the list is automatically created via a "LATEST" command. (Useful for backing up all the changes made to functions in a workspace.).

Website address:

www.vector.org.uk/v151/quadse.zip

DDE in APL+Win

by Ajay Askoolum

Introduction

Dynamic Data Exchange (DDE) is a means of exchanging data between two *concurrently running* Windows applications which support DDE. DDE is 'old' technology, has inherent limitations and is no longer recommended for distributed applications. However, it can be very useful to the system developer for one off data acquisition.

This article demonstrates DDE conversations between APL+Win version 3.0 and Microsoft Access version 2.0. The discussion of DDE properties and Windows Application Programming Interface (API) should be relevant to other APL interpreters.

Starting Applications

DDE requires that the two applications sharing data are running concurrently. In practice, this would mean that the server application (Access) should be started, the database to be shared be open and then for APL+Win to establish communication.

It is preferable that APL+Win itself starts Access prior to establishing a conversation, thereby ensuring that Access is ready. Then, only the database name and location need be known.

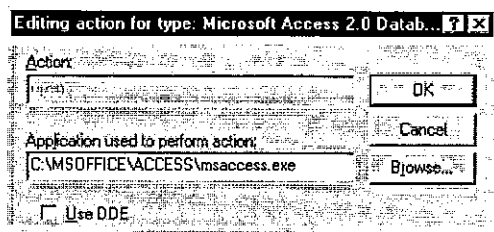
For this article, it is NWIND.MDB which is a sample database packaged with Access, found at `c:\msoffice\access\sampapps` on my computer.

For APL+Win to start Access, it must

- identify the exe which runs Access
- call the exe file and open NWIND.MDB

Identifying exe

If MSAccess is correctly installed (confirmed by dialogue below: within Explorer, click View, click Options, {locate Microsoft Access 2.0 Database}, click Edit, Click Edit...),



the Find Executable API call can be used to locate the exe file.

```
HINSTANCE FindExecutable(lpszFile, lpszDir, lpszResult)
```

```
LPCSTR lpszFile; /* address of string for filename*/
LPCSTR lpszDir; /* address of string for default directory*/
LPSTR lpszResult; /* address of string for executable file on return*/
```

The FindExecutable function finds and retrieves the executable filename that is associated with a specified filename.

Parameter	Description
lpszFile	Points to a null-terminated string specifying a filename. This can be a document or executable file.
lpszDir	Points to a null-terminated string specifying the drive letter and path for the default directory.
lpszResult	Points to a buffer that receives the name of an executable file when the function returns. This null-terminated string specifies the application that is started when the Open command is chosen from the File menu in File Manager.
Returns	The return value is greater than 32 if the function is successful. If the return value is less than or equal to 32, it specifies an error code.

In APL+Win this is :

```
v Z←Locate_Exe R
[1]  a Return name of EXE for file R
[2]  Z←1000p⌊tcnul
[3]  Z←⌊wcall 'FindExecutable' R '' Z
[4]  :if 32<⌊io>Z
[5]      Z←(⌊io+1)>Z
[6]      Z←(¯4+(4/1),^)\-(Z ⌊SS '.exe')vZ ⌊SS '.EXE')/Z
[7]  :else
```

```

[8]      Z←1
[9]      'Failed to locate exe for file ',R
[10]     ÷0
[11]     a Normally you'd use MessageBox API call for error trapping!
[12]     :endif
v

```

Locate_Exe 'C:\MSOFFICE\ACCESS\SAMPAPPS\NWIND.MDB'

returns

C:\MSOFFICE\ACCESS\msaccess.exe

Starting exe with Source File

Another Windows API, WinExec does this.

```
UINT WinExec(lpszCmdLine, fuCmdShow)
```

```
LPCSTR lpszCmdLine; /* address of command line*/
UINT fuCmdShow;     /* window state of new app.*/
```

The WinExec function runs the specified application.

Parameter	Description
lpszCmdLine	Points to a null-terminated Windows character string that contains the command line (filename plus optional parameters) for the application to be run. If the string does not contain a path, Windows searches the directories in this order:
Returns	The return value identifies the instance of the loaded module, if the function is successful. Otherwise, the return value is an error value less than 32.

In APL+Win, this is:

```

v Z←Start_Exe R
[1]  a Open file R in a hidden window
[2]  Z←□wcall 'WinExec' ((Locate_Exe R),' ',R) 'SW_HIDE'
[3]  :if 32>Z
[4]      'Error Opening ',R
[5]      ÷0
[6]  :endif
v

```

The shwCmd SW_HIDE can be substituted by SW_SHOWMAXIMIZED if you want to see Access running. Refer to the WINDOWPLACEMENT API call for documentation of other shwCmd options.

Thus,

```
Start_Exec 'c:\msoffice\access\sampapps\nwind.mdb'
```

will start Access and open NWIND.MDB.

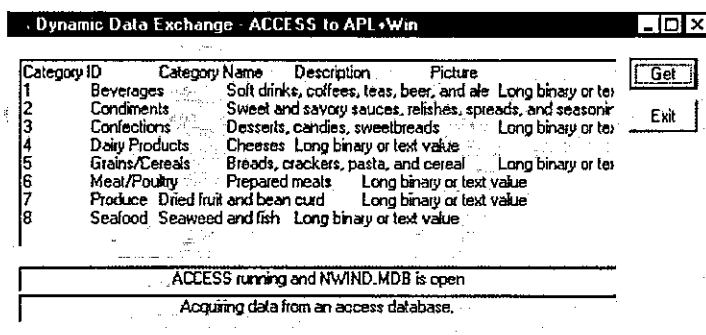
This approach will enable APL to start any exe application, preferably from a menu. For example,

```
Start_Exec 'c:\myfile.htm'
```

will launch the default internet browser (if it exists) and open the file c:\myfile.htm.

Getting Table *Categories* from NWIND.MDB

Run the function Access_DDE and click the Get button for the following to appear:



The function Access_DDE is :

```

v Access_DDE;⎕wself
[1]
[2] 'Access' ⎕wi 'Delete'
[3]
[4] ⎕wself+'Access' ⎕wi 'New' 'Form' 'Close'
[5] ⎕wi 'caption' 'Dynamic Data Exchange - Access to APL+Win'
[6] ⎕wi 'extent' 13.125 62.25
[7] ⎕wi 'where' 9.5 11.375
[8]
[9] ⎕wself+'Access.bn1' ⎕wi 'New' 'Button'
[10] ⎕wi 'caption' 'Get'
[11] ⎕wi 'where' 0.875 55 1.375 6

```

```

[12]      []wi 'onClick' ''Access.ed1' []wi 'DdeRequest''
[13]
[14]      []wself+'Access.l1' []wi 'New' 'Label'
[15]      []wi 'caption' 'Access running and NWIND.MDB is open'
[16]      []wi 'style' 1
[17]      []wi 'edge' 1
[18]      []wi 'where' 10.125 1 1.25 53
[19]
[20]      []wself+'Access.ed1' []wi 'New' 'Edit'
[21]      []wi 'style' 4100
[22]      []wi 'ddeMode' 'cold'
[23]      []wi 'ddeTopic' 'MSACCESS\NWIND;TABLE Categories'
[24]      []wi 'ddeItem' 'All'
[25]      []wi 'where' 0.875 1 8.5 53
[26]      []wi 'onChange' 'APLvar+'Access.ed1' []wi 'text''
[27]
[28]      []wself+'Access.bn3' []wi 'New' 'Button'
[29]      []wi 'caption' 'Exit'
[30]      []wi 'where' 2.75 55 1.375 6
[31]      []wi 'onClick' ''Access'' []wi 'Delete''
[32]
[33]      []wself+'Access.l2' []wi 'New' 'Label'
[34]      []wi 'caption' 'Acquiring data from an access database.'
[35]      []wi 'style' 1
[36]      []wi 'edge' 1
[37]      []wi 'where' 11.375 1 1.375 53
[38]
[39]      'Access' []wi 'Wait'
v

```

It is assumed that:

- you understand APL+Win and therefore Access_DDE
- you can write Access_DDE for your APL interpreter, if different

However, the DDE properties deserve further explanation.

```

[]wi 'ddeMode' 'cold'

```

The ddeMode property has three possible values:

Value	Meaning
'hot'	requests an automatic link, and the server application will send the requested data ddeItem as soon as it changes. Nothing is sent until the data is changed.

'cold' 'cold' specifies a manual link. The server sends a data item when the client application requests it by using the DdeRequest method

" severs the client server link.

Access_DDE specifies a *cold* link, that is, will use NWIND.MDB on a *read only* basis.

```
□wi 'ddeTopic' 'MSACCESS\NWIND;TABLE Categories'
```

The ddeTopic property defines the topic of a DDE conversation with another Windows application. It is used on both client (APL+Win) and server (Access) sides of a conversation.

The options (the phrase following NWIND, the database file name) are :

Value	Meaning
nothing	specialist, see ddelItem below
TABLE tablename	will send/receive the tablename specified
QUERY queryname	will send/receive the queryname specified
SQL sql string	a valid SQL string up to 255 characters

```
□wi 'ddeItem' 'All'
```

When ddeTOPIC omits a specification including and beyond ; (see below) , the possible values for ddelItem are :

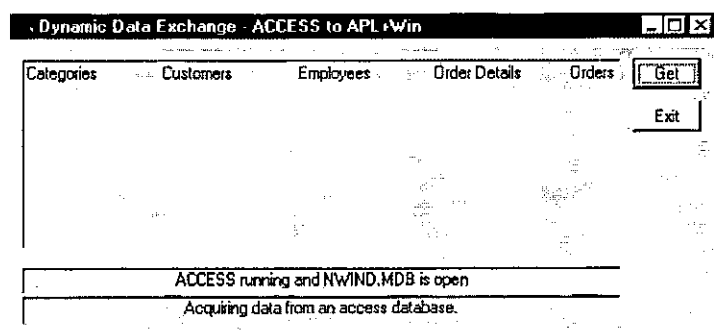
Value	Meaning
TableList	returns list of tables in database
QueryList	returns list of queries in database
FormList	returns list of forms in database

ReportList	returns list of reports in database
MacroList	returns list of macros in database
ModuleList	returns list of modules in database

That is, the structure of the data base can be queried. For example,

```
□wi 'ddeTopic' 'MSACCESS\NWIND'  
□wi 'ddeItem' 'TableList'
```

will return



Note that Access table names may contain imbedded spaces in their names and therefore they are unsuitable as APL+Win variable names. A good strategy is to turn space into underscore for workspace manipulation of those names.

When ddeTOPIC is not omitted, the possible values for ddeItem are :

Value	Meaning
All	return all data, including field names
Data	return all data, excluding field names
FieldNames	return field names
FieldNames;T	return field names and Access field types

NextRow	data in next row
PrevRow	data in previous row
FirstRow	data in the first row
LastRow	data in the first row
FieldCount	the number of fields
SQL Text	SQL statement representing a table or query
SQL Text;n	n allows SQL statement to be broken down in n character long chunks.

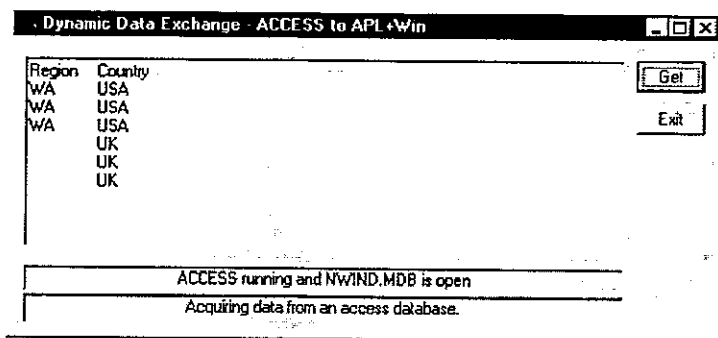
An example:

```

[]wi 'ddeTopic' 'MSACCESS\NWIND;SQL SELECT Region,Country FROM
Employees WHERE Title="Sales Representative"'
[]wi 'ddeItem' 'All'

```

Note, only two fields are being selected, namely *Region* and *Country*. Specify * for all fields.



```

[]wi 'onChange' 'APLvar+''Access.ed1'' []wi ''text''

```

On APL+Win receiving data from Access, the variable APLvar is updated in the workspace.

```

pAPLvar
47

```

```

]DISPLAY APLvar
-----
Region Country
WA USA
WA USA
WA USA
UK
UK
UK|
-----

```

APLvar is a simple text vector, delimited with tabs and newlines.

Inherent Limitations

As APLvar comes from an Edit box,

- it is restricted to 64K in length (can be overcome when ddelItem 'NextRow' is used, in most cases)
- it is text

It requires further manipulation in APL. This requires some fluency in nested APL.

First,

```

APLvar2←(APLvar*⎕tcn1)⊖APLvar

ρAPLvar2
7
]display APLvar2

```

```

-----
|Region Country|WA USA|WA USA|WA USA|UK|UK|UK|
-----

```

Second,

```
APLvar3+> (<|tcht="APLvar2")<"APLvar2
          ρAPLvar3
7 2
display APLvar3
```

Region	Country
WA	USA
WA	USA
WA	USA
UK	
UK	
UK	

Thus,

```
APLvar3[;⍋io] gives the variable Region
Region WA WA WA UK UK UK
      =APLvar3[;⍋io]
2
```

For record by record indexing,

```
▽ ByRecord R
[1]
[2]   :for i :in 1+⍋1+ρR
[3]     (Region Country)+APLvar3[i;]
[4]     a 'Your Code For Processing Records Goes Here'
[5]   :endfor
▽
```

The first index is dropped because ddelitem was All: this would not be necessary if ddelitem were specified as Data.

Third, since the original variable APLvar is read as text (there is no other way), the values need to be converted to the appropriate types. APL+Win has `⎕vi` and `⎕fi` for this purpose.

Conclusion

Having received the topic from Access, the link must be severed:

```
'Access.ed1' ⎕wi 'ddeItem' ''
```

and you need to close Access.

In this article, Access acts as a server to APL+Win. Another contribution can explore Access as a client to APL?

References:

1. Microsoft Windows 3.1 Resource Kit
 2. Microsoft Access 2.0 On Line Help
 3. APL+Win On Line Help
-

From Fibonacci to Horadam

by Joseph De Kerf

Abstract

Details are given about the generalization of the Fibonacci sequence as investigated by A. F. Horadam. Defined functions generating these so-called Horadam sequences, respectively based on implicit recursion, programming of a recursive loop, and explicit calculation from the auxiliary equation are given. Benchmarks demonstrate that explicit calculation is much more efficient in CPU time than the calculation based on recursion.

Fibonacci Sequences

The Fibonacci sequence is a special case of the sequence [1]

$$T_1 = p$$

$$T_2 = q$$

...

$$T_n = T_{n-2} + T_{n-1} \quad (\text{for } n > 2)$$

the domain of p and q being the set of integers. For $p=1$ and $q=1$, the sequence is called the Fibonacci sequence F_n . For $p=1$ and $q=3$, the sequence is known in the literature as the Lucas sequence L_n . It may be shown that:

$$T_n = pF_{n-2} + qF_{n-1}$$

The sequences may be generated by application of implicit recursion or by programming a recursive loop. J. Sullivan [2] has shown that, as far as the Fibonacci sequence is concerned, explicit calculation based on the golden section formula of Binet, is much more efficient in CPU time. J. De Kerf [3] has indicated how, apart from some overhead, by calculating only the first term of the golden section formula and rounding off, the efficiency may be speeded up by an extra factor of 2:

$$F_n = \frac{1}{\sqrt{5}} \left(\frac{1 + \sqrt{5}}{2} \right)^n \quad (\text{rounded off})$$

$$F = \lfloor 0.5 + (\div R) * (0.5 * 1 + R + 5 * 0.5) * 1 N$$

APL defined functions based on implicit recursion (*FLT1*), programming of a recursive loop (*FLT2*), and explicit calculation from F_{n-2} and F_{n-1} as described above (*FLT3*), are given below:

```
[0]  Z←P FLT1 N
[1]  →(N≤2)/0,Z←(2⌊N)ρP
[2]  Z←P FLT1 N-1
[3]  Z←Z,+ /~2↑Z
```

```
[0]  Z←P FLT2 N
[1]  →(N≤2)/0,Z←(2⌊N)ρP
[2]  END:→(N>ρZ←Z,+ /~2↑Z)/END
```

```
[0]  Z←P FLT3 N;F;R
[1]  →(N≤2)/0,Z←(2⌊N)ρP
[2]  F←⌊0.5+(÷R)×(0.5×1+R←5×0.5)×1N-1
[3]  Z←Z,(P[1]×~1+F)+P[2]×1+F
```

The left argument P is a two-element vector containing the parameters p and q . The right argument N is the non-negative integer scalar or one-element vector specifying the number of terms of the sequence required. The explicit result Z is a vector containing those terms. If N is 0, the empty numeric vector is returned. If N is a negative integer or not an integer, an error report is generated.

Examples:

```
1 3 FLT1 10
1 3 4 7 11 18 29 47 76 123
1 3 FLT2 10
1 3 4 7 11 18 29 47 76 123
1 3 FLT3 10
1 3 4 7 11 18 29 47 76 123
```

```
3 1 FLT1 10
3 1 4 5 9 14 23 37 60 97
3 1 FLT2 10
3 1 4 5 9 14 23 37 60 97
3 1 FLT3 10
3 1 4 5 9 14 23 37 60 97
```

the first of the two examples given being the Lucas sequence.

Horadam Sequences

A much wider generalization of the Fibonacci sequence has been thoroughly discussed by A.F. Horadam [4]:

$$H_1 = p$$

$$H_2 = q$$

...

$$H_n = rH_{n-2} + sH_{n-1} \quad (\text{for } n > 2)$$

Originally, the domain of the parameters $p q r s$ was restricted to integers. In most current publications however, the domain is extended to the set of real numbers. Specific sequences from the literature are:

Sequence	p	q	r	s
Fibonacci (F_n)	1	1	1	1
Lucas (L_n)	1	3	1	1
Pell (P_n)	1	2	1	2
Pell-Lucas (Q_n)	1	3	1	2

such that: $F_n = 1 \ 1 \ 2 \ 3 \ 5 \ 8 \ 13 \ 21 \ 34 \ 55$
 $L_n = 1 \ 3 \ 4 \ 7 \ 11 \ 18 \ 29 \ 47 \ 76 \ 123$
 $P_n = 1 \ 2 \ 5 \ 12 \ 29 \ 70 \ 169 \ 408 \ 985 \ 2378$
 $Q_n = 1 \ 3 \ 7 \ 17 \ 41 \ 99 \ 239 \ 577 \ 1393 \ 3363$

To calculate the sequence explicitly we may solve the auxiliary equation $h^2 - sh - r = 0$, the solution depending on the signum of the discriminant $s^2 + 4r$ [5]:

$$(a) \quad s^2 + 4r > 0: \quad H_n = ca^{n-1} + db^{n-1} \quad \text{with} \quad \Delta = \sqrt{s^2 + 4r}$$

$$a = (s + \Delta) / 2$$

$$b = (s - \Delta) / 2$$

$$c = (q - pb) / \Delta$$

$$d = (pa - q) / \Delta$$

$$(b) \quad s^2 + 4r = 0: \quad H_n = (c + d)(s / 2)^{n-2} \quad \text{with} \quad c = ps - q$$

$$d = n(2q - ps) / 2$$

$$(c) \quad s^2 + 4r < 0: \quad H_n = (c + d)\rho^{n-1} \text{ with } \Delta = \sqrt{|s^2 + 4r|}$$

$$\rho = \pm\sqrt{-r} \quad (+ \text{ for } s \geq 0 \text{ and } - \text{ for } s < 0)$$

$$\theta = \arcsin(\Delta / 2\rho)$$

$$c = -(2p\rho \cdot \sin(n-2)\theta) / \Delta$$

$$d = (2q \cdot \sin(n-1)\theta) / \Delta$$

Defined Functions

APL defined functions for calculating the sequence $H_1 \dots H_n$, based on implicit recursion (*HOR1*), programming of a recursive loop (*HOR2*), and explicit calculation from the auxiliary equation (*HOR3*) are given below:

```
[0]  Z←P HOR1 N
[1]  →(N≤2)/0,Z←(2[N])ρP
[2]  Z←P HOR1 N-1
[3]  Z←Z,+/ (2+P)×~2+Z

[0]  Z←P HOR2 N
[1]  →(N≤2)/0,Z←(2[N])ρP
[2]  END:→(N>ρZ+Z,+/ (2+P)×~2+Z)/END

[0]  Z←P HOR3 N;A;B;C;D;R;T;Δ
[1]  →(N≤2)/0,Z←(2[N])ρP
[2]  →((P[4]*2)~4×P[3])/LABDP
[3]  →((P[4]*2)<~4×P[3])/LABDN
[4]  C←(P[1]×P[4])~P[2]
[5]  D←(P[2]~P[1]×P[4]÷2)×N+2+~N
[6]  →0,Z←Z,(C~-D)×(C+D)×(P[4]÷2)×N-2
[7]  LABDP:Δ←((P[4]*2)+4×P[3])×0.5
[8]  B←P[4]~A×0.5×P[4]+Δ
[9]  C←(P[2]×P[1]×B)×(P[2]~P[1]×B)÷Δ
[10] D←(P[2]×P[1]×A)×((P[1]×A)~P[2])÷Δ
[11] →0,Z←Z,(C×A×N)+D×B×N+1+~N-1
[12] LABDN:Δ←((P[4]*2)+4×P[3])×0.5
[13] R←((~P[3])×0.5)×~1×P[4]<0
[14] T←~1○Δ÷2×R
[15] C←~(2×P[1]×R×1○T×N+~N-2)÷Δ
[16] D←(2×P[2]×1○T×N+1)÷Δ
[17] Z←Z,(C~-D)×(C+D)×R×N+1
```

The left argument P is a four-element vector containing the parameters pqr s. The right argument N is the non-negative integer scalar or one-element vector specifying the number of terms of the sequence requested. The explicit result is a vector containing those terms. If N is 0, the empty numeric vector is returned. If N

is a negative integer or not an integer, an error report is generated. Results are not rounded off, as the parameters may be any reals. Examples:

```

      1 2 1 2 HOR1 10
1 2 5 12 29 70 169 408 985 2378
      1 2 1 2 HOR2 10
1 2 5 12 29 70 169 408 985 2378
      1 2 1 2 HOR3 10
1 2 5 12 29 70 169 408 985 2378

      1 3 1 2 HOR1 10
1 3 7 17 41 99 239 577 1393 3363
      1 3 1 2 HOR2 10
1 3 7 17 41 99 239 577 1393 3363
      1 3 1 2 HOR3 10
1 3 7 17 41 99 239 577 1393 3363

```

The two examples given being the Pell sequence and the Pell-Lucas sequence respectively.

With *HOR3* some pitfalls are cured. If $s^2 + 4r$ is 0, the calculated value $(P[4]*2)+4*P[3]$ may be slightly different from 0. If compared with 0, the program would jump to *LABDP* if the approximation is positive and to *LABDN* if the approximation is negative. To avoid this, in lines 2 and 3 the values of $P[4]*2$ and $-4*P[3]$ are compared. In addition, depending on the signum of $s^2 + 4r$:

(a) $s^2 + 4r > 0$. If $c = 0$ and $|b| < 1$ or $d = 0$ and $|a| < 1$, the sequence H_n converges to 0. If $c = 0$ or $d = 0$ however, their calculated value may be slightly different from 0, such that for $|a| > 1$ and $|b| > 1$ respectively, the calculated sequence H_n converges to $\pm \infty$. Correction is done in line 9 by setting $C=0$ when $P[2]=P[1]*B$ and in line 10 by setting $D=0$ when $P[2]=P[1]*A$.

Examples:

```

p=1.24 q=-0.558 r=0.81 s=1.35
(c=0 a=1.80 b=-0.45)
p=1.24 q=0.558 r=0.81 s=-1.35
(d=0 a=0.45 b=-1.80)

```

(b) $s^2 + 4r = 0$. If n is a positive integer such that $c + d = 0$, then $H_n = 0$ for that value of n . The calculated value of $c + d$ however, may be slightly different from 0, and if $|s/2| > 1$ the value of H_n returned may be considerably different from 0. Correction is done in line 6 by setting the corresponding value of the explicit result Z to 0 when $C = -D$.

Examples:

$p=0.6 \quad q=0.98 \quad r=-2.7225 \quad s=3.3$
 $p=-0.6 \quad q=0.98 \quad r=-2.7225 \quad s=-3.3$
 ($|s/2|=1.65$ and $c+d=0$ for $n=100$)

(c) $s^2 + 4r < 0$. If for specific values of n the sum $c + d = 0$, then $H_n = 0$ for those values of n . The calculated value of $c + d$ however, may be slightly different from 0, and, if $|\rho| > 1$, the value of H_n returned may be considerably different from 0, the error increasing in absolute value with n . Correction is done in line 17 by setting the corresponding value of Z to 0 when $C = -D$.

Examples:

$p=2 \quad q=3 \quad r=-3 \quad s=3$
 $p=2 \quad q=-3 \quad r=-3 \quad s=-3$
 ($|\rho| = \sqrt{3}$ and $c+d=0$ for $n=4(6) \dots$)

Finally, if the value of H_n is only requested for one specific value of n , when applying recursion, the entire sequence $H_1 \dots H_n$ must be calculated. When applying explicit calculation from the auxiliary equation however, H_n must only be calculated for that specific value of n . A modified version *HOR4* of the defined function *HOR3*, that only calculates H_n for a given non-negative scalar or one-element vector is given below:

```
[0]  Z←P HOR4 N;A;B;C;D;R;T;Δ
[1]  →(N≤2)/0,Z+P[2]N
[2]  →((P[4]*2)>~4*P[3])/LABDP
[3]  →((P[4]*2)<~4*P[3])/LABDN
[4]  C←(P[1]*P[4])-P[2]
[5]  D←(P[2]-P[1]*P[4]÷2)*N
[6]  →0,Z+(C≠-D)*(C+D)*(P[4]÷2)*N-2
[7]  LABDP:Δ←((P[4]*2)+4*P[3])*0.5
[8]  B←P[4]-A+0.5*P[4]+Δ
[9]  C←(P[2]*P[1]*B)*(P[2]-P[1]*B)÷Δ
[10] D←(P[2]*P[1]*A)*((P[1]*A)-P[2])÷Δ
[11] →0,Z+(C*A*N-1)+D*B*N-1
[12] LABDN:Δ←(|(P[4]*2)+4*P[3])*0.5
[13] R←((-P[3])*0.5)*~1*P[4]<0
[14] T←~10Δ÷2*R
[15] C←-(2*P[1]*R*10T*N-2)÷Δ
[16] D←(2*P[2]*10T*N-1)÷Δ
[17] Z←(C≠-D)*(C+D)*R*N-1
```

Note: Rather unknown is the fact that applying recursion, as done in the defined functions *HOR1* and *HOR2*, also gives erroneous results for the special cases

described as pitfalls above. Results for those special cases are compared with the exact solutions as returned by *HOR3* in the accompanying Appendix. It would be a challenge to incorporate the corrections needed such as is done in the defined functions *HOR3* and *HOR4* (if possible!).

Benchmarks

Benchmarks have been done for $N = 250(250)1000$, in order to compare the effectiveness in CPU times for the functions defined. Results are given below (in ms):

N	250	500	750	1000
<i>HOR1</i>	98.6	217.6	359.2	524.7
<i>HOR2</i>	64.6	145.2	248.0	366.8
<i>HOR3</i> ($s^2 + 4r > 0$)	4.12	6.61	9.26	12.12
<i>HOR3</i> ($s^2 + 4r = 0$)	3.77	5.77	7.82	10.18
<i>HOR3</i> ($s^2 + 4r < 0$)	7.60	12.91	18.55	24.16
<i>HOR4</i> ($s^2 + 4r > 0$)	1.99	1.99	1.99	1.99
<i>HOR4</i> ($s^2 + 4r = 0$)	1.72	1.72	1.72	1.72
<i>HOR4</i> ($s^2 + 4r < 0$)	2.14	2.14	2.14	2.14

Depending on N , the CPU times are from 34% to 30% lower for *HOR2* than for *HOR1*. Depending on N and the signum of $s^2 + 4r$, the efficiency factor in CPU time is from 9 to 36 for *HOR3* versus *HOR2*, and from 30 to 213 for *HOR4* versus *HOR2*. Of course, those figures depend on the hardware and software used.

Functions defined conform to the standard ISO APL [6]. Programming, debugging, illustrations and benchmarks have been done on a MicroLine Pentium S-100, with mathematical co-processor, using Dyalog APL/W Version 7.1.2, under Windows 3.11 [7]. Comparison Tolerance was set to $1E^{-13}$, the default value being $1E^{-14}$. Benchmarks have been done using the system function *QdMONITOR*.

References

- [1] V.E. Hoggatt: *Fibonacci and Lucas Numbers*. Houghton Mifflin Company, Boston, Mass. 1969
- [2] J. Sullivan: *Fast Fibbing*. Vector Vol.3 No.1, July 1986, pp 113-114

- [3] J. De Kerf: *More on "Fast Fibbing"*. Vector Vol.3 No.4, April 1987, p 120
- [4] A.F. Horadam: *Basic Properties of a Certain Generalized Sequence of Numbers*. The Fibonacci Quarterly, Vol.3 No.3, August 1965, pp 161-176
- [5] A. Brousseau: *Linear Recursion and Fibonacci Sequences*. The Fibonacci Association, San Jose State College, San Jose, California, 1971
- [6] International Standard ISO 8485: 1989 (E), First Edition; 1989-11-01; *Programming Languages - APL*. International Organization for Standardization ISO, Geneva, Switzerland, 1989
- [7] *Dyalog APL/W Reference Manual - Version 7*. Dyadic Systems Ltd., Basingstoke, 1994

Appendix

Results for the special cases described in the text, returned by *HOR1* and *HOR2*, are compared with the exact solutions returned by *HOR3*. Results shown are those for Dyalog APL/W [7].

(a) $s^2 + 4r > 0$

```
P←1.24 0.558 0.81 1.35
R1←-5+P HOR1 502
R2←-5+P HOR2 502
R3←-5+P HOR3 502
(⌈5+⌈502⌋,R1,R2,[1.5]R3
498 -2.359343255E110 -2.359343255E110 -5.495927961E-173
499 -4.246817860E110 -4.246817860E110 2.473167583E-173
500 -7.644272148E110 -7.644272148E110 1.112925412E-173
501 -1.3759688987E111 -1.3759688987E111 5.008164355E-174
502 -2.476744176E111 -2.476744176E111 -2.253673960E-174

P←1.24 0.558 0.81 -1.35
R1←-5+P HOR1 502
R2←-5+P HOR2 502
R3←-5+P HOR3 502
(⌈5+⌈502⌋,R1,R2,[1.5]R3
498 2.359343255E110 2.359343255E110 5.495927961E-173
499 -4.246817860E110 -4.246817860E110 2.473167583E-173
500 7.644272148E110 7.644272148E110 1.112925412E-173
501 1.3759688987E111 1.3759688987E111 5.008164355E-174
502 2.476744176E111 2.476744176E111 2.253673960E-174
```

Results returned by *HOR1* and *HOR2* converge to $\pm \infty$ and are erroneous. Results returned by *HOR3* converge to 0 and are correct.

(b) $s^2 + 4r = 0$

```
P←0.6 0.98 -2.7225 3.3
R1←-5+P HOR1 102
R2←-5+P HOR2 102
R3←-5+P HOR3 102
(⌈5+⌈102⌋,R1,R2,[1.5]R3
```

```

98 1.511780155E19 1.511780155E19 1.511780155E19
99 1.247218628E19 1.247218628E19 1.247218628E19
100 -1.005223936E9 -1.005223936E9 0.000000000E0
101 -3.395552715E19 -3.395552715E19 -3.395552715E19
102 -1.120532396E20 -1.120532396E20 -1.120532396E20

```

```

P+0.6 0.98 -2.7225 -3.3
R1+5+P HOR1 102
R2+5+P HOR2 102
R3+5+P HOR3 102
(5+102),R1,R2,[1.5]R3

```

```

98 1.511780155E19 1.511780155E19 1.511780155E19
99 1.247218628E19 1.247218628E19 1.247218628E19
100 -1.005223936E9 -1.005223936E9 0.000000000E0
101 3.395552715E19 3.395552715E19 3.395552715E19
102 -1.120532396E20 -1.120532396E20 -1.120532396E20

```

Results returned by *HOR1* and *HOR2* for $N = 100$ are considerably different from 0 and are erroneous. Results returned by *HOR3* for $N = 100$ are 0 and are correct.

(c) $s^2 + 4r < 0$

```

P+2 3 -3 3
R1+5+P HOR1 252
R2+5+P HOR2 252
R3+5+P HOR3 252
(5+1252),R1,R2,[1.5]R3

```

```

248 -1.455578343E59 -1.455578343E59 -1.455578343E59
249 -1.455578343E59 -1.455578343E59 -1.455578343E59
250 -3.568119232E44 -3.568119232E44 0.000000000E0
251 4.366735029E59 4.366735029E59 4.366735029E59
252 1.310020509E60 1.310020509E60 1.310020509E60

```

```

P+2 -3 -3 -3
R1+5+P HOR1 252
R2+5+P HOR2 252
R3+5+P HOR3 252
(5+1252),R1,R2,[1.5]R3

```

```

248 1.455578343E59 1.455578343E59 1.455578343E59
249 -1.455578343E59 -1.455578343E59 -1.455578343E59
250 3.568119232E44 3.568119232E44 0.000000000E0
251 4.366735029E59 4.366735029E59 4.366735029E59
252 -1.310020509E60 -1.310020509E60 -1.310020509E60

```

Results returned by *HOR1* and *HOR2* for $N = 250$ are totally different from 0 and are erroneous. Results returned by *HOR3* for $N = 250$ are 0 and are correct.

Joseph De Kerf
 Rooienberg 72
 B-2570 Duffel
 Belgium

Index to Advertisers

Dyadic Systems Ltd	6
Strand Software	2
APL Booklist	48
Russian Fund	76
Vector Back Numbers	7

All queries regarding advertising in VECTOR should be made to Gill Smith, at 01439-788385, Email: apl385@compuserve.com.

Submitting Material to Vector

The Vector working group meets towards the end of the month in which Vector appears; we review material for issue n+1 and discuss themes for issues n+2 onwards. Please send the text of submitted articles (hardcopy with diskette as appropriate) to the Vector Working Group via:

Vector Administration, c/o Gill Smith
Brook House
Gilling East
YORK, YO62 4JJ
Tel: +44 (0) 1439-788385
Email: apl385@compuserve.com

Authors wishing to use Word for Windows should contact Vector Production for a copy of the APL2741 TrueType font, and a suitable Winword template. These may also be downloaded from the Vector web site at www.vector.org.uk

Camera-ready artwork (e.g. advertisements) and diskettes of 'standard' material (e.g. sustaining members' news) should be sent to Vector Production, Brook House, Gilling East, YORK YO62 4JJ. Please also copy us with all electronically submitted material so that we have early warning of possible problems.

British APL Association: Membership Form

Membership is open to anyone interested in APL. The membership year normally runs from 1st May to 30th April, but new members may join from 1st August, November or February if preferred. The British APL Association is a special interest group of the British Computer Society, Reg. Charity No. 292,786

Name: _____

Address: _____

Postcode / Country: _____

Telephone Number: _____

Email Address: _____

Category (please tick box) to run from: 1st May ☐ August ☐ Nov ☐ Feb ☐

UK private membership £12 ☐

Overseas private membership £14 ☐

Airmail supplement (not needed for Europe) £4 ☐

UK Corporate membership £100 ☐

Corporate membership overseas £135 ☐

Sustaining membership £430 ☐

Non-voting UK member (student/OAP/unemployed only) £6 ☐

PAYMENT — in Sterling or by Visa/Mastercard/JCB only

Payment should be enclosed with membership applications in the form of a UK Sterling cheque to "The British APL Association", or you may quote your Mastercard, Visa or JCB number.

I authorise you to debit my Visa/Mastercard/JCB account

Number: Expiry date: |

for the membership category indicated above,

- ☐ annually, at the prevailing rate, until further notice
☐ one year's subscription only

(please tick the required option above)

*Data Protection Act:
The information supplied may be
stored on computer and processed
in accordance with the registration
of the British Computer Society.*

Signature: _____ Send the completed form to:

British APL Association, c/o Rowena Small, 8 Cardigan Road, LONDON E3 5HU, UK

The British APL Association

The British APL Association is a Specialist Group of the British Computer Society. It is administered by a Committee of officers who are elected by a postal ballot of Association members prior to the Annual General Meeting. Working groups are also established in areas such as activity planning and journal production. Offers of assistance and involvement with any Association matters are welcomed and should be addressed in the first instance to the Secretary.

1998/99 Committee

Chairman	Anthony Camacho 0117-973 0036 100612.1057@compuserve.com	11 Auburn Road, Redland, BRISTOL, BS6 6LS
Secretary	Ajay Askoolum 01737-771643 106173.3347@compuserve.com	42 Hanworth Road, Redhill, Surrey RH1 5HT
Treasurer	Nicholas Small 0181-980 7870 treas.apl@bcs.org.uk	8 Cardigan Road, LONDON E3 5HU
Journal Editor	Stefano Lanzavecchia stf@adaytum.dk	c/o Vector Administration, Brook House, Gilling East YORK YO62 4JJ
Projects and Publicity	Dr Alan Mayer 01792-205678x4274 a.d.mayer@swansea.ac.uk	European Business Management School, Swansea University, Singleton Park, SWANSEA SA2 8PP
Webmaster	Ray Cannon 01252-874697 100430.740@compuserve.com	21 Woodbridge Road, Blackwater, Camberley, Surrey GU17 0BS
Activities	Jon Sandles 01904-612882 jon_sandles@csi.com	138 Burton Stone Lane, YORK YO30 6DF
Education	Dr Ian Clark 01388-605544 100021.3073@compuserve.com	Unit R21, Auckland New Business Centre, St. Helen Auckland, Bishop Auckland, Co. Durham DL14 9TX
Administration	Rowena Small 0181-980 7870 treas.apl@bcs.org.uk	8 Cardigan Road, LONDON E3 5HU

Journal Working Group

Editor:	Stefano Lanzavecchia	see above
Production:	Adrian & Gill Smith	Brook House, Gilling East, YORK (01439-788385)
Advertising:	Gill Smith	Brook House, Gilling East, YORK (01439-788385)
Support Team:	Jonathan Barman (01488-648575), Richard and Adam Weber (0121-3546550), Anthony & Sylvia Camacho, Ray Cannon (01252-874697), Marc Griffiths (01653-691745), Bob Hoekstra (01483-771028), Jon Sandles (01904-612882)	

Typeset by APL-385 with MS Word 5.0 and GoScript

Printed in England by Short-Run Press Ltd, Exeter

VECTOR

VECTOR is the quarterly Journal of the British APL Association and is distributed to Association members in the UK and overseas. The British APL Association is a Specialist Group of the British Computer Society. APL stands for 'A Programming Language' - an interactive computer language noted for its elegance, conciseness and fast development speed. It is supported on most mainframes, workstations and personal computers.

SUSTAINING MEMBERS

The Committee of the British APL Association wish to acknowledge the generous financial support of the following Association Sustaining Members. In many cases these organisations also provide manpower and administrative assistance to the Association at their own cost.

Causeway Graphical Systems Ltd
The Mallings, Castlegate,
MALTON, North Yorks YO17 7DP
Tel: 01653-696760
Fax: 01653-697719
Email: causeway@cs4.com
Web: www.causeway.co.uk

Compass Ltd
Compass House
60 Priestley Road
GUILDFORD, Surrey GU2 5YU
Tel: 01433-514500

Dyadic Systems Ltd
Riverside View, Basing Road
Old Basing, BASINGSTOKE,
Hants. RG24 0AL
Tel: 01256-811125
Fax: 01256-811130
Email: sales@dyadic.com
Web: www.dyadic.com

HMW Trading Systems Ltd
Hamilton House
1 Temple Avenue
LONDON EC4Y 0JA
Tel: 0171-858 8800
Fax: 0171-858 8825
Email: 1000202682@compuserve.com

Insight Systems ApS
Nordre Strandvej 119C
DK-8160 Hellebæk
Denmark
Tel: +45 49 76 20 20
Fax: +45 49 76 20 30
Email: info@insight.dk
Web: www.insight.dk

APL2000
Rapid Application Development
6510 Rockledge Drive
Suite 502
Bethesda MD 20817
USA
Email: sales@apl2000.com
Web: www.apl2000.com

Soliton Associates Ltd
Groet Blankenberg 53
1082 AC Amsterdam
Netherlands
Tel: +31 20 646 4475
Fax: +31 20 644 1205
Email: sales@soliton.com

Dutch APL Association
Postbus 1841
8430 ZH Nieuwegein
Netherlands
Tel: +31 847 342 837